

Program Analysis 2025-26

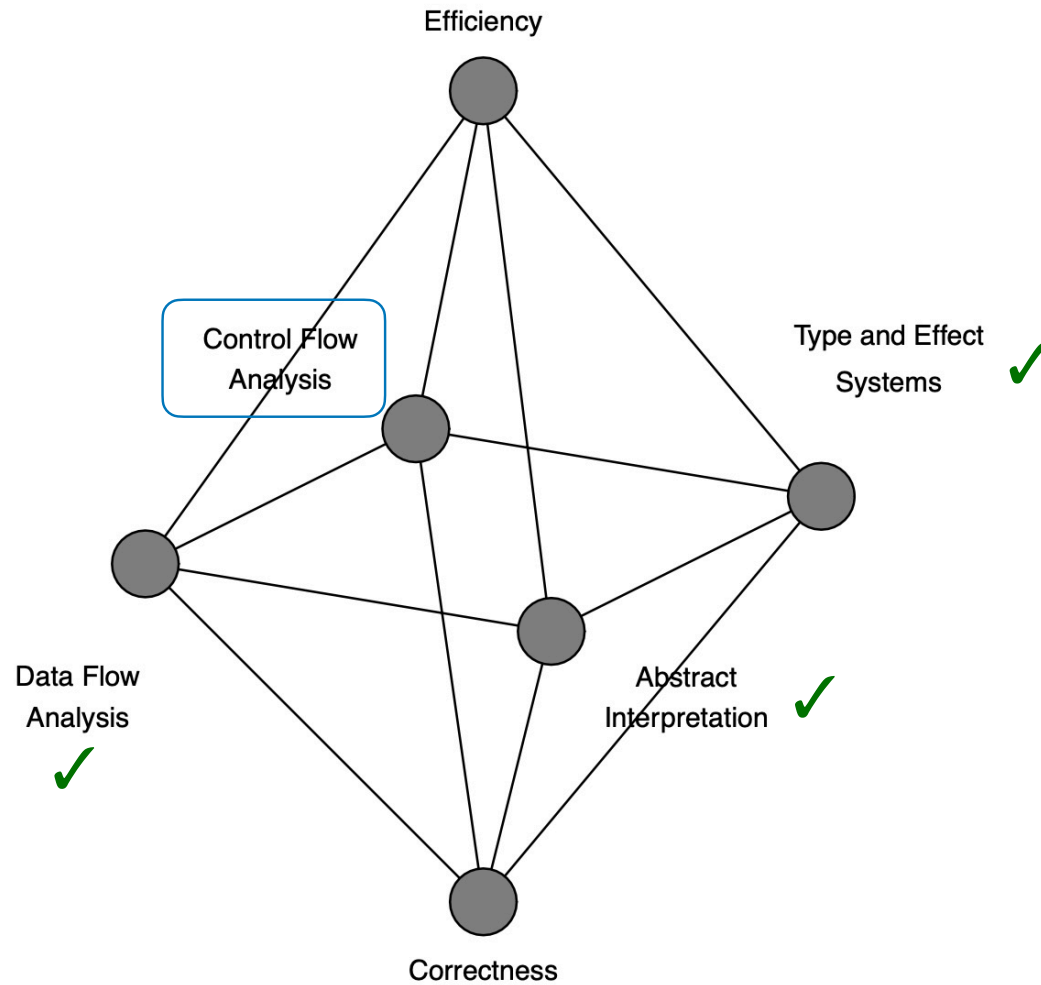
Lecture #17

Control Flow Analysis



UNIVERSITÀ
DI PISA

Program Analysis



Chasing the flow of values

Flow information is essential for the specification of Data Flow Analyses (e.g., Monotone Framework over Control Flow Graphs)

Simple imperative languages: flow information can be extracted directly from the program text: procedure calls are performed by explicitly mentioning the name of a procedure

But what if functions are values?

Now we have a problem: we often do not know at compile time which function is called

Control Flow Analysis (CFA) statically approximates which functions **may** be called at each call site

Control Flow Analysis

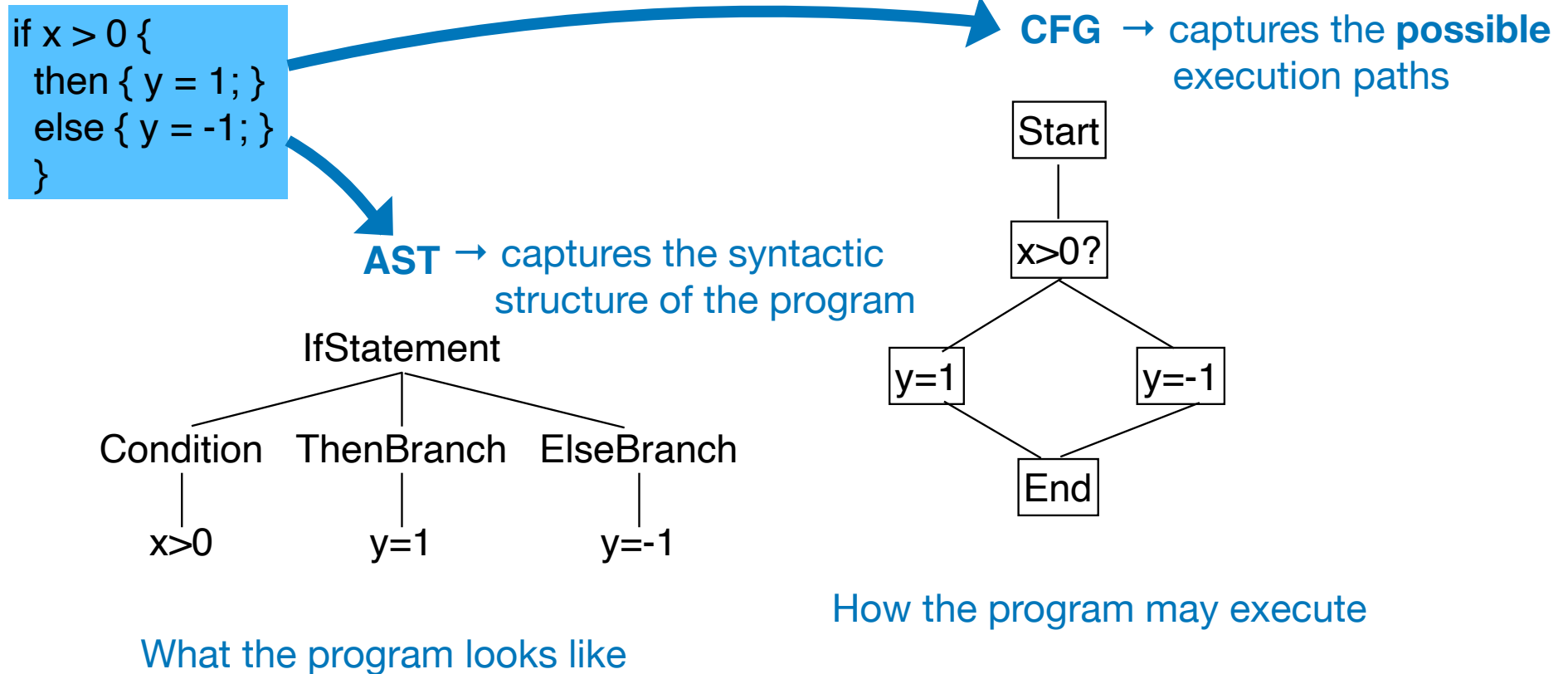
Control flow describes the rules underlying the evolution of the program

- implicit in the program code and in the **Abstract Syntax Tree (AST)**
- explicit in **Control Flow Graph (CFG)**, which captures the order of execution and possible paths (assuming the targets of calls are known)

The purpose of **Control Flow Analysis (CFA)** is to determine which blocks may lead to which other blocks

But in modern languages, even building the CFG is not trivial

AST and CFG



To build the CFG, we must know how control may flow at runtime

Analyzing CFGs

So far looked in previous
courses

- **intraprocedural** analysis: → analyze a single function
- **interprocedural** analysis: → analyze the whole program with function calls

Until now, all targets were known statically

Interprocedural analysis

- Construct a CFG for each function
- Glue them together via **function calls** and **returns**

Interprocedural analysis

Interprocedural analysis:

tractable when the program has a **simple structure**

→ calls have **statically known targets**

Simple imperative language with higher-order functions

Higher-order functions complicate CFG construction:

- several functions may be invoked at a call site
- this depends on the dataflow
- but dataflow analysis first requires a CFG

Simple imperative language with higher-order functions (cont.)

```
inc(i) { return i+1; }  
dec(j) { return j-1; }  
ide(k) { return k; }
```

What is f?

```
foo(n,f) {  
  var r;  
  if (n==0) { f=ide; }  
  r = f(n);  
  return r;  
}
```

```
main() {  
  var x,y;  
  x = input;  
  if (x>0) { y = foo(x,inc); } else { y = foo(x,dec); }  
  return y;  
}
```

Example by A. Møller & M. I. Schwartzbach

A **control flow analysis**
approximates the call graph

$\llbracket f \rrbracket = \{\text{inc}, \text{dec}, \text{ide}\}$

Functional languages

Control-flow (CFG) + data “flowing” through it

→ no longer works

Why?

- The boundary between control and data becomes unclear
- Everything is both data and control

Functions are higher-order: they can be at the same time:

- code to be executed and
- data, i.e., arguments or results of other function

No clear separation between control-flow and data-flow

When the program decides “at runtime” which function to call, the control flow is no longer explicit

Dynamic Dispatch Problem

Consider the following program fragment

```
let
  f = fn x => x 1;
  g = fn y => y + 2;
  h = fn z => z + 3
in
  (f g) + (f h)
```

variables can denote functions and
which procedure/function gets invoked
depends on runtime values

- What does this program do?
- For each function application, which functions may be applied?

- x is a function variable
- at runtime: $x \in \{g, h\}$

So call $x \ 1$ may target g or h (from calls $(f \ g)$ and $(f \ h)$)

Here it is clear which functions are actually called, but in general it is not

Dynamic Dispatch Problem (cont.)

Consider the following program fragment

```
let
  f = fn x => x 1;
  g = fn y => y + 2;
  h = fn z => z + 3
in
  (f g) + (f h)
```

We cannot know the exact target of a call

So we compute a **safe approximation**

Solution: static analysis

→ **over-approximate** the **possible targets** of each call

Control Flow Analysis (CFA):

→ For each function application (call site), compute a **safe approximation**

How? By solving a system of constraints, where constraints describe how values flow between variables

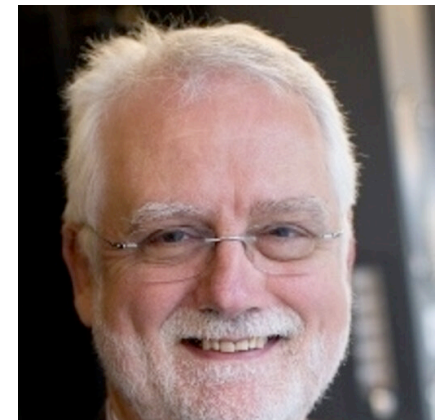
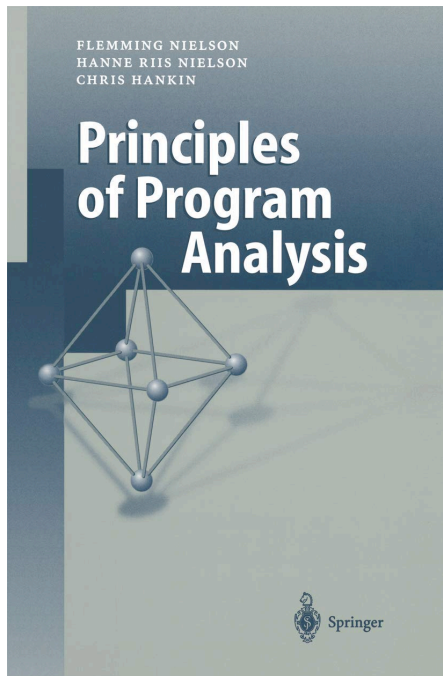
Control Flow Analysis

Control Flow Analysis: historically

- Reynolds: early analyses of LISP programs
 - Jones: first to study lambda expressions → introduced control-flow analysis
 - Shivers: formalized CFA for Scheme → basis for constraint-based approaches
 - ...
-
- Today: constraint-based CFA for higher-order programs

The Constraint Based Approach

We adopt the approach illustrated in *Principles of Program Analysis* by Flemming Nielson, Hanne Riis Nielson, Chris Hankin **[Chapter 3]**



FUN: syntax

term in **Term**

$t ::= c$ // constant in **Const**

| x // variable in **Var**

| $\text{fn } x \Rightarrow e_0$ // function definition (abstraction)

| $\text{fun } f \ x \Rightarrow e_0$ // recursive function definition

| $e_1 \ e_2$ // function application

| $\text{if } e_0 \text{ then } e_1 \text{ else } e_2$ // conditional

| $\text{let } x = e_1 \text{ in } e_2$ // local definition

| $e_1 \ \text{op} \ e_2$ // binary operation (op in **Op**)

expression
(labelled term)
in **Exp**

$e ::= t^\ell$

Label:
needed for the CFA
identify program
points (e.g., call sites)

Each label ℓ in **Lab** corresponds to a program point where CFA computes information

CFA of FUN

The goal

Associate for each variable x and each expression e , a **safe over-approximation** of the set of functions that they may evaluate to at runtime

valid

This determines the possible targets of function applications

Example of function application

```
( if e then ((*) 2) else ((+) 1) ) 10
```

- e evaluates to either $((*) 2)$ or $((+) 1)$
- the call may target $\{(*), (+)\}$

Example 1: labelling

Consider the following expression and label it

`((fn x => x )(fn y => y ))`

`((fn x => x1)2 (fn y => y3)4)5`

`(fn x => x1)2`

`(fn y => y3)4`

`x` `x1`

`y` `y3`

Example 1: what happens at runtime?

Consider the following expression

$$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$$

Let us look at what happens at runtime: $((\lambda x . x)(\lambda y . y)) \rightarrow x\{(\lambda y . y)/x\} \equiv (\lambda y . y)$

- x is bound to $(\text{fn } y \Rightarrow y^3)$
- the subexpression at label 1 evaluates to that function
- the overall expression at label 5 evaluates to that function

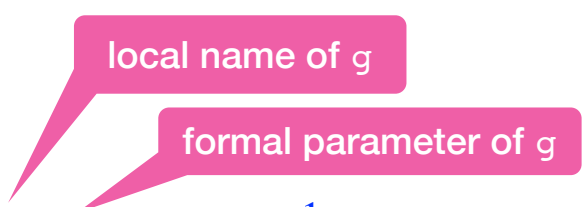
in λ -calculus

CFA will compute a **safe approximation** of these results

Example 2

Consider the following expression

$\text{loop} \triangleq (\text{let } g = (\text{fun } f \ x \Rightarrow (f^1 \ (\text{fn } y \Rightarrow y^2)^3)^4)^5$
 $\text{in } (g^6 \ (\text{fn } z \Rightarrow z^7)^8)^9)^{10}$



g is a **recursive function** applied to the identity function $(\text{fn } z \Rightarrow z^7)$ that

- ignores its actual parameter and
- calls itself recursively with the argument $(\text{fn } y \Rightarrow y^2)$
- so the program loops

$\Rightarrow$ possible recursive flow of function values:

CFA must **approximate this recursive behavior**

The Control Flow Analysis

CFA as a Constraint Based Analysis

- A 0-CFA is a **Constraint Based Analysis**
- it is defined via **inference rules that generate constraints** over the possible dataflow values for each variable or labeled location; those
- constraints are then solved

CFA with Flow Logic

Flow Logic is a declarative approach to program analysis that mainly separates the **specification** from the actual **computation**

- **specification**: describes when the analysis results are acceptable
- **computation**: solving the constraints derived by the specification

acceptability
relation

constraint
satisfaction
problem

CFA: overview

We define our CFA in several steps:

- **Specification:** define which sets of functions are acceptable (acceptability relation)
- **Syntax-directed rules:** refine the acceptability relation to be syntax directed
- **Constraint generation:** turn the specification into a finite set of constraints
- **Constraint solving:** compute the least solution of the constraints (the analysis result)

Abstract domains: informally

the simplest
form of CFA

The result of analyzing an expression e is a pair $(\hat{C}, \hat{\rho})$:

- $\hat{C}: \ell \mapsto$ set of function abstractions the subexpression labelled ℓ **may** evaluate to
- $\hat{\rho}: x \mapsto$ set of function abstractions, x **may** be bound to

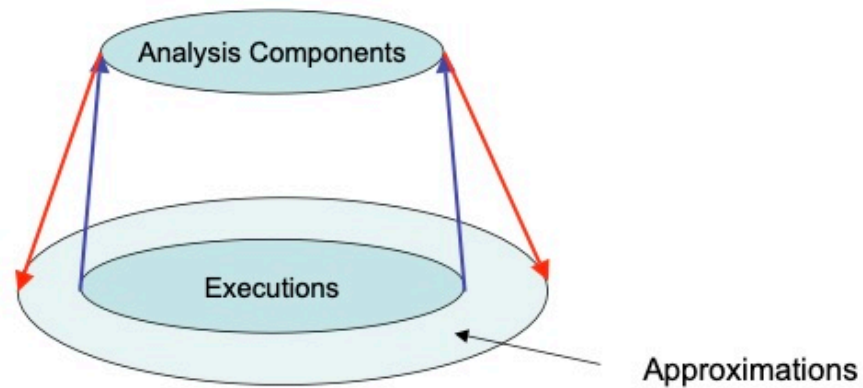
These are abstract values

Goal:

For each $(t_1^{\ell_1} t_2^{\ell_2})^{\ell}$ determine the possible functions for t_1 from the set of function abstractions t_2 **may** evaluate to

0-CFA means that **no context information** is considered, e.g., same function set at all call sites

Over-approximation



Static Event E is included	Dynamic E can happen
Event E is not included	E never happens

Abstract domains: formally

The result of a CFA analysis is a pair $(\hat{C}, \hat{\rho})$:

$$\hat{C} \in \widehat{\text{Cache}} : \text{Lab} \rightarrow \widehat{\text{Val}}$$

$$\hat{\rho} \in \widehat{\text{Env}} : \text{Var} \rightarrow \widehat{\text{Val}}$$

$$\hat{v} \in \widehat{\text{Val}} = \wp(\text{Term})$$

Only functions,
no constants
(no DFA component):
pure CFA

$\text{fn } x \Rightarrow e_0$ abstracts the
corresponding function
in all contexts

- An **abstract value** abstracts a **set of functions**:,
i.e., terms in the form $\text{fn } x \Rightarrow e_0$ or $\text{fun } f \ x \Rightarrow e_0$
- An **abstract cache** $\hat{C}$ associates abstract values
with each labelled program point
- An **abstract environment** $\hat{\rho}$ associates abstract
values with each variable

We assume that all variables and labels are distinct

Back to Example 1: make a guess

Consider the following 0-CFA guess

$$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$$

	$(\hat{C}_e, \hat{\rho}_e)$
$\hat{C}_e$	1 {fn y => y ³ }
	2 {fn x => x ¹ }
	3 $\emptyset$
	4 {fn y => y ³ }
	5 {fn y => y ³ }
$\hat{\rho}_e$	x {fn y => y ³ }
	y $\emptyset$

We want to know which functions **may** be bound to **variables** or **labels** (terms)

The variable y is never evaluated

Does $(\hat{C}_e, \hat{\rho}_e)$ safely over-approximate the runtime behavior? Is it an acceptable guess? 

Back to Example 1

Consider also the following 0-CFA guesses

$$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$$

	$(\hat{C}_e, \hat{\rho}_e)$	$(\hat{C}'_e, \hat{\rho}'_e)$	$(\hat{C}''_e, \hat{\rho}''_e)$
1	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
2	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
3	$\emptyset$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
4	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
x	$\{\text{fn } y \Rightarrow y^3\}$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
y	$\emptyset$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$





X

false
negative



Are they acceptable? Conservative (over-approximating) guesses are acceptable, incomplete ones are not

The specification of a solution

Given a guess $(\hat{C}, \hat{\rho})$, we validate whether it is an **acceptable** CFA

A guess is acceptable if it safely over-approximates the runtime behavior

For instance, if at runtime

- t^ℓ evaluates to a function, then this must be duly recorded in $\hat{C}(\ell)$
- x is bound to a function at runtime, this must also be duly recorded in $\hat{\rho}(x)$

Validation

check clauses on judgements

$$(\hat{C}, \hat{\rho}) \models e$$

relation

$$\models: \widehat{\mathbf{Cache}} \times \widehat{\mathbf{Env}} \times \mathbf{Exp} \rightarrow \{\text{true}, \text{false}\}$$

It means that the guess $(\hat{C}, \hat{\rho})$ is acceptable for expression e

Acceptability relation

$$(\hat{C}, \hat{\rho}) \models e$$

To define acceptability, we specify correctness rules for each syntactic category in FUN

Each rule enforces a correctness condition, i.e., it ensures that the guess includes all possible flows

We do not compute the solution here: we propose one and validate it using these clauses

Clause for constants

$$[con] (\hat{C}, \hat{\rho}) \models c^\ell \text{ always}$$

that can be reformulated as

$$[con] (\hat{C}, \hat{\rho}) \models c^\ell \text{ iff } \emptyset \subseteq \hat{C}(\ell)$$

No demands on $\hat{C}(\ell)$: constants do not evaluate to functions
In pure 0-CFA, we do **not** track data values

(no functions
can flow here)

Example

$$e \triangleq 7^1$$

$$\hat{C}(1) = \emptyset$$

Clause for variables

$$[var] (\hat{C}, \hat{\rho}) \models x^\ell \text{ iff } \hat{\rho}(x) \subseteq \hat{C}(\ell)$$

Everything the variable x can be evaluated to must be included in what is observed at program point ℓ

This propagates information from variables to program points: the flow goes from $\hat{\rho}$ to $\hat{C}$ (we get the value from the variable into the program)

Example

$$e \triangleq x^1$$

$$\hat{C}(1) \supseteq \hat{\rho}(x) \supseteq \{\text{fn } y \Rightarrow y^3\} \text{ [See example 1]}$$

Clause for functions

$$[fn] (\hat{C}, \hat{\rho}) \models (fn\ x \Rightarrow e_0)^\ell \text{ iff } \{fn\ x \Rightarrow e_0\} \subseteq \hat{C}(\ell)$$

functions are
abstract
values

The functional term $fn\ x \Rightarrow e_0$ must be included in $\hat{C}(\ell)$

This means that it may occur as part of a **closure** at program point ℓ during evaluation

No checks on the body e_0 : the clause for application will handle the body

Example

$$e \triangleq (fn\ x \Rightarrow x^1)^2$$

$$\hat{C}(2) \supseteq \{fn\ x \Rightarrow x^1\} \dots$$

Clause for recursive functions

$$[fun] (\hat{C}, \hat{\rho}) \models (\text{fun } f \ x \Rightarrow e_0)^\ell \text{ iff } \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{C}(\ell)$$

functions are
abstract
values

The functional term $\text{fn } x \Rightarrow e_0$ must be included in $\hat{C}(\ell)$

This means that it may occur as part of a **closure** at program point ℓ during evaluation

No checks on the body e_0 : the clause for application will handle the body

Example

$$e \triangleq (\text{fun } f \ x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4)^5$$
$$\hat{C}(5) \supseteq \{\text{fun } f \ x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4\} \dots$$

Clause for conditional

$$\begin{aligned} [if] \ (\hat{C}, \hat{\rho}) \models (\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell \text{ iff } & (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \\ & \wedge (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge \hat{C}(\ell_1) \subseteq \hat{C}(\ell) \\ & \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell) \end{aligned}$$

Clause for conditional

$$[if] (\hat{C}, \hat{\rho}) \models (\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge \hat{C}(\ell_1) \subseteq \hat{C}(\ell) \\ \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$$

This clause

- requires checking the sub-expressions:
hence it uses “recursive calls” $(\hat{C}, \hat{\rho}) \models t_i^{\ell_i}$
- propagates the values produced by sub-expression $\hat{C}(\ell_i)$ to the overall expression $\hat{C}(\ell)$

Condition is ignored:
over-approximation

This corresponds to merging the possible values from **both** branches

Example

$e \triangleq (\text{if } (x^1 > 0^2)^3 \text{ then } (\text{fn } y \Rightarrow y^4)^5 \text{ else } (\text{fn } z \Rightarrow z^6)^7)^8$

$\hat{C}(5) \subseteq \hat{C}(8)$ and $\hat{C}(7) \subseteq \hat{C}(8) \dots$

Clause for binary operation

$$[op] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} \text{ op } t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}$$

This clause simply

- requires checking the **sub-expressions**: hence it uses “recursive calls”

$$(\hat{C}, \hat{\rho}) \models t_i^{\ell_i}$$

No demands on $\hat{C}(\ell)$

In pure 0-CFA, we do **not** track data values

Example

$$e \triangleq (7^1 + 4^2)^3$$

$$\hat{C}(3) = \emptyset \dots$$

Clause for the let binding

$$[let] (\hat{C}, \hat{\rho}) \models (\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge \\ \hat{C}(\ell_1) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$$

Clause for the let binding

$[let] (\hat{C}, \hat{\rho}) \models (\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge$

$\hat{C}(\ell_1) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$

This clause

- requires checking the **sub-expressions**:
hence it uses “recursive calls” $(\hat{C}, \hat{\rho}) \models t_i^{\ell_i}$

the value of x must be used when analysing t_2

Double flow: propagation of the values

- from $\hat{C}(\ell_1)$ to $\hat{\rho}(x)$ [**binding**]
- from $\hat{C}(\ell_2)$ to $\hat{C}(\ell)$ [**evaluation**]

Example

$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow x^1)^2 \text{ in } (f^3 3^4)^5)^6$

$\hat{C}(2) \subseteq \hat{\rho}(x)$ and $\hat{C}(5) \subseteq \hat{C}(6) \dots$

Clause for function application

$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell$ iff $(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge$

$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$

$\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$

$(\forall (\text{fun } f x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$

$\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$
 $\{\text{fun } f x \Rightarrow e_0\} \subseteq \hat{\rho}(f))$



KEY CLAUSE
 It simulates all possible function calls

Clause for function application

$$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge$$

This clause requires checking the **sub-expressions**: hence it uses “recursive calls”

$$(\hat{C}, \hat{\rho}) \models t_i^{\ell_i}$$

Clause for function application

$$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff}$$

For each term $(\text{fn } x \Rightarrow t_0^{\ell_0})$ that may reach ℓ_1 it requires that

- the body can be analysed
- the actual parameter (with label ℓ_2) is linked to the formal parameter x
- the result of the function evaluation (with label ℓ_0) is linked to the result of the application itself (with label ℓ)

for each function
the can be applied

$$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\ (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \\ \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$$

the argument “flows
into” the variable

What comes out the
evaluation of the body
comes out also from the
application

Clause for function application

$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell$ iff

For each term $(\text{fun } f \ x \Rightarrow t_0^{\ell_0})$
that may reach ℓ_1 it requires that

- ... same as before and
- the term must be included in $\hat{\rho}(f)$ to reflect the recursive nature of the construct

$$\begin{aligned}
 & (\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\
 & \quad (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \\
 & \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell) \wedge \\
 & \quad \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{\rho}(f))
 \end{aligned}$$

Clause for function application

$$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge$$

Application = simulate all possible calls

For each function in $\hat{C}(\ell_1)$ (the function position):

- analyze body ($t_0^{\ell_0}$)
- bind argument (via $\hat{\rho}(x)$)
- propagate results

Results are merged in $\hat{C}(\ell)$

$$(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge$$

$$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$$

$$(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$$

$$\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$$

$$(\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$$

$$(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$$

$$\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$$

$$\{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{\rho}(f))$$

Clause for function application (recap)

Application clause

- combines function and argument
- drives propagation

See the more complete **Example 1** next

Back to Example 1

$(\hat{C}_e, \hat{\rho}_e)$ is an **acceptable** guess for $((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

$(\hat{C}_e, \hat{\rho}_e) \models ((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$ iff

	$(\hat{C}_e, \hat{\rho}_e)$
1	$\{\text{fn } y \Rightarrow y^3\}$
2	$\{\text{fn } x \Rightarrow x^1\}$
3	$\emptyset$
4	$\{\text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$
x	$\{\text{fn } y \Rightarrow y^3\}$
y	$\emptyset$

$$\left\{ \begin{array}{l} (\hat{C}_e, \hat{\rho}_e) \models (\text{fn } x \Rightarrow x^1)^2 \text{ iff } \{\text{fn } x \Rightarrow x^1\} \subseteq \hat{C}_e(2) \\ (\hat{C}_e, \hat{\rho}_e) \models (\text{fn } y \Rightarrow y^3)^4 \text{ iff } \{\text{fn } y \Rightarrow y^3\} \subseteq \hat{C}_e(4) \end{array} \right.$$

Since $(\text{fn } x \Rightarrow x^1) = \hat{C}_e(2)$ so we must also check

$$\left\{ \begin{array}{l} (\hat{C}_e, \hat{\rho}_e) \models x^1 \text{ iff } \hat{\rho}(x) \subseteq \hat{C}_e(1) \\ \hat{C}_e(4) \subseteq \hat{\rho}(x) \\ \hat{C}_e(1) \subseteq \hat{C}_e(5) \end{array} \right.$$

$$\begin{aligned} [app] \ (\tilde{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell \text{ iff } & (\tilde{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\tilde{C}, \hat{\rho}) \models t_2^{\ell_2} \\ & (\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\ & \quad (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \\ & \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell) \wedge \end{aligned}$$

The analysis generates constraints

Back to Example 1

Is $(\hat{C}_e, \hat{\rho}_e)$ an **acceptable** guess for $((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$?

	$(\hat{C}_e, \hat{\rho}_e)$		
1	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1\} \subseteq \hat{C}_e(2)$	✓
2	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } y \Rightarrow y^3\} \subseteq \hat{C}_e(4)$	✓
3	$\emptyset$		
4	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{\rho}(x) \subseteq \hat{C}_e(1)$	✓
5	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{C}_e(4) \subseteq \hat{\rho}(x)$	✓
x	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{C}_e(1) \subseteq \hat{C}_e(5)$	✓
y	$\emptyset$		

A simple check shows that $(\hat{C}_e, \hat{\rho}_e) \models ((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

Back to Example 1

$(\hat{C}'_e, \hat{\rho}'_e)$ is **non acceptable** for $((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

$(\hat{C}'_e, \hat{\rho}'_e) \models ((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$ iff

	$(\hat{C}_e, \hat{\rho}_e)$	$(\hat{C}'_e, \hat{\rho}'_e)$
1	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
2	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1\}$
3	$\emptyset$	$\emptyset$
4	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
x	$\{\text{fn } y \Rightarrow y^3\}$	$\emptyset$
y	$\emptyset$	$\emptyset$

$(\hat{C}'_e, \hat{\rho}'_e) \models (\text{fn } x \Rightarrow x^1)^2$ iff $\{\text{fn } x \Rightarrow x^1\} \subseteq \hat{C}'_e(2)$

$(\hat{C}'_e, \hat{\rho}'_e) \models (\text{fn } y \Rightarrow y^3)^4$ iff $\{\text{fn } y \Rightarrow y^3\} \subseteq \hat{C}'_e(4)$

Since $(\text{fn } x \Rightarrow x^1) = \hat{C}_e(e)$ so we must also check

$(\hat{C}'_e, \hat{\rho}'_e) \models x^1$ iff $\hat{\rho}'_e(x) \subseteq \hat{C}'_e(1)$

$\hat{C}'_e(4) \subseteq \hat{\rho}'_e(x)$ **X**

$\hat{C}'_e(1) \subseteq \hat{C}'_e(5)$

missing possible
value of x

A guess is **acceptable** only if all constraints hold

Back to Example 1

$(\hat{C}_e, \hat{\rho}_e)$ is an **acceptable** guess for $((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

Intuitively, we solve the constraints iteratively until reaching a fix point

$$\{\text{fn } x \Rightarrow x^1\} \subseteq \hat{C}_e(2)$$

$$\{\text{fn } y \Rightarrow y^3\} \subseteq \hat{C}_e(4)$$

$$\hat{\rho}(x) \subseteq \hat{C}_e(1)$$

$$\hat{C}_e(4) \subseteq \hat{\rho}(x)$$

$$\hat{C}_e(1) \subseteq \hat{C}_e(5)$$

$$\hat{C}_e(1)$$

$$\hat{C}_e(2)$$

$$\hat{C}_e(3)$$

$$\hat{C}_e(4)$$

$$\hat{C}_e(5)$$

$$\hat{\rho}(x)$$

$$\hat{\rho}(y)$$

Back to Example 1

$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

$\{\text{fn } x \Rightarrow x^1\} \subseteq \hat{C}_e(2)$

$\{\text{fn } y \Rightarrow y^3\} \subseteq \hat{C}_e(4)$

$\hat{\rho}(x) \subseteq \hat{C}_e(1)$

$\hat{C}_e(4) \subseteq \hat{\rho}(x)$

$\hat{C}_e(1) \subseteq \hat{C}_e(5)$

$\hat{C}_e(1)$	$\emptyset$	
$\hat{C}_e(2)$	$\emptyset$	
$\hat{C}_e(3)$	$\emptyset$	
$\hat{C}_e(4)$	$\emptyset$	
$\hat{C}_e(5)$	$\emptyset$	
$\hat{\rho}(x)$	$\emptyset$	
$\hat{\rho}(y)$	$\emptyset$	

Back to Example 1

$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

$\{\text{fn } x \Rightarrow x^1\} \subseteq \hat{C}_e(2)$

$\{\text{fn } y \Rightarrow y^3\} \subseteq \hat{C}_e(4)$

$\hat{\rho}(x) \subseteq \hat{C}_e(1)$

$\hat{C}_e(4) \subseteq \hat{\rho}(x)$

$\hat{C}_e(1) \subseteq \hat{C}_e(5)$

$\hat{C}_e(1)$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1\}$	
$\hat{C}_e(2)$	$\emptyset$	$\emptyset$	
$\hat{C}_e(3)$	$\emptyset$	$\emptyset$	
$\hat{C}_e(4)$	$\emptyset$	$\{\text{fn } y \Rightarrow y^3\}$	
$\hat{C}_e(5)$	$\emptyset$	$\emptyset$	
$\hat{\rho}(x)$	$\emptyset$	$\emptyset$	
$\hat{\rho}(y)$	$\emptyset$	$\emptyset$	

Back to Example 1

$$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$$

	$(\hat{C}_e, \hat{\rho}_e)$			
1	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{C}_e(1)$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1\}$
2	$\{\text{fn } x \Rightarrow x^1\}$	$\hat{C}_e(2)$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1\}$
3	$\emptyset$	$\hat{C}_e(3)$	$\emptyset$	$\emptyset$
4	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{C}_e(4)$	$\emptyset$	$\{\text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{C}_e(5)$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1\}$
x	$\{\text{fn } y \Rightarrow y^3\}$	$\hat{\rho}(x)$	$\emptyset$	$\{\text{fn } y \Rightarrow y^3\}$
y	$\emptyset$	$\hat{\rho}(y)$	$\emptyset$	$\emptyset$

CFA

- [con] $(\widehat{C}, \widehat{\rho}) \models c^\ell$ always
- [var] $(\widehat{C}, \widehat{\rho}) \models x^\ell$ iff $\widehat{\rho}(x) \subseteq \widehat{C}(\ell)$
- [fn] $(\widehat{C}, \widehat{\rho}) \models (\mathbf{fn} \ x \Rightarrow e_0)^\ell$ iff $\{\mathbf{fn} \ x \Rightarrow e_0\} \subseteq \widehat{C}(\ell)$
- [fun] $(\widehat{C}, \widehat{\rho}) \models (\mathbf{fun} \ f \ x \Rightarrow e_0)^\ell$ iff $\{\mathbf{fun} \ f \ x \Rightarrow e_0\} \subseteq \widehat{C}(\ell)$
- [app] $(\widehat{C}, \widehat{\rho}) \models (t_1^{\ell_1} \ t_2^{\ell_2})^\ell$
iff $(\widehat{C}, \widehat{\rho}) \models t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models t_2^{\ell_2} \wedge$
 $(\forall (\mathbf{fn} \ x \Rightarrow t_0^{\ell_0}) \in \widehat{C}(\ell_1) :$
 $\quad (\widehat{C}, \widehat{\rho}) \models t_0^{\ell_0} \wedge$
 $\quad \widehat{C}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_0) \subseteq \widehat{C}(\ell)) \wedge$
 $(\forall (\mathbf{fun} \ f \ x \Rightarrow t_0^{\ell_0}) \in \widehat{C}(\ell_1) :$
 $\quad (\widehat{C}, \widehat{\rho}) \models t_0^{\ell_0} \wedge$
 $\quad \widehat{C}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_0) \subseteq \widehat{C}(\ell) \wedge$
 $\quad \{\mathbf{fun} \ f \ x \Rightarrow t_0^{\ell_0}\} \subseteq \widehat{\rho}(f))$
- [if] $(\widehat{C}, \widehat{\rho}) \models (\mathbf{if} \ t_0^{\ell_0} \ \mathbf{then} \ t_1^{\ell_1} \ \mathbf{else} \ t_2^{\ell_2})^\ell$
iff $(\widehat{C}, \widehat{\rho}) \models t_0^{\ell_0} \wedge$
 $(\widehat{C}, \widehat{\rho}) \models t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models t_2^{\ell_2} \wedge$
 $\widehat{C}(\ell_1) \subseteq \widehat{C}(\ell) \wedge \widehat{C}(\ell_2) \subseteq \widehat{C}(\ell)$
- [let] $(\widehat{C}, \widehat{\rho}) \models (\mathbf{let} \ x = t_1^{\ell_1} \ \mathbf{in} \ t_2^{\ell_2})^\ell$
iff $(\widehat{C}, \widehat{\rho}) \models t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models t_2^{\ell_2} \wedge$
 $\widehat{C}(\ell_1) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_2) \subseteq \widehat{C}(\ell)$
- [op] $(\widehat{C}, \widehat{\rho}) \models (t_1^{\ell_1} \ \mathbf{op} \ t_2^{\ell_2})^\ell$ iff $(\widehat{C}, \widehat{\rho}) \models t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models t_2^{\ell_2}$
-

Inclusions

Note that the clauses contain a number of inclusions of the form

$$lhs \subseteq rhs$$

where:

- $lhs ::= \hat{C}(\ell) \mid \hat{\rho}(x), \{t\}$ where t is a function abstraction
- $rhs ::= \hat{C}(\ell) \mid \hat{\rho}(x)$

These express how functions may flow through the program, e.g.,

$$\hat{C}(\ell) \subseteq \hat{\rho}(x)$$

Observation 1

$[fn] (\hat{C}, \hat{\rho}) \models (fn\ x \Rightarrow e_0)^\ell$ iff $\{fn\ x \Rightarrow e_0\} \subseteq \hat{C}(\ell)$

$[fn]$ (and $[fun]$) do **not** recursively analyze subexpressions

$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell$ iff $(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge$
 $(\forall (fn\ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$
 $\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$
 $(\forall (fun\ f\ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$
 $\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$
 $\{fun\ f\ x \Rightarrow e_0\} \subseteq \hat{\rho}(f))$

$[app]$ does the job (is responsible for analyzing function bodies)

the analysis is
driven by
reachability

In this way, we analyse the body of a function **only** if the function may be applied somewhere

Unreachable functions are not analysed

Observation 2

$$\begin{aligned}
 [app] \ (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell \text{ iff } & \boxed{(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}} \wedge \\
 & (\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \quad t_0^{\ell_0} \text{ is not a subexpression} \\
 & \quad \boxed{(\hat{C}, \hat{\rho}) \models t_0^{\ell_0}} \wedge \\
 & \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge \\
 & (\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\
 & \quad (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \\
 & \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge \\
 & \quad \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{\rho}(f))
 \end{aligned}$$

The relation $\models$ is not syntax-directed (due to the application clause)

So we cannot define $\models$ by structural induction on expressions

Instead, define $\models$ **coinductively**

Back to Example 2

loop $\triangleq$ (let g = (fun f x => (f¹ (fn y => y²)³)⁴)⁵
in (g⁶ (fn z => z⁷)⁸)⁹)¹⁰

Here is an acceptable guess

$\hat{C}_{lp}(1) = \{f\}$	$\hat{C}_{lp}(6) = \{f\}$	$\hat{\rho}_{lp}(f) = \{f\}$
$\hat{C}_{lp}(2) = \emptyset$	$\hat{C}_{lp}(7) = \emptyset$	$\hat{\rho}_{lp}(g) = \{f\}$
$\hat{C}_{lp}(3) = \{id_y\}$	$\hat{C}_{lp}(8) = \{id_z\}$	$\hat{\rho}_{lp}(x) = \{id_y, id_z\}$
$\hat{C}_{lp}(4) = \emptyset$	$\hat{C}_{lp}(9) = \emptyset$	$\hat{\rho}_{lp}(y) = \emptyset$
$\hat{C}_{lp}(5) = \{f\}$	$\hat{C}_{lp}(10) = \emptyset$	$\hat{\rho}_{lp}(z) = \emptyset$

$f \triangleq \text{fun } f \ x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4$
 $id_y \triangleq \text{fn } y \Rightarrow y^2$
 $id_z \triangleq \text{fn } z \Rightarrow z^7$

- $\hat{\rho}_{lp}(g) = \{f\}$ reflects that g will be evaluated to a closure constructed from that abstraction
- $\hat{\rho}_{lp}(x) = \{id_y, id_z\}$ reflects that x will be bound to closures constructed from both abstractions in the course of evaluation
- $\hat{C}_{lp}(10) = \emptyset$ (no function value can be produced at this point) reflects that the evaluation of loop never terminates

Back to Example 2

$\text{loop} \triangleq (\text{let } g = (\text{fun } f \text{ } x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4)^5 \text{ in } (g^6 (\text{fn } z \Rightarrow z^7)^8)^9)^{10}$

$\hat{C}_{lp}(1) = \{f\}$	$\hat{C}_{lp}(6) = \{f\}$	$\hat{\rho}_{lp}(f) = \{f\}$
$\hat{C}_{lp}(2) = \emptyset$	$\hat{C}_{lp}(7) = \emptyset$	$\hat{\rho}_{lp}(g) = \{f\}$
$\hat{C}_{lp}(3) = \{\text{id}_y\}$	$\hat{C}_{lp}(8) = \{\text{id}_z\}$	$\hat{\rho}_{lp}(x) = \{\text{id}_y, \text{id}_z\}$
$\hat{C}_{lp}(4) = \emptyset$	$\hat{C}_{lp}(9) = \emptyset$	$\hat{\rho}_{lp}(y) = \emptyset$
$\hat{C}_{lp}(5) = \{f\}$	$\hat{C}_{lp}(10) = \emptyset$	$\hat{\rho}_{lp}(z) = \emptyset$

$f \triangleq \text{fun } f \text{ } x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4$
 $\text{id}_y \triangleq \text{fn } y \Rightarrow y^2$
 $\text{id}_z \triangleq \text{fn } z \Rightarrow z^7$

from [fun]

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models \text{loop}$ iff $(\hat{C}_{lp}, \hat{\rho}_{lp}) \models f^5 \wedge ((\hat{C}_{lp}, \hat{\rho}_{lp}) \models g^6 (\text{fn } z \Rightarrow z^7)^8)$
 $\wedge \hat{C}_{lp}(5) \subseteq \hat{\rho}_{lp}(g) \wedge \hat{C}_{lp}(9) \subseteq \hat{C}_{lp}(10)$ ✓

using [app] and
that $\hat{C}(6) = \{f\}$

immediate

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models g^6 \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models (\text{fn } z \Rightarrow z^7)^8 \wedge ((\hat{C}_{lp}, \hat{\rho}_{lp}) \models (f^1 (\text{fn } y \Rightarrow y^2)^3)^4)$
 $\wedge \hat{C}_{lp}(8) \subseteq \hat{\rho}_{lp}(x) \wedge \hat{C}_{lp}(4) \subseteq \hat{C}_{lp}(9) \wedge f \in \hat{\rho}_{lp}(f)$ ✓

immediate

Immediate

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models f^1 \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models (\text{fn } y \Rightarrow y^2)^3 \wedge ((\hat{C}_{lp}, \hat{\rho}_{lp}) \models (f^1 (\text{fn } y \Rightarrow y^2)^3)^4)$
 $\wedge \hat{C}_{lp}(3) \subseteq \hat{\rho}_{lp}(x) \wedge \hat{C}_{lp}(4) \subseteq \hat{C}_{lp}(4) \wedge f \in \hat{\rho}_{lp}(f)$ ✓

Circular dependency => coinduction: assume it holds and verify it is preserved

Coinductive nature of the CFA

The **specification** ($\models$) is **coinductive**

loose: includes many solutions

- It defines **all acceptable solutions**
- A solution is accepted by default: it is **rejected only if it violates a constraint**

Semantically this is a **greatest fixed point**

Computation

Among all acceptable solutions, we are interested in the smallest

We compute **the least solution satisfying the specification**

Algorithmically, this is a **least fixed point**

most precise safe approximation

The nature of this CFA

This is a 0-CFA analysis, characterised as:

```
let x = 0 in  
  let x = 1 in  
    x
```

Flow-sensitive $\rightarrow x = 1$
Flow-insensitive $\rightarrow x \in \{0, 1\}$

- **flow-insensitive**: it computes a single abstract value for each variable (or expression) that over-approximates all values it may take during execution, without distinguishing between different program points or execution orders
- **context-insensitive**: the same function is analyzed in the same way at all call sites

```
let f = fn x => x in  
  let y = f 1 in  
    let z = f 2 in  
      ...
```

0-CFA analyzes f **once**
merging all calls

no information
about calling
context is used

Questions

Question 1

Let $e \triangleq ((\text{fn } y \Rightarrow (y^1 \ 2^2)^3)^4 (\text{fn } x \Rightarrow (x^5 * 3^6)^7)^8)^9$

Which are the labels?

Which is the flow from

1. the program point with label 5 to program point with label 7?
2. the program point with label 8 to variable y ?

1. No flow: constants are not tracked in 0-CFA
2. Flow from 8 to y : $\{\text{fn } x \Rightarrow (x^3 * 3^4)^5\}$ (via application)

Only functions flow in 0-CFA, not data

Question 2

Let $e \triangleq (\text{let } f = (\text{fn } x \Rightarrow x^1)^2 \text{ in } (f^3 4^4)^5)^6$

Which is the flow

- from the program point with label 2 to f ?
- from the program point with label 4 to x ?
- from the program point with label 5 to the program point with label 6?
- Flow from 2 to f : $\{\text{fn } x \Rightarrow x^1\}$
- No flow: 4 is a constant and data are not tracked
- No flow: since no value flows to x , nothing reaches label 6

Program Analysis 2025-26

Lecture #18

Control Flow Analysis



UNIVERSITÀ
DI PISA

Exercises

Exercises

Consider and add labels to the following expressions
Guess analysis results and verify them

$$e \triangleq ((\text{fn } x \Rightarrow (x + 1)) (3))$$

$$e \triangleq ((\text{fn } f \Rightarrow ((f\ 76) + (f\ 77))) (\text{fn } a \Rightarrow a))$$

$$e \triangleq \text{let } f = \text{fn } x \Rightarrow x + 1 \\ \text{in let } g = \text{fn } y \Rightarrow y + 2 \\ \text{in let } h = \text{fn } z \Rightarrow z + 3 \\ \text{in } (f\ g) + (f\ h)$$

Exercise 1

$$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}$$

$$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$$

Consider and add labels to the following expression
Guess an analysis result and verify it

$$e \triangleq ((\text{fn } x \Rightarrow (x^1 + 1^{23})^4) (3)^5)^6$$

$\hat{C}(1)$	$\emptyset$ (3)
$\hat{C}(2)$	$\emptyset$ (1)
$\hat{C}(3)$	$\emptyset$ (4)
$\hat{C}(4)$	$\{\text{fn } x \Rightarrow (x^1 + 1^{23})^4\}$
$\hat{C}(5)$	$\emptyset$ (3)
$\hat{C}(6)$	$\emptyset$ (4)
$\hat{\rho}(x)$	$\emptyset$ (3)

Exercise 1

$$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}$$

$$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$$

Consider and add labels to the following expression
Guess an analysis result and verify it

$$e \triangleq ((\text{fn } x \Rightarrow (x^1 + 1^2)^3)^4 (3)^5)^6$$

$$(\hat{C}, \hat{\rho}) \models e \text{ iff } (\hat{C}, \hat{\rho}) \models (\text{fn } x \Rightarrow (x^1 + 1^2)^3)^4 \text{ iff } (\hat{C}, \hat{\rho}) \models \{\text{fn } x \Rightarrow (x^1 + 1^2)^3\} \subseteq \hat{C}(4)$$

$$\wedge (\hat{C}, \hat{\rho}) \models (3)^5 \text{ iff } \emptyset \subseteq \hat{C}(5)$$

$$\{\text{fn } x \Rightarrow (x^1 + 1^2)^3\} \subseteq \hat{C}(4) \Rightarrow (\hat{C}, \hat{\rho}) \models (x^1 + 1^2)^3 \wedge$$

$$\hat{C}(5) \subseteq \hat{\rho}(x) \wedge \hat{C}(3) \subseteq \hat{C}(6)$$

$$(\hat{C}, \hat{\rho}) \models (x^1 + 1^2)^3 \text{ iff } (\hat{C}, \hat{\rho}) \models x^1 \text{ iff } \hat{\rho}(x) \subseteq \hat{C}(1)$$

$$\wedge (\hat{C}, \hat{\rho}) \models 1^2 \text{ iff } \emptyset \subseteq \hat{C}(2)$$

Exercise 1

$$[app] \ (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}$$

$$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$$

Consider and add labels to the following expression
Guess an analysis result and verify it

$$e \triangleq ((\text{fn } x \Rightarrow (x^1 + 1^{23})^4) (3)^5)^6$$

$$\{\text{fn } x \Rightarrow (x^1 + 1^{23})\} \subseteq \hat{C}(4) \quad \checkmark$$

$$\emptyset \subseteq \hat{C}(5) \quad \checkmark$$

$$\hat{C}(5) \subseteq \hat{\rho}(x) \quad \checkmark$$

$$\hat{C}(3) \subseteq \hat{C}(6) \quad \checkmark$$

$$\hat{\rho}(x) \subseteq \hat{C}(1) \quad \checkmark$$

$$\emptyset \subseteq \hat{C}(2) \quad \checkmark$$

$\hat{C}(1)$	$\emptyset$ (3)
$\hat{C}(2)$	$\emptyset$ (1)
$\hat{C}(3)$	$\emptyset$ (4)
$\hat{C}(4)$	$\{\text{fn } x \Rightarrow (x^1 + 1^{23})\}$
$\hat{C}(5)$	$\emptyset$ (3)
$\hat{C}(6)$	$\emptyset$ (4)
$\hat{\rho}(x)$	$\emptyset$ (3)



Exercise 2

Consider and add labels to the following expression: guess an analysis result and verify it

$$e \triangleq ((\text{fn } f \Rightarrow ((f^{1} 76^{2})^{3} + (f^{4} 77^{5})^{6})^{7})^{8} (\text{fn } a \Rightarrow a^{9})^{10})^{12}$$

Exercise 2

$$e \triangleq ((\text{fn } f \Rightarrow ((f^1 76^2)^3 + (f^4 77^5)^6)^7)^8 (\text{fn } a \Rightarrow a^9)^{10})^{12}$$

$$\hat{C}(1) = \{\text{fn } a \Rightarrow a^9\}$$

$$\hat{C}(2) = \emptyset \text{ (76)}$$

$$\hat{C}(3) = \emptyset \text{ (76)}$$

$$\hat{C}(4) = \{\text{fn } a \Rightarrow a^9\}$$

$$\hat{C}(5) = \emptyset \text{ (77)}$$

$$\hat{C}(6) = \emptyset \text{ (77)}$$

$$\hat{C}(7) = \emptyset \text{ (76+77)}$$

$$\hat{C}(8) = \{\text{fn } f \Rightarrow ((f^1 76^2)^3 + (f^4 77^5)^6)^2\}$$

$$\hat{C}(9) = \emptyset \text{ (76, 77)}$$

$$\hat{C}(10) = \{\text{fn } a \Rightarrow a^9\}$$

$$\hat{C}(11) = \emptyset$$

$$\hat{C}(12) = \{\text{fn } f \Rightarrow ((f^1 76^2)^3 + (f^4 77^5)^6)^2\}$$

$$\hat{\rho}(f) = \{\text{fn } a \Rightarrow a^9\}$$

$$\hat{\rho}(a) = \emptyset \text{ (76, 77)}$$

Exercise 2

[app] $(\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell$ iff $(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}$
 $(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $(\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$
 $\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$

$$e \triangleq ((\text{fn } f \Rightarrow ((f^1 76^2)^3 + (f^4 77^5)^6)^7)^8 (\text{fn } a \Rightarrow a^9)^{10})^{12}$$

$$\begin{aligned} (\hat{C}, \hat{\rho}) \models e &\text{ iff } (\text{fn } f \Rightarrow ((f^1 76^2)^3 + (f^4 77^5)^6)^7)^8 \text{ iff } \{\text{fn } f \Rightarrow ((f^1 76^2)^3 + (f^4 77^5)^6)^7\} \in \hat{C}(8) \\ &\wedge (\hat{C}, \hat{\rho}) \models (\text{fn } a \Rightarrow a^9)^{10} \text{ iff } \{\text{fn } a \Rightarrow a^9\} \in \hat{C}(10) \wedge \\ &\wedge (\hat{C}, \hat{\rho}) \models ((f^1 76^2)^3 + (f^4 77^5)^6)^7 \text{ iff } (\hat{C}, \hat{\rho}) \models (f^1 76^2)^3 \wedge (\hat{C}, \hat{\rho}) \models (f^4 77^5)^6 \\ &\wedge \hat{C}(10) \subseteq \hat{\rho}(f) \wedge \hat{C}(8) \subseteq \hat{C}(12) \end{aligned}$$

$$\begin{aligned} (\hat{C}, \hat{\rho}) \models (f^1 76^2)^3 &\text{ iff } (\hat{C}, \hat{\rho}) \models f^1 \wedge (\hat{C}, \hat{\rho}) \models 76^2 \wedge (\hat{C}, \hat{\rho}) \models a^9 \wedge \hat{C}(2) \subseteq \hat{\rho}(a) \wedge \hat{C}(9) \subseteq \hat{C}(3) \\ (\hat{C}, \hat{\rho}) \models (f^4 77^5)^6 &\text{ iff } (\hat{C}, \hat{\rho}) \models f^4 \wedge (\hat{C}, \hat{\rho}) \models 77^5 \wedge (\hat{C}, \hat{\rho}) \models a^9 \wedge \hat{C}(5) \subseteq \hat{\rho}(a) \wedge \hat{C}(9) \subseteq \hat{C}(6) \\ (\hat{C}, \hat{\rho}) \models f^1 &\text{ iff } \hat{\rho}(f) \subseteq \hat{C}(1) \wedge (\hat{C}, \hat{\rho}) \models a^9 \text{ iff } \hat{\rho}(a) \subseteq \hat{C}(9) \\ (\hat{C}, \hat{\rho}) \models f^4 &\text{ iff } \hat{\rho}(f) \subseteq \hat{C}(4) \wedge \end{aligned}$$

Exercise 2

$((\text{fn } f \Rightarrow ((f^1 76^{23}) + (f^4 77^{56})^7)^8 (\text{fn } a \Rightarrow a^9)^{10})^{12}$

$\{\text{fn } f \Rightarrow ((f^1 76^{23}) + (f^4 77^{56})^2)\} \in \hat{C}(8)$

$\{\text{fn } a \Rightarrow a^9\} \in \hat{C}(10)$

$\hat{C}(10) \subseteq \hat{\rho}(f)$

$\hat{C}(8) \subseteq \hat{C}(12)$

$\hat{C}(2) \subseteq \hat{\rho}(a)$

$\hat{C}(9) \subseteq \hat{C}(3)$

$\hat{C}(9) \subseteq \hat{C}(6)$

$\hat{\rho}(f) \subseteq \hat{C}(1)$

$\hat{\rho}(f) \subseteq \hat{C}(4)$

$\hat{\rho}(a) \subseteq \hat{C}(9)$

$\hat{C}(1) = \{\text{fn } a \Rightarrow a^9\}$

$\hat{C}(2) = \emptyset \text{ (76)}$

$\hat{C}(3) = \emptyset \text{ (76)}$

$\hat{C}(4) = \{\text{fn } a \Rightarrow a^9\}$

$\hat{C}(5) = \emptyset \text{ (77)}$

$\hat{C}(6) = \emptyset \text{ (77)}$

$\hat{C}(7) = \emptyset \text{ (76+77)}$

$\hat{C}(8) = \{\text{fn } f \Rightarrow ((f^1 76^{23}) + (f^4 77^{56})^2)\}$

$\hat{C}(9) = \emptyset \text{ (76, 77)}$

$\hat{C}(10) = \{\text{fn } a \Rightarrow a^9\}$

$\hat{C}(11) = \emptyset$

$\hat{C}(12) = \{\text{fn } f \Rightarrow ((f^1 76^{23}) + (f^4 77^{56})^2)\}$

$\hat{\rho}(f) = \{\text{fn } a \Rightarrow a^9\}$

$\hat{\rho}(a) = \emptyset \text{ (76, 77)}$



Exercise 3

Consider and add labels to the following expression

Guess an analysis result and verify it

```
let f = fn x => x 1
  in let g = fn y => y + 2
    in let h = fn z => z + 3
      in (f g) + (f h)
```

$\text{id}_w \triangleq \text{fn } w \Rightarrow w^{\ell}$

```
e  $\triangleq$  (let f = (fn x => (x1 12)3)4
  in (let g = (fn y => (y5 + 26)7)8
    in (let h = (fn z => (z9 + 310)11)12
      in ((f13 g14)15 + (f16 h17)18)19)20)21)22
```

Complete the exercise

Theoretical properties

Semantic correctness

Goal: show that acceptable CFA results are semantically correct

The analysis information is a safe description of what will happen during the evaluation of the program, i.e., a **sound over-approximation** of the runtime behaviour. For instance, if at runtime

- t^ℓ evaluates to a function, then this must be duly recorded in $\hat{C}(\ell)$
- x is bound to a function at runtime, this must also be duly recorded in $\hat{\rho}(x)$

Semantic correctness

The analysis information is a safe description of what will happen during the evaluation of the program, i.e., a **sound over-approximation** of the runtime behaviour

To reason on correctness, we define a **Small Step Structural Operational Semantics** for FUN, where:

- We use **intermediate expressions** ie to handle the binding of local variables
- We choose an approach based on **explicit environments** (matching the CFA abstraction) rather than substitutions

to preserve the identity of functions (and hence abstract values) during evaluation

Subject reduction result

If the abstract ($\hat{\rho}$) and concrete environments (ρ) agree ($\rho R \hat{\rho}$):

$(\hat{C}, \hat{\rho}) \models ie$ and $\rho \vdash ie \rightarrow ie'$ then $(\hat{C}, \hat{\rho}) \models ie'$

$\rho R \hat{\rho}$ iff $dom(\rho) \subseteq dom(\hat{\rho}) \wedge \forall x \in dom(\rho) \forall t_x \forall \rho_x : (\rho(x) = \text{close } t_x \text{ in } \rho_x) \Rightarrow (t_x \in \hat{\rho}(x) \wedge \rho_x R \hat{\rho})$

Configurations

A function definition will evaluate to a **closure** containing

- the syntax of the function definition, together with
- an environment mapping its free variables to their values

$v \in \mathbf{Val}$: values (evaluation results)

$\rho \in \mathbf{Env}$: environments (mapping free variables to their *values*)

$v ::= c \mid \text{close } t \text{ in } \rho$

$\rho ::= [] \mid \rho[x \mapsto v]$

Static binding:
functions evaluate to
closures capturing their
environment

Structural Operational Semantic

Terms and expressions

$ie \in \mathbf{IExp}$: intermediate expressions

$it \in \mathbf{ITerm}$: intermediate terms

$ie ::= it^\ell$

$it ::= c \mid x \mid \text{fn } x \Rightarrow e_0 \mid \text{fun } f \ x \Rightarrow e_0$
| $ie_1 \ ie_2$ | if ie_0 then e_1 else e_2 | let $x = ie_1$ in e_2 | ie_1 op ie_2
| bind ρ in ie // bind the environment (encoding of frames)
| close t in ρ // closure

Transitions: $\boxed{\rho \vdash ie \rightarrow ie'}$ one step of computation of the expression ie

Rule for variables

$$[var] \rho \vdash x^\ell \rightarrow v^\ell \text{ iff } x \in Dom(\rho) \text{ and } \rho(x) = v$$

The value of a variable is obtained from the environment

Example

$$\rho \vdash x^1 \rightarrow 7^1 \text{ if } x \in Dom(\rho) \text{ and } \rho(x) = 7$$

Rule for functions

$$[fn] \rho \vdash (fn\ x \Rightarrow e_0)^\ell \rightarrow (close\ (fn\ x \Rightarrow e_0)\ in\ \rho_0)^\ell$$

where $\rho_0 = \rho \mid FV(fn\ x \Rightarrow e_0)$

Static scoping:
freeze the environment
at definition time

This axiom constructs the appropriate closure

It restricts the environment ρ to the free variables of the abstraction

Note that the label ℓ is preserved in the closure

Example

$$[y \mapsto 7] \vdash (fn\ x \Rightarrow (x^1\ y^{23})^4) \rightarrow (close\ (fn\ x \Rightarrow (x^1\ y^{23}))\ in\ [y \mapsto 7])^4$$

$\rho_0 = \rho \mid \{y\} = [y \mapsto 7]$, since $FV(fn\ x \Rightarrow (x^1\ y^{23})) = \{y\}$

only free variables
are kept

Rule for let binding

$$[let_2] \ \rho \vdash (\text{let } x = v^{\ell_1} \text{ in } e_2)^{\ell} \rightarrow (\text{bind } \rho_0[x \mapsto v] \text{ in } e_2)^{\ell}$$

where $\rho_0 = \rho \upharpoonright FV(e_2)$

The expression e_2 is evaluated in the environment augmented with the new binding

Example

$$[g \mapsto (\text{fn } y \Rightarrow y^1)^2] \vdash (\text{let } f = (\text{fn } x \Rightarrow x^1)^2 \text{ in } (f^3 \ 3^4)^5) \\ \rightarrow (\text{bind } \rho_0[f \mapsto (\text{fn } x \Rightarrow x^1)^2] \text{ in } (f^3 \ 3^4)^5_4, \text{ where } \rho_0 = \rho \upharpoonright \{f\} = [])$$

Rule for application

$$[app_{fn}] \rho \vdash ((\text{close}(\text{fn } x \Rightarrow e_1) \text{ in } \rho_1)^{\ell_1} v_2^{\ell_2})^{\ell} \rightarrow (\text{bind } \rho_1[x \mapsto v_2] \text{ in } e_1)^{\ell}$$

This axiom describes the binding of the actual parameter to the formal parameter

Example

$$[y \mapsto 5] \vdash ((\text{close}(\text{fn } x \Rightarrow (x^1 + y^2)^3)^4 \text{ in } [y \mapsto 7])^4 3^5)^6 \rightarrow (\text{bind } [y \mapsto 7, x \mapsto 3] \text{ in } (x^1 + y^2)^3)^6_{85}, \text{ where } y = 7 \text{ (not 5)}$$

Static scoping:
we use the definition
environment, not the call

Rules for binding

$$[bind_1] \frac{\rho \vdash ie_1 \rightarrow ie'_1}{\rho \vdash (\text{bind } \rho_1 \text{ in } ie_1)^\ell \rightarrow (\text{bind } \rho_1 \text{ in } ie'_1)^\ell}$$

$$[bind_2] \rho \vdash (\text{bind } \rho_1 \text{ in } v_1^{\ell_1})^\ell \rightarrow v_1^\ell$$

- The bind construct is evaluated by repeatedly applying $[bind_1]$
- The final evaluation result is get by using rule $[bind_2]$

Example

$$[y \mapsto 5] \vdash (\text{bind } [y \mapsto 7, x \mapsto 3] \text{ in } x^1 + y^2)^6 \rightarrow * 10^6$$

Structural Operational Semantic I

[var]	$\rho \vdash x^\ell \rightarrow v^\ell \text{ if } x \in \text{dom}(\rho) \text{ and } v = \rho(x)$
[fn]	$\rho \vdash (\text{fn } x \Rightarrow e_0)^\ell \rightarrow (\text{close } (\text{fn } x \Rightarrow e_0) \text{ in } \rho_0)^\ell$ where $\rho_0 = \rho \mid FV(\text{fn } x \Rightarrow e_0)$
[fun]	$\rho \vdash (\text{fun } f \ x \Rightarrow e_0)^\ell \rightarrow (\text{close } (\text{fun } f \ x \Rightarrow e_0) \text{ in } \rho_0)^\ell$ where $\rho_0 = \rho \mid FV(\text{fun } f \ x \Rightarrow e_0)$
[app₁]	$\frac{\rho \vdash ie_1 \rightarrow ie'_1}{\rho \vdash (ie_1 \ ie_2)^\ell \rightarrow (ie'_1 \ ie_2)^\ell}$
[app₂]	$\frac{\rho \vdash ie_2 \rightarrow ie'_2}{\rho \vdash (v_1^{\ell_1} \ ie_2)^\ell \rightarrow (v_1^{\ell_1} \ ie'_2)^\ell}$
[app_{fn}]	$\rho \vdash ((\text{close } (\text{fn } x \Rightarrow e_1) \text{ in } \rho_1)^{\ell_1} \ v_2^{\ell_2})^\ell \rightarrow$ $(\text{bind } \rho_1[x \mapsto v_2] \text{ in } e_1)^\ell$
[app_{fun}]	$\rho \vdash ((\text{close } (\text{fun } f \ x \Rightarrow e_1) \text{ in } \rho_1)^{\ell_1} \ v_2^{\ell_2})^\ell \rightarrow$ $(\text{bind } \rho_2[x \mapsto v_2] \text{ in } e_1)^\ell$ where $\rho_2 = \rho_1[f \mapsto \text{close } (\text{fun } f \ x \Rightarrow e_1) \text{ in } \rho_1]$
[bind₁]	$\frac{\rho_1 \vdash ie_1 \rightarrow ie'_1}{\rho \vdash (\text{bind } \rho_1 \text{ in } ie_1)^\ell \rightarrow (\text{bind } \rho_1 \text{ in } ie'_1)^\ell}$
[bind₂]	$\rho \vdash (\text{bind } \rho_1 \text{ in } v_1^{\ell_1})^\ell \rightarrow v_1^\ell$

Structural Operational Semantic II

$$\begin{array}{l}
 [if_1] \quad \frac{\rho \vdash ie_0 \rightarrow ie'_0}{\rho \vdash (\text{if } ie_0 \text{ then } e_1 \text{ else } e_2)^\ell \rightarrow (\text{if } ie'_0 \text{ then } e_1 \text{ else } e_2)^\ell} \\
 [if_2] \quad \rho \vdash (\text{if true}^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell \rightarrow t_1^\ell \\
 [if_3] \quad \rho \vdash (\text{if false}^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell \rightarrow t_2^\ell \\
 [let_1] \quad \frac{\rho \vdash ie_1 \rightarrow ie'_1}{\rho \vdash (\text{let } x = ie_1 \text{ in } e_2)^\ell \rightarrow (\text{let } x = ie'_1 \text{ in } e_2)^\ell} \\
 [let_2] \quad \rho \vdash (\text{let } x = v^{\ell_1} \text{ in } e_2)^\ell \rightarrow (\text{bind } \rho_0[x \mapsto v] \text{ in } e_2)^\ell \\
 \quad \text{where } \rho_0 = \rho \mid FV(e_2) \\
 [op_1] \quad \frac{\rho \vdash ie_1 \rightarrow ie'_1}{\rho \vdash (ie_1 \text{ op } ie_2)^\ell \rightarrow (ie'_1 \text{ op } ie_2)^\ell} \\
 [op_2] \quad \frac{\rho \vdash ie_2 \rightarrow ie'_2}{\rho \vdash (v_1^{\ell_1} \text{ op } ie_2)^\ell \rightarrow (v_1^{\ell_1} \text{ op } ie'_2)^\ell} \\
 [op_3] \quad \rho \vdash (v_1^{\ell_1} \text{ op } v_2^{\ell_2})^\ell \rightarrow v^\ell \quad \text{if } v = v_1 \text{ op } v_2
 \end{array}$$

Back to Example 1: evaluation

$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

$[] \vdash ((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$
→ $((\text{close}(\text{fn } x \Rightarrow x^1) \text{ in } [])^2 (\text{fn } y \Rightarrow y^3)^4)^5$
→ $((\text{close}(\text{fn } x \Rightarrow x^1) \text{ in } [])^2 (\text{close } (\text{fn } y \Rightarrow y^3) \text{ in } [])^4)^5$
→ $(\text{bind } [x \mapsto (\text{close } (\text{fn } y \Rightarrow y^3) \text{ in } [])] \text{ in } x^1)^5$
→ $(\text{bind } [x \mapsto (\text{close } (\text{fn } y \Rightarrow y^3) \text{ in } [])] \text{ in } (\text{close } (\text{fn } y \Rightarrow y^3)^4 \text{ in } [])^1)^5$
→ $(\text{close } (\text{fn } y \Rightarrow y^3) \text{ in } [])^5$

New CFA clauses

$$\begin{aligned} [bind] \quad (\hat{C}, \hat{\rho}) \models (\text{bind } \rho \text{ in } it_0^{\ell_0})^\ell &\text{ iff } (\hat{C}, \hat{\rho}) \models it_0^{\ell_0} \wedge \\ &\hat{C}(\ell_0) \subseteq \hat{C}(\ell) \wedge \rho R \hat{\rho} \\ [close] \quad (\hat{C}, \hat{\rho}) \models (\text{close } t_0 \text{ in } \rho)^\ell &\text{ iff } \{t_0\} \subseteq \hat{C}(\ell) \wedge \rho R \hat{\rho} \end{aligned}$$

$$\boxed{\begin{aligned} \rho R \hat{\rho} &\text{ iff } \text{dom}(\rho) \subseteq \text{dom}(\hat{\rho}) \wedge \forall x \in \text{dom}(\rho) \, \forall t_x \, \forall \rho_x : \\ &(\rho(x) = \text{close } t_x \text{ in } \rho_x) \Rightarrow (t_x \in \hat{\rho}(x) \wedge \rho_x R \hat{\rho}) \end{aligned}}$$

For the other clauses just replace e with ie

Relation between environments

$$\rho \ R \ \hat{\rho} \text{ iff } \text{dom}(\rho) \subseteq \text{dom}(\hat{\rho}) \wedge \forall x \in \text{dom}(\rho) \ \forall t_x \ \forall \rho_x : \\ (\rho(x) = \text{close } t_x \text{ in } \rho_x) \Rightarrow (t_x \in \hat{\rho}(x) \wedge \rho_x \ R \ \hat{\rho})$$

- The concrete domain must be included in the abstract one (is an over-approximation)
- For all variables $x \in \text{dom}(\rho)$, if $\rho(x) = \text{close } t_x \text{ in } \rho_x$ then
 - $t_x \in \hat{\rho}(x)$: the function is predicted
 - $\rho_x \ R \ \hat{\rho}$: the captured environment ρ_x is also correct
 - This second condition is recursive: ρ_x may contain closures, whose environments must also be checked

The abstract environment over-approximates every concrete closure, including all environments they recursively capture

Relation between values

The definition of R can be split into two mutually recursive definitions

$$v \ V (\hat{\rho}, \hat{v}) \text{ iff } \forall t \ \forall \rho : (v = \text{close } t \text{ in } \rho) \Rightarrow (t \in \hat{v} \wedge \rho \ R \hat{\rho})$$

$$\rho \ R \hat{\rho} \text{ iff } \text{dom}(\rho) \subseteq \text{dom}(\hat{\rho}) \wedge \forall x \in \text{dom}(\rho) : \rho(x) \ V (\hat{\rho}, \hat{\rho}(x))$$

The two definitions of R are equivalent. We focus on $v \ V (\hat{\rho}, \hat{v})$

- The values must agree as well
- For all t , and for all ρ in $(v = \text{close } t \text{ in } \rho)$
 - $t_x \in \hat{v}$: the value is predicted
 - $\rho_x \ R \hat{\rho}$: the captured environment ρ_x is also correct
 - This check is recursive: environments may contain closures

A concrete value is a closure, so we must approximate both the function and all the environments it captures

Subject reduction

Theorem

If $\rho \ R \ \hat{\rho}$ and $\rho \vdash ie \rightarrow ie'$ then $(\hat{C}, \hat{\rho}) \models ie$ implies $(\hat{C}, \hat{\rho}) \models ie'$

Idea: if there is a possible evaluation of the program s.t. the function at a call point evaluates to some abstraction, then this abstraction has to be in the set of possible abstractions computed by the analysis

All acceptable analysis results remain acceptable under evaluation

Proof idea

We assume that $\rho \ R \ \hat{\rho}$ and $\rho \vdash ie \rightarrow ie'$ and prove $(\hat{C}, \hat{\rho}) \models ie'$ by **induction** on the structure of the **inference tree** for $\rho \vdash ie \rightarrow ie'$

For each rule, we show CFA is preserved

Subject reduction

Theorem

If $\rho R \hat{\rho}$ and $\rho \vdash ie \rightarrow ie'$ then $(\hat{C}, \hat{\rho}) \models ie$ implies $(\hat{C}, \hat{\rho}) \models ie'$

Proof

We assume that $\rho R \hat{\rho}$ and $\rho \vdash ie \rightarrow ie'$ and prove $(\hat{C}, \hat{\rho}) \models ie'$ by **induction** on the structure of the **inference tree** for $\rho \vdash ie \rightarrow ie'$

The case **[var]**. Here $\rho \vdash ie \rightarrow ie'$ is $\rho \vdash x^\ell \rightarrow v^\ell$ because $x \in \text{dom}(\rho) \wedge v = \rho(x)$

[Case $v = c$]: nothing to prove.

[Case $v = \text{close } t_0 \text{ in } \rho_0$].

- From $\rho R \hat{\rho}$ we get $v V (\hat{\rho}, \hat{\rho}(x))$ and hence $t_0 \in \hat{\rho}(x)$ and $\rho_0 R \hat{\rho}$.
- From $(\hat{C}, \hat{\rho}) \models ie$ we get $\hat{\rho}(x) \subseteq \hat{C}(\ell)$, and hence $t_0 \in \hat{C}(\ell)$.

Since $t_0 \in \hat{C}(\ell)$ and $\rho_0 R \hat{\rho}$ we have established $(\hat{C}, \hat{\rho}) \models ie'$.

[var] $(\hat{C}, \hat{\rho}) \models x^\ell$ iff $\hat{\rho}(x) \subseteq \hat{C}(\ell)$

$v V (\hat{\rho}, \hat{v})$ iff $\forall t \forall \rho : (v = \text{close } t \text{ in } \rho) \Rightarrow (t \in \hat{v} \wedge \rho R \hat{\rho})$

the value is already
predicted by the
analysis

Subject reduction

Theorem

If $\rho R \hat{\rho}$ and $\rho \vdash ie \rightarrow ie'$ then $(\hat{C}, \hat{\rho}) \models ie$ implies $(\hat{C}, \hat{\rho}) \models ie'$

Proof

The case **[fn]**. Here $\rho \vdash ie \rightarrow ie'$ is $\rho \vdash (\text{fn } x \Rightarrow e_0)^\ell \rightarrow (\text{close } (\text{fn } x \Rightarrow e_0) \text{ in } \rho_0)^\ell$ where $\rho_0 = \rho \upharpoonright FV(\text{fn } x \Rightarrow e_0)$

- From $(\hat{C}, \hat{\rho}) \models ie$ we get $\{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell)$.
- From $\rho R \hat{\rho}$ we get we immediately get $\rho_0 R \hat{\rho}$ and then we can establish $(\hat{C}, \hat{\rho}) \models ie'$.

the abstraction is already recorded in $\hat{C}(\ell)$

[fn] $(\hat{C}, \hat{\rho}) \models (\text{fn } x \Rightarrow e_0)^\ell$ iff $\{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell)$

[close] $(\hat{C}, \hat{\rho}) \models (\text{close } t_0 \text{ in } \rho)^\ell$ iff $\{t_0\} \subseteq \hat{C}(\ell) \wedge \rho R \hat{\rho}$

Subject reduction

Theorem

If $\rho R \hat{\rho}$ and $\rho \vdash ie \rightarrow ie'$ then $(\hat{C}, \hat{\rho}) \models ie$ implies $(\hat{C}, \hat{\rho}) \models ie'$

Proof

The case **[app₁]**. Here $\rho \vdash ie \rightarrow ie'$ is $\rho \vdash (ie_1 ie_2)^\ell \rightarrow (ie'_1 ie_2)^\ell$ because $\rho \vdash ie_1 \rightarrow ie'_1$

- From the induction hypothesis applied to $(\hat{C}, \hat{\rho}) \models ie_1$, $\rho R \hat{\rho}$ and $\rho \vdash ie_1 \rightarrow ie'_1$, we get $(\hat{C}, \hat{\rho}) \models ie'$.

This suffices to establish $(\hat{C}, \hat{\rho}) \models (ie'_1 ie_2)^\ell$.

Subject reduction

Theorem

If $\rho R \hat{\rho}$ and $\rho \vdash ie \rightarrow ie'$ then $(\hat{C}, \hat{\rho}) \models ie$ implies $(\hat{C}, \hat{\rho}) \models ie'$

Proof

The case **[app_{fn}]**.

Here $\rho \vdash ie \rightarrow ie'$ is $\rho \vdash (\text{close } (\text{fn } x \Rightarrow t_0^{\ell_0}) \text{ in } \rho_1)^{\ell_1} v_2^{\ell_2} \rightarrow (\text{bind } \rho_1[x \mapsto v_2] \text{ in } t_0^{\ell_0})^{\ell}$

- From $(\hat{C}, \hat{\rho}) \models ie$ we have $(\hat{C}, \hat{\rho}) \models (\text{close } (\text{fn } x \Rightarrow t_0^{\ell_0}) \text{ in } \rho_1)^{\ell_1}$, which yields

$\{\text{fn } x \Rightarrow t_0^{\ell_0}\} \subseteq \hat{C}(\ell_1)$ and $\rho R \hat{\rho}$.

- Further, we have $(\hat{C}, \hat{\rho}) \models v_2^{\ell_2}$

[close] $(\hat{C}, \hat{\rho}) \models (\text{close } t_0 \text{ in } \rho)^{\ell}$ iff $\{t_0\} \subseteq \hat{C}(\ell) \wedge \rho R \hat{\rho}$

[bind] $(\hat{C}, \hat{\rho}) \models (\text{bind } \rho \text{ in } it_0^{\ell_0})^{\ell}$ iff $(\hat{C}, \hat{\rho}) \models it_0^{\ell_0} \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell) \wedge \rho R \hat{\rho}$

[case $v_2 = c$]: immediate.

[case $v_2 = \text{close } t_2 \text{ in } \rho_2$]: it follows immediately from $(\hat{C}, \hat{\rho}) \models v_2^{\ell_2}$.

we simulate all possible function calls

Finally, the first universally quantified formula of the definition of $(\hat{C}, \hat{\rho}) \models ie$ gives

$(\hat{C}, \hat{\rho}) \models t_0^{\ell_0}$, $\hat{C}(\ell_2) \subseteq \hat{C}(\ell)$ and $(\rho_1[x \mapsto v_2]) R \hat{\rho}$ leading to establish $(\hat{C}, \hat{\rho}) \models ie'$.

Subject reduction

Theorem

If $\rho R \hat{\rho}$ and $\rho \vdash ie \rightarrow ie'$ then $(\hat{C}, \hat{\rho}) \models ie$ implies $(\hat{C}, \hat{\rho}) \models ie'$

Fact

$(\hat{C}, \hat{\rho}) \models it^{\ell_1}$ and $\hat{C}(\ell_1) \subseteq \hat{C}(\ell_2)$ then $(\hat{C}, \hat{\rho}) \models it^{\ell_2}$

Proof

The case **[bind₂]**. Here $\rho \vdash ie \rightarrow ie'$ is $\rho \vdash (\text{bind } \rho_1 \text{ in } v_1^{\ell_1})^\ell \rightarrow v_1^\ell$

• From $(\hat{C}, \hat{\rho}) \models ie$ we have $(\hat{C}, \hat{\rho}) \models v_1^{\ell_1}$ and $\hat{C}(\ell_1) \subseteq \hat{C}(\ell)$

The desired $(\hat{C}, \hat{\rho}) \models v_1^\ell$ follows from **Fact**.

This completes the proof. ■

[bind] $(\hat{C}, \hat{\rho}) \models (\text{bind } \rho \text{ in } it_0^{\ell_0})^\ell$ iff $(\hat{C}, \hat{\rho}) \models it_0^{\ell_0} \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell) \wedge \rho R \hat{\rho}$

CFA predictions are stable under evaluation

Back to Example 1: correctness of the solution

$$\begin{array}{c} ((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5 \\ \downarrow^* \\ (\text{Close}(\text{fn } y \Rightarrow y^3) \text{ in } [])^5 \end{array}$$

$$\begin{array}{c} (\hat{C}, \hat{\rho}) \models ((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5 \\ \downarrow \text{By Subject Reduction Theorem} \\ (\hat{C}, \hat{\rho}) \models (\text{Close}(\text{fn } y \Rightarrow y^3) \text{ in } [])^5 \end{array}$$

Exercise: Verify that the result (the closure) is acceptable

Back to Example 1: correctness of the solution (cont.)

the closure carries an empty environment

$$(\hat{C}_e, \hat{\rho}_e) \models (\text{Close}(\text{fn } y \Rightarrow y^3) \text{ in } [])^5$$

iff

$$\{(\text{fn } y \Rightarrow y^3)\} \subseteq \hat{C}(5) \wedge [] R \hat{\rho}$$

	$(\hat{C}_e, \hat{\rho}_e)$
1	$\{\text{fn } y \Rightarrow y^3\}$
2	$\{\text{fn } x \Rightarrow x^1\}$
3	$\emptyset$
4	$\{\text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$
x	$\{\text{fn } y \Rightarrow y^3\}$
y	$\emptyset$

✓

✓

trivially true

The solution $(\hat{C}_e, \hat{\rho}_e)$ correctly predicts the result of evaluation

$$\rho R \hat{\rho} \text{ iff } \text{dom}(\rho) \subseteq \text{dom}(\hat{\rho}) \wedge \forall x \in \text{dom}(\rho) \forall t_x \forall \rho_x : (\rho(x) = \text{close } t_x \text{ in } \rho_x) \Rightarrow (t_x \in \hat{\rho}(x) \wedge \rho_x R \hat{\rho})$$

$$[\text{close}] (\hat{C}, \hat{\rho}) \models (\text{close } t_0 \text{ in } \rho)^\ell \text{ iff } \{t_0\} \subseteq \hat{C}(\ell) \wedge \rho R \hat{\rho}$$

Program Analysis 2025-26

Lecture #19

Control Flow Analysis



UNIVERSITÀ
DI PISA

Existence

Existence of solutions

- So far we have only considered a procedure for validating whether or not a proposed solution $(\hat{C}, \hat{\rho})$ is acceptable
- Now it is natural to ask the following questions:
 - Does each expression e admit a Control Flow Analysis, i.e., does there **exist** $(\hat{C}, \hat{\rho})$ such that $(\hat{C}, \hat{\rho}) \models e$? **Yes**
- There may be **different possible analyses** for the same expression e
 - Does each expression e have a "least" CFA? **Yes**

Comparing analyses

Fewer possible flows $\Rightarrow$ more precise analysis

Which **partial order** to compare different CFAs?

The set of proposed solutions can be partially ordered by setting

pointwise inclusion

$$(\hat{C}_1, \hat{\rho}_1) \sqsubseteq (\hat{C}_2, \hat{\rho}_2) \text{ iff } \begin{cases} \forall \ell \in \mathbf{Lab} : \hat{C}_1(\ell) \subseteq \hat{C}_2(\ell) \\ \forall x \in \mathbf{Var} : \hat{\rho}_1(x) \subseteq \hat{\rho}_2(x) \end{cases}$$

smaller = more precise

This suffices for making the set of proposed solutions into a **complete lattice**

Comparing analyses

Back to our example $e \triangleq ((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

	$(\hat{C}_e, \hat{\rho}_e)$	$(\hat{C}'_e, \hat{\rho}'_e)$	$(\hat{C}''_e, \hat{\rho}''_e)$
1	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
2	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
3	$\emptyset$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
4	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
x	$\{\text{fn } y \Rightarrow y^3\}$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$
y	$\emptyset$	$\emptyset$	$\{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^3\}$

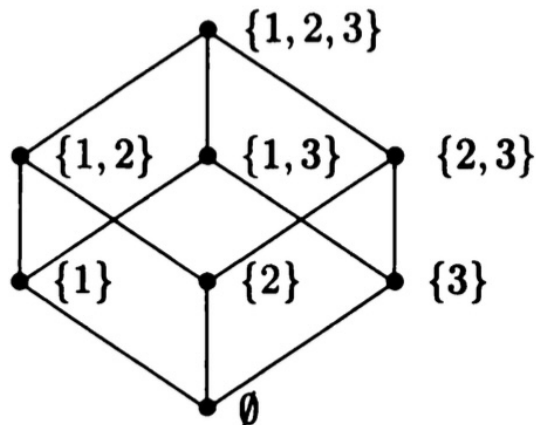
$$(\hat{C}_e, \hat{\rho}_e) \sqsubseteq (\hat{C}''_e, \hat{\rho}''_e)$$

Moore family

model intersection
property

A **Moore family** is a subset Y of a complete lattice $L = (L, \subseteq)$ that is closed under greatest lower bounds: $(\cap Y') \in Y$ for all $Y' \subseteq Y$

- It follows that a Moore family always contains a **least element** $\cap Y$, and a **greatest element** $\cap \emptyset$, which equals the greatest element $\top$ from L
- A Moore family is never empty



The complete lattice $(\wp(\{1,2,3\}), \subseteq)$

The subsets:

- $\{\{2\}, \{1,2\}, \{2,3\}, \{1,2,3\}\}$ and $\{\emptyset, \{1,2,3\}\}$ are both Moore families, while
- $\{\{1\}, \{2\}\}$ and $\{\emptyset, \{1\}, \{2\}, \{1,2\}\}$ are not

$\{1,2\} \cap \{2,3\} = \{2\} \in S \dots$
 $\cap \emptyset = \{1,2,3\} \in S$

$\emptyset \cap \{1,2,3\} = \emptyset \in S$ and
 $\cap \emptyset = \{1,2,3\} \in S$

$\{1\} \cap \{2\} = \emptyset \notin S$

$\cap \emptyset = \{1,2,3\} \notin S$

Existence of least CFA

The set of acceptable analyses is a complete lattice closed under intersection and it forms a **Moore family**

In particular, the set of acceptable analyses has a **least element**

Theorem

For all $ie \in \mathbf{IExp}$ the set $\{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$ is a **Moore family**

Proof idea

By **coinduction** on ie : we assume that $\forall i \in I : (\hat{C}_i, \hat{\rho}_i) \models ie$ and that $\sqcap_i (\hat{C}_i, \hat{\rho}_i) \models ie$

The least CFA is obtained by intersection

Existence

Theorem

For all $ie \in \mathbf{IExp}$ the set $\{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$ is a Moore family

taking the intersection over an empty set yields a valid analysis

Corollaries:

- all intermediate expressions admit a CFA: let $Y' = \emptyset$, then $(\sqcap Y') \in \{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$. So there is at least an analysis of ie
- all intermediate expressions have a least CFA: let $Y' = \{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$, then $(\sqcap Y') \in \{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models ie\}$. So it will also be an analysis of ie . Clearly $(\sqcap Y') \sqsubseteq (\hat{C}, \hat{\rho})$ for all other analyses $(\hat{C}, \hat{\rho})$ of ie so it is the least analysis result.

Towards the least solution

Back to our example $e \triangleq ((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

The last two are different solutions w.r.t. the previously introduced ones

	$(\hat{C}_e, \hat{\rho}_e)$	$(\hat{C}'_e, \hat{\rho}'_e)$	$(\hat{C}''_e, \hat{\rho}''_e)$
1	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
2	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1\}$
3	$\emptyset$	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } y \Rightarrow y^3\}$
4	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
x	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\}$
y	$\emptyset$	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } y \Rightarrow y^3\}$

- $(\hat{C}'_e, \hat{\rho}'_e)$ and $(\hat{C}''_e, \hat{\rho}''_e)$ are acceptable, but neither is the least solution
- Their pointwise “intersection” $(\hat{C}'_e \sqcap \hat{C}''_e, \hat{\rho}'_e \sqcap \hat{\rho}''_e) = (\hat{C}_e, \hat{\rho}_e)$ is still a solution
- It is smaller, hence more precise, and is the **least acceptable solution**

Coinduction vs Induction

If acceptability were defined **inductively** with an acceptability relation $\models'$ then there exists an expression e_* such that acceptable analyses are not closed under intersection

Idea:

- If there are circular dependencies, it cannot resolve anything and may therefore return an empty solution
- Inductive definitions are too strict: they exclude valid over-approximations

Coinduction vs Induction

If acceptability were defined **inductively** with an acceptability relation $\models'$ then there exists an expression e_* such that acceptable analyses are not closed under intersection

Proposition

There exists $e_* \in \mathbf{Exp}$ such that the set $\{(\hat{C}, \hat{\rho}) \mid (\hat{C}, \hat{\rho}) \models' e_*\}$ is **not** a Moore family

Proof idea: Use a very simple but “tricky” program and examine its CFA solutions:

- with the greatest fixed point $\rightarrow$ everything is fine
- with the least fixed point $\rightarrow$ we get a “bad” set ($\emptyset$)

We conclude it is not a Moore family

How to compute the least solution?

Least solutions (w.r.t. $\sqsubseteq$) exist, but how can we compute them?

Naive approach:

- 1.enumerate all candidate solutions
- 2.determine whether they are indeed solutions,
- 3.if so, take the **greatest lower bound** w.r.t. the others found so far

How to compute the least solution?

Least solutions (w.r.t. $\sqsubseteq$) exist, but how can we compute them?

Alternative:

1. derive a finite **set of constraints** from the program, such as the ones of the form $lhs \sqsubseteq rhs$,
2. take the **least solution** to this system of constraints (using fixpoint computation)

First, we will introduce a more computationally oriented specification: $\models_s$

Syntax directed specification

Syntax directed CFA

Syntax directed specification

Look for a more computationally oriented specification

Goal: analyze each function body *at most once* rather than each time the function could be applied

Idea: one way to obtain this is to analyse each function body *exactly once*, at its definition

This is done in the so-called **syntax directed O-CFA analysis**:

- acceptability relation: $\models_s$
- judgments: $(\hat{C}, \hat{\rho}) \models_s e_*$

Since semantic correctness was dealt with for the previous CFA there is no longer any need to consider intermediate expressions

Syntax directed specification

If we want to analyze a given expression e_* , we reformulate the specification of $(\hat{C}, \hat{\rho}) \models e_*$ into the more computationally oriented specification $(\hat{C}, \hat{\rho}) \models_s e_*$ such that:

$$(\hat{C}, \hat{\rho}) \models_s e_* \Rightarrow (\hat{C}, \hat{\rho}) \models e_* \text{ (if restricted to program terms)}$$

i.e., $\models_s$ is a safe approximation of $\models$: all solutions accepted by $\models_s$ are also valid for $\models$

We need to focus on solutions satisfying some additional properties (e.g., that only program fragments of e_* appear in the range of $\hat{C}$ and $\hat{\rho}$)

Clause for functions

$$[fn] (\hat{C}, \hat{\rho}) \models_s (fn\ x \Rightarrow e_0)^\ell \text{ iff } \{fn\ x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge (\hat{C}, \hat{\rho}) \models_s e_0$$

NEW: Function body e_0 is now analyzed at definition time, even if the function is never applied

Example

$$e \triangleq (fn\ x \Rightarrow x^1)^2$$

$$\hat{C}(2) \supseteq \{fn\ x \Rightarrow x^1\} \text{ and } \hat{\rho}(x) \subseteq \hat{C}(1)$$

Clause for functions (cont.)

$$[fn] (\hat{C}, \hat{\rho}) \models_s (fn\ x \Rightarrow e_0)^\ell \text{ iff } \{fn\ x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge$$

$$(\hat{C}, \hat{\rho}) \models_s e_0$$

Example

$$e \triangleq (fn\ x \Rightarrow ((fn\ y \Rightarrow y^2)^3\ x^4)^5)^6$$

$$(\hat{C}, \hat{\rho}) \models_s e \text{ iff } \{fn\ x \Rightarrow ((fn\ y \Rightarrow y^2)^3\ x^4)^5\} \subseteq \hat{C}(2) \wedge$$

$$(\hat{C}, \hat{\rho}) \models_s ((fn\ y \Rightarrow y^2)^3\ x^4)^5 \text{ iff } (\hat{C}, \hat{\rho}) \models_s (fn\ y \Rightarrow y^2)^3 \wedge (\hat{C}, \hat{\rho}) \models_s x^4 \wedge$$

$$(\hat{C}, \hat{\rho}) \models_s (fn\ y \Rightarrow y^2)^3 \text{ iff } \{fn\ y \Rightarrow y^2\} \subseteq \hat{C}(3) \wedge (\hat{C}, \hat{\rho}) \models_s y^2$$

$$(\hat{C}, \hat{\rho}) \models_s x^4 \text{ iff } \hat{\rho}(x) \subseteq \hat{C}(4) \quad (\hat{C}, \hat{\rho}) \models_s y^2 \text{ iff } \hat{\rho}(y) \subseteq \hat{C}(2)$$

$$\{fn\ y \Rightarrow y^2\} \in \hat{C}(3) : \hat{C}(4) \subseteq \hat{\rho}(y) \wedge \hat{C}(2) \subseteq \hat{C}(5)$$

These additional constraints are not required in $\models$, where only reachable behaviors matter

Clause for recursive functions

$$[fun] (\hat{C}, \hat{\rho}) \models_s (\text{fun } f \ x \Rightarrow e_0)^\ell \text{ iff } \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge$$

$$(\hat{C}, \hat{\rho}) \models_s e_0$$
$$\{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{\rho}(f)$$

NEW:

- **Function body** e_0 is now analysed at definition time, even if the function is never applied
- checks on the **recursive binding** for [fun] is included: self-binding is enforced immediately

See **Example 2** next

Clause for function application

$$[app] (\hat{C}, \hat{\rho}) \models_s (t_1^{\ell_1} t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_s t_2^{\ell_2} \wedge$$

$$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\ \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$$

$$(\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\ \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell))$$

NEW: The body constraints have already been generated by [fn]

All other rules are unchanged: just replace $\models$ with $\models_s$

See **Example 2** next

Syntax directed CFA

- [con] $(\hat{C}, \hat{\rho}) \models_s c^\ell$ always
- [var] $(\hat{C}, \hat{\rho}) \models_s x^\ell$ iff $\hat{\rho}(x) \subseteq \hat{C}(\ell)$
- [fn] $(\hat{C}, \hat{\rho}) \models_s (\text{fn } x \Rightarrow e_0)^\ell$
iff $\{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge$
 $(\hat{C}, \hat{\rho}) \models_s e_0$
- [fun] $(\hat{C}, \hat{\rho}) \models_s (\text{fun } f x \Rightarrow e_0)^\ell$
iff $\{\text{fun } f x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge$
 $(\hat{C}, \hat{\rho}) \models_s e_0 \wedge \{\text{fun } f x \Rightarrow e_0\} \subseteq \hat{\rho}(f)$
- [app] $(\hat{C}, \hat{\rho}) \models_s (t_1^{\ell_1} t_2^{\ell_2})^\ell$
iff $(\hat{C}, \hat{\rho}) \models_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_s t_2^{\ell_2} \wedge$
 $(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell) \wedge$
 $(\forall (\text{fun } f x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell))$
- [if] $(\hat{C}, \hat{\rho}) \models_s (\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell$
iff $(\hat{C}, \hat{\rho}) \models_s t_0^{\ell_0} \wedge$
 $(\hat{C}, \hat{\rho}) \models_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_s t_2^{\ell_2} \wedge$
 $\hat{C}(\ell_1) \subseteq \hat{C}(\ell) \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$
- [let] $(\hat{C}, \hat{\rho}) \models_s (\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^\ell$
iff $(\hat{C}, \hat{\rho}) \models_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_s t_2^{\ell_2} \wedge$
 $\hat{C}(\ell_1) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$
- [op] $(\hat{C}, \hat{\rho}) \models_s (t_1^{\ell_1} \text{ op } t_2^{\ell_2})^\ell$ iff $(\hat{C}, \hat{\rho}) \models_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_s t_2^{\ell_2}$

Back to Example 2

$\text{loop} \triangleq (\text{let } g = (\text{fun } f \text{ } x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4)^5 \text{ in } (g^6 (\text{fn } z \Rightarrow z^7)^8)^9)^{10}$

$\hat{C}_{lp}(1) = \{f\}$	$\hat{C}_{lp}(6) = \{f\}$	$\hat{\rho}_{lp}(f) = \{f\}$
$\hat{C}_{lp}(2) = \emptyset$	$\hat{C}_{lp}(7) = \emptyset$	$\hat{\rho}_{lp}(g) = \{f\}$
$\hat{C}_{lp}(3) = \{\text{id}_y\}$	$\hat{C}_{lp}(8) = \{\text{id}_z\}$	$\hat{\rho}_{lp}(x) = \{\text{id}_y, \text{id}_z\}$
$\hat{C}_{lp}(4) = \emptyset$	$\hat{C}_{lp}(9) = \emptyset$	$\hat{\rho}_{lp}(y) = \emptyset$
$\hat{C}_{lp}(5) = \{f\}$	$\hat{C}_{lp}(10) = \emptyset$	$\hat{\rho}_{lp}(z) = \emptyset$

$f \triangleq \text{fun } f \text{ } x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4$
 $\text{id}_y \triangleq \text{fn } y \Rightarrow y^2$
 $\text{id}_z \triangleq \text{fn } z \Rightarrow z^7$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s \text{loop}$ iff $(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s f^5 \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s g^6 (\text{fn } z \Rightarrow z^7)^8$
 $\wedge \hat{C}_{lp}(5) \subseteq \hat{\rho}_{lp}(g) \wedge \hat{C}_{lp}(9) \subseteq \hat{C}_{lp}(10)$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s f^5$ iff $(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s (f^1 (\text{fn } y \Rightarrow y^2)^3)^4 \wedge \{f\} \subseteq \hat{C}_{lp}(5) \wedge \{f\} \subseteq \hat{\rho}_{lp}(x)$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s (f^1 (\text{fn } y \Rightarrow y^2)^3)^4$ iff $(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s f^1 \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s (\text{id}_y)^3$
 $\wedge \hat{C}_{lp}(3) \subseteq \hat{\rho}_{lp}(x) \wedge \hat{C}_{lp}(4) \subseteq \hat{C}_{lp}(4)$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s (\text{id}_y)^3$ iff $\{\text{id}_y\} \subseteq \hat{C}_{lp}(3) \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s y^2$

Back to Example 2

$\text{loop} \triangleq (\text{let } g = (\text{fun } f \ x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4)^5 \text{ in } (g^6 (\text{fn } z \Rightarrow z^7)^8)^9)^{10}$

$\hat{C}_{lp}(1) = \{f\}$	$\hat{C}_{lp}(6) = \{f\}$	$\hat{\rho}_{lp}(f) = \{f\}$
$\hat{C}_{lp}(2) = \emptyset$	$\hat{C}_{lp}(7) = \emptyset$	$\hat{\rho}_{lp}(g) = \{f\}$
$\hat{C}_{lp}(3) = \{\text{id}_y\}$	$\hat{C}_{lp}(8) = \{\text{id}_z\}$	$\hat{\rho}_{lp}(x) = \{\text{id}_y, \text{id}_z\}$
$\hat{C}_{lp}(4) = \emptyset$	$\hat{C}_{lp}(9) = \emptyset$	$\hat{\rho}_{lp}(y) = \emptyset$
$\hat{C}_{lp}(5) = \{f\}$	$\hat{C}_{lp}(10) = \emptyset$	$\hat{\rho}_{lp}(z) = \emptyset$

$f \triangleq \text{fun } f \ x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4$
 $\text{id}_y \triangleq \text{fn } y \Rightarrow y^2$
 $\text{id}_z \triangleq \text{fn } z \Rightarrow z^7$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s \text{loop}$ iff $(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s f^5 \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s g^6 (\text{fn } z \Rightarrow z^7)^8$
 $\wedge \hat{C}_{lp}(5) \subseteq \hat{\rho}_{lp}(g) \wedge \hat{C}_{lp}(9) \subseteq \hat{C}_{lp}(10)$

using [app] and
that $\hat{C}(6) = \{f\}$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s g^6 (\text{fn } z \Rightarrow z^7)^8$ iff $(\hat{C}_{lp}, \hat{\rho}_{lp}) \models g^6 \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s (\text{id}_z)^8$
 $\wedge \hat{C}_{lp}(8) \subseteq \hat{\rho}_{lp}(x) \wedge \hat{C}_{lp}(4) \subseteq \hat{C}_{lp}(9)$

$(\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s (\text{id}_z)^8$ iff $\{\text{id}_z\} \subseteq \hat{C}_{lp}(8) \wedge (\hat{C}_{lp}, \hat{\rho}_{lp}) \models_s z^7$

Immediate

Back to Example 2

loop $\triangleq$ (let g = (fun f x => (f¹ (fn y => y²)³)⁴)⁵
in (g⁶ (fn z => z⁷)⁸)⁹)¹⁰

We have proved that $(\hat{C}_{lp}, \hat{\rho}_{lp})$ is an acceptable guess for $\models_s$

$\hat{C}_{lp}(1)$	=	{f}	$\hat{C}_{lp}(6)$	=	{f}	$\hat{\rho}_{lp}(f)$	=	{f}
$\hat{C}_{lp}(2)$	=	$\emptyset$	$\hat{C}_{lp}(7)$	=	$\emptyset$	$\hat{\rho}_{lp}(g)$	=	{f}
$\hat{C}_{lp}(3)$	=	{id _y }	$\hat{C}_{lp}(8)$	=	{id _z }	$\hat{\rho}_{lp}(x)$	=	{id _y , id _z }
$\hat{C}_{lp}(4)$	=	$\emptyset$	$\hat{C}_{lp}(9)$	=	$\emptyset$	$\hat{\rho}_{lp}(y)$	=	$\emptyset$
$\hat{C}_{lp}(5)$	=	{f}	$\hat{C}_{lp}(10)$	=	$\emptyset$	$\hat{\rho}_{lp}(z)$	=	$\emptyset$

$f \triangleq \text{fun } f \ x \Rightarrow (f^1 (\text{fn } y \Rightarrow y^2)^3)^4$
 $\text{id}_y \triangleq \text{fn } y \Rightarrow y^2$
 $\text{id}_z \triangleq \text{fn } z \Rightarrow z^7$

No circular dependency => no need of coinduction

Observation

We can take $\models_s$ as the **largest relation satisfying the specification**

However, given the syntax directed nature of the specification (the relation is fully determined by the structure of expressions), there is in fact **only one** relation that satisfies the specification

unique

Hence it would be technically correct, but intuitively misleading, to claim that we take the **least relation that satisfies the specification**, because it suggests a choice among many relations

Before: many acceptable relations $\rightarrow$ take the least

Now: only one relation $\rightarrow$ no minimisation needed

Whether or not $\models_s$ is defined inductively or conductively, the same relation is defined. This is typical of syntax-directed definitions

Exercise

Let $\models'_s$ and $\models''_s$ be two relations satisfying the syntax-directed CFA clauses. Show, by *structural induction* on e that

$$(\hat{C}, \hat{\rho}) \models'_s e \text{ iff } (\hat{C}, \hat{\rho}) \models''_s e$$

[Case c^ℓ]: trivial

[Case x^ℓ]: $(\hat{C}, \hat{\rho}) \models'_s x^\ell$ iff $\hat{\rho}(x) \subseteq \hat{C}(\ell)$ and $(\hat{C}, \hat{\rho}) \models''_s x^\ell$ iff $\hat{\rho}(x) \subseteq \hat{C}(\ell)$

Both relations satisfy the same clause on $(\hat{C}, \hat{\rho})$, so they are equivalent.

[Case $(f\ x \Rightarrow t_0^{\ell_0})^\ell$]: $(\hat{C}, \hat{\rho}) \models'_s (f\ x \Rightarrow t_0^{\ell_0})^\ell$ iff $\{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge (\hat{C}, \hat{\rho}) \models'_s e_0$,
and $(\hat{C}, \hat{\rho}) \models''_s (f\ x \Rightarrow t_0^{\ell_0})^\ell$ iff $\{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge (\hat{C}, \hat{\rho}) \models''_s e_0$

In each case, both $\models'_s$ and $\models''_s$ satisfy the same defining clause. Hence their equivalence reduces to the equivalence of the premises, which follows by the induction hypothesis $((\hat{C}, \hat{\rho}) \models'_s t_0^{\ell_0} \text{ iff } (\hat{C}, \hat{\rho}) \models''_s t_0^{\ell_0})$.

The remaining cases are analogous.

Observation on the exercise

If $\models'$ and $\models''$ are two relations satisfying the **non** syntax-directed CFA clauses. Then, we cannot establish that

$$(\hat{C}, \hat{\rho}) \models' e \text{ iff } (\hat{C}, \hat{\rho}) \models'' e$$

$a^3 \triangleq (0^1 0^2)^3$: and suppose to have that

- $(\text{fn } x \Rightarrow a^4) \in \hat{C}(1)$,
- $\hat{C}(2) = \hat{C}(2) = \rho(x) = \emptyset$

The function in $\hat{C}(1)$ introduces a dependency outside the syntactic structure

$$(\hat{C}, \hat{\rho}) \models' a^3 \text{ iff } (\hat{C}, \hat{\rho}) \models' 0^1 \wedge (\hat{C}, \hat{\rho}) \models' 0^2 \wedge (\hat{C}, \hat{\rho}) \models' a^4 \wedge \hat{C}(2) \subseteq \rho(x) \wedge \hat{C}(4) \subseteq \hat{C}(3)$$

The only critical point is that $(\hat{C}, \hat{\rho}) \models' a^3$ iff $(\hat{C}, \hat{\rho}) \models' a^4$

This equivalence holds true whether or not you include 'a' in the relation, i.e., $(\hat{C}, \hat{\rho}) \models' a^3$ either is true or is false.

Therefore, we can have two different relations that satisfy the same clauses: one that includes and one that does not.

Preservation of solutions

The specification of the analysis uses **potentially infinite value spaces** although this is not really necessary

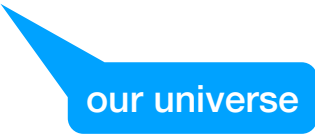
We can easily restrict ourselves to entities occurring in the original expression and this forms the basis for relating the results of the analysis of syntax directed CFA to those of the previous CFA

We obtain finite sets

Lab_{*} $\subseteq$ **Lab**: finite set of labels occurring in e_* ,

Var_{*} $\subseteq$ **Var**: finite set of variables occurring in e_*

Term_{*} $\subseteq$ **Term**: finite set of subterms occurring in e_*



our universe

Preservation of solutions

We introduce the solution $(\hat{C}^\top, \hat{\rho}^\top)$

the largest possible solution
within the bounds of the
expression

$$\hat{C}^\top(\ell) = \begin{cases} \emptyset & \text{if } \ell \notin \mathbf{Lab}_* \\ \mathbf{Term}_* & \text{if } \ell \in \mathbf{Lab}_* \end{cases} \quad \hat{\rho}^\top(x) = \begin{cases} \emptyset & \text{if } x \notin \mathbf{Var}_* \\ \mathbf{Term}_* & \text{if } x \in \mathbf{Var}_* \end{cases}$$

$(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}^\top, \hat{\rho}^\top)$: intuitively, $(\hat{C}, \hat{\rho})$ is concerned **only with subterms occurring in the expression e_***

We can directly reformulate the CFA in terms of restricted sets

$$(\hat{C}, \hat{\rho}) \in \widehat{\mathbf{Cache}_*} \times \widehat{\mathbf{Env}_*}$$

$$\widehat{\mathbf{Cache}_*} : \mathbf{Lab}_* \rightarrow \widehat{\mathbf{Val}_*}$$

$$\widehat{\mathbf{Env}_*} : \mathbf{Var}_* \rightarrow \widehat{\mathbf{Val}_*}$$

$$\widehat{\mathbf{Val}_*} = \wp(\mathbf{Term}_*)$$

Why syntax-directed CFA is correct

check when needed

Two views of the analysis

- Original CFA $\models$: checks correctness during the simulation of the evaluation, i.e., when analysing function application
- Syntax-directed CFA $\models_s$: analyzes all subterms upfront, so when we reach a call site, everything has already been analysed

As a consequence:

check in advance: nothing missing at call time

$$(\hat{C}, \hat{\rho}) \models_s e_* \Rightarrow (\hat{C}, \hat{\rho}) \models e_* \text{ (if restricted to program terms)}$$

Preservation of solutions

- We can prove that all the solutions to $\models_s e_*$ that are "less than" $(\hat{C}^\top, \hat{\rho}^\top)$ are also solutions to $\models e_*$
- Syntax-directed CFA is a **sound approximation** of the original CFA when restricted to program terms

boundedness condition

Proposition

If $(\hat{C}, \hat{\rho}) \models_s e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^\top, \hat{\rho}_*^\top)$ then $(\hat{C}, \hat{\rho}) \models e_*$

Idea: syntax-directed CFA already enforces what the original CFA requires

Proof idea:

- Restrict to program terms and use coinduction
- Show that every syntax-directed clause implies the original one
- Therefore, the solution also holds for the original CFA

Preservation of solutions

Proposition

If $(\hat{C}, \hat{\rho}) \models_s e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}^\top, \hat{\rho}^\top)$ then $(\hat{C}, \hat{\rho}) \models e_*$

Proof sketch:

1. Restrict to program terms $\mathbf{Exp}_*$, the set of all sub-expressions occurring in e_*
2. By syntax-directedness: $(\hat{C}, \hat{\rho}) \models_s e_* \Rightarrow \forall e \in \mathbf{Exp}_* : (\hat{C}, \hat{\rho}) \models_s e$
3. By coinduction, show that every syntax-directed clause (its *rhs*) implies the original one. Intuitively:
 - [con], [var], [if], [let], [op]: immediate, since identical.
 - [fn], [fun]: syntax-directed rules are stronger than the original ones.
 - [app]: $\models_s$ already enforces all constraints required by $\models$ (function bodies are already analyzed by (2)).

So, $\models_s$ captures all semantically required flows on reachable terms

Does the converse hold?

$(\hat{C}, \hat{\rho}) \models_s e_* \stackrel{?}{\Leftarrow} (\hat{C}, \hat{\rho}) \models e_*$ (if restricted to program terms)

✗ No

f never applied
program evaluates to 0

$\text{loop} \triangleq \text{let } f = \text{fn } x \Rightarrow (x \ x) \ (* \text{ problematic body } *)$
in 0

- Original CFA: checks only when needed
- Syntax-directed CFA: checks all bodies
- Unreachable code is ignored by $\models$, but not by $\models_s$
- $\models_s \subseteq \models$: Syntax-directed CFA is more restrictive

Existence

Theorem

$\{(\hat{C}, \hat{\rho}) \in (\widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_*) \mid (\hat{C}, \hat{\rho}) \models_s e\}$ is a Moore family

Corollaries:

- Each expression e_* admits a CFA that is "less than" $(\hat{C}^\top, \hat{\rho}^\top)$
- Each expression e_* has a "least" CFA that is "less than" $(\hat{C}^\top, \hat{\rho}^\top)$

This means that the properties obtained for the original analysis holds for the syntax-directed one

Existence

Theorem

$\{(\hat{C}, \hat{\rho}) \in (\widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_*) \mid (\hat{C}, \hat{\rho}) \models_s e\}$ is a Moore family

Proof:

$(\hat{C}_*^\top, \hat{\rho}_*^\top)$ is the greatest element of $\widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_*$. It is immediate to show that:

(a) $(\hat{C}_*^\top, \hat{\rho}_*^\top) \models_s e$

(b) $(\hat{C}_1, \hat{\rho}_1) \models_s e$ and $(\hat{C}_2, \hat{\rho}_2) \models_s e$ then $((\hat{C}_1, \hat{\rho}_1) \cap (\hat{C}_2, \hat{\rho}_2)) \models_s e$

for all **subexpressions** e of e_* , by means of *structural induction* on e . Hence:

(a) and (b) also for $e = e_*$.

Next consider some

$$Y \subseteq \{(\hat{C}, \hat{\rho}) \in (\widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_*) \mid (\hat{C}, \hat{\rho}) \models_s e\}$$

and note that one can write

$Y = \{(\hat{C}_i, \hat{\rho}_i) \mid i \in \{1, \dots, n\}\}$ for some $n \geq 0$ since $(\widehat{\mathbf{Cache}}_* \times \widehat{\mathbf{Env}}_*)$ is finite. So

$$\sqcap Y \in \{(\hat{C}_i, \hat{\rho}_i) \mid i \in \{1, \dots, n\}\}$$

follows from (a) and (b), because

$$\sqcap Y = (\hat{C}_*^\top, \hat{\rho}_*^\top) \sqcap (\hat{C}_1, \hat{\rho}_1) \sqcap \dots \sqcap (\hat{C}_n, \hat{\rho}_n)$$

Exercise 1

Modify the original Control Flow Analysis to take account of the left to right evaluation order imposed by a call-by-value semantics:

- in the clause [app] there is no need to analyse the operand if the operator cannot produced any closures

Clause for function application

$$\begin{aligned} [app] \ (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff } & (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models t_2^{\ell_2} \wedge \\ & (\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\ & \quad (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \\ & \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge \\ & (\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\ & \quad (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge \\ & \quad \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge \\ & \quad \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{\rho}(f)) \end{aligned}$$

New clause for function application

$$[app] (\hat{C}, \hat{\rho}) \models (t_1^{\ell_1} t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge$$

guard operand analysis

$$(\exists (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) \Rightarrow (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}) \wedge$$
$$\dots$$

If the only abstract values are only abstractions, we can use also

$$((\hat{C}(\ell_1) \cap \widehat{Val}) \neq \emptyset \Rightarrow (\hat{C}, \hat{\rho}) \models t_2^{\ell_2}) \wedge \dots$$

Example: Try to find a program where the modified analysis accepts analysis results rejected by the original analysis

$e \triangleq 4 \text{ (fn } y \Rightarrow y)$: the empty analysis is acceptable for the modified analysis but not for the original one

Reachability analysis

- The syntax directed analysis analyzes each subexpression of e_* “exactly once”, rather than “at most once” as really called for
- This may introduce constraints from code that is never executed
- How to amend this?

Goal:

- only expressions that are reachable will actually be analysed, and
- the same expression is analysed “at most once”

Idea:

1. modify the analysis to track the reachability of subexpressions explicitly
2. modify it such that subexpressions are analysed “at most once”

Reachability analysis

How?

By introducing a new abstract reachability component

$$\hat{R} \in \mathbf{Reach} : \mathbf{Lab} \rightarrow \wp(\{\mathbf{on}\})$$

and modifying the syntax directed analysis

$$(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e$$

$\{\mathbf{on}\} \subseteq \hat{R}(\ell)$ means: the expression at label ℓ is reachable

- $\hat{R}(\ell) = \emptyset$: the program point ℓ is not reachable (t_ℓ must be analysed)
- $\{\mathbf{on}\} \subseteq \hat{R}(\ell_0)$: the program point ℓ is reachable
- $(\text{fn } x \Rightarrow t_0^{\ell_0})^\ell$ has $\{\mathbf{on}\} \subseteq \hat{R}(\ell_0)$ iff the function is indeed applied somewhere
- A function body is analyzed only if it is reachable: the "recursive call"
 $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_0^{\ell_0}$ is performed if and only if $\{\mathbf{on}\} \subseteq \hat{R}(\ell_0)$
- Hence, function bodies are analyzed “at most once”

Clause for functions

$$[fn] (\hat{C}, \hat{\rho}, \hat{R}) \models'_s (\text{fn } x \Rightarrow e_0)^\ell \text{ iff } \{\text{fn } x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge$$
$$(\{\mathbf{on}\} \subseteq \hat{R}(\ell) \Rightarrow \wedge (\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_0)$$

NOW: Function body e_0 is analysed at definition time, but only if the function is applied somewhere

Clause for variables

$$[var] (\hat{C}, \hat{\rho}, \hat{R}) \models'_s x^\ell \text{ iff } (\{\mathbf{on}\} \subseteq \hat{R} \Rightarrow \rho(x) \subseteq \hat{C}(\ell))$$

NOW: x is analyzed only if reachable

Clause for function application

$$[app] (\hat{C}, \hat{\rho}, \hat{R}) \models'_s (t_1^{\ell_1} t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_2^{\ell_2} \wedge$$

$$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$$

$$(\{\mathbf{on}\} \subseteq \hat{R}(\ell) \Rightarrow \{\mathbf{on}\} \subseteq \hat{R}(\ell_0)) \wedge$$

$$(\{\mathbf{on}\} \subseteq \hat{R}(\ell) \Rightarrow \hat{C}(\ell_2) \subseteq \hat{\rho}(x)) \wedge$$

$$(\{\mathbf{on}\} \subseteq \hat{R}(\ell) \Rightarrow \hat{C}(\ell_0) \subseteq \hat{C}(\ell))) \wedge \dots$$

NEW: If the function possibly occurs in an application, then this means it is reachable

All other rules are unchanged: just replace $\models_s$ with $\models'_s$

Example

$$e_* \triangleq (\text{fn } x \Rightarrow ((\text{fn } y \Rightarrow y^1)^2 x^3)^4)^5$$

We start knowing that $\hat{R}(\ell_*) = \hat{R}(5) = \emptyset$

$$(\hat{C}, \hat{\rho}, \hat{R}) \vDash'_s e_*$$

As a consequence, $\{\text{fn } x \Rightarrow ((\text{fn } y \Rightarrow y^1)^2 x^3)^4\} \subseteq \hat{C}(5)$, but we cannot analyse the body with label 4, since $\hat{R}(5) = \emptyset$

Example

$$e_* \triangleq ((\text{fn } x \Rightarrow ((\text{fn } y \Rightarrow y^1)^2 x^3)^4)^5 \textcolor{red}{7})^6$$

$$(\hat{C}, \hat{\rho}, \hat{R}) \vDash'_s e_*$$

As a consequence, $\{\text{fn } x \Rightarrow ((\text{fn } y \Rightarrow y^1)^2 x^3)^4\} \subseteq \hat{C}(5)$

This triggers the propagation in the [app] clause that enforces $\{\mathbf{on}\} \subseteq \hat{R}(4)$, so allowing to analyse the body of the function

Reachability

Proposition

If $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_*^{\ell_*}$ and $\{\mathbf{on}\} \subseteq \hat{R}(\ell_*)$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^\top, \hat{\rho}_*^\top)$ then $(\hat{C}, \hat{\rho}) \models t_*^{\ell_*}$

Proof sketch:

1. Restrict to program terms, $\mathbf{Exp}_*$, the set of all reachable subexpressions occurring in e_*
2. By syntax-directedness: $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_*^{\ell_*} \Rightarrow \forall e \in \mathbf{Exp}_* : (\{\mathbf{on}\} \subseteq \hat{R}(\ell_*) \Rightarrow (\hat{C}, \hat{\rho}) \models_s e)$
3. By coinduction, show that every syntax-directed clause (its *rhs*) implies the original one.

Intuitively:

- [con], [var], [if], [let], [op]: immediate, since identical.
- [fn],[fun]: syntax-directed rules with the additional constraint on $\hat{R}$ are equivalent to the original ones, on reachable terms.
- [app]: $\models_s$ enforces all constraints required by $\models$ on reachable terms, and propagates reachability information $\hat{R}$.

So, $\models'_s$ captures all semantically required flows on reachable terms, provided that $\hat{R}$ correctly over-approximates reachability

Reachability

Proposition

If $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_*^{\ell_*}$ and $\{\mathbf{on}\} \subseteq \hat{R}(\ell_*)$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^{\top}, \hat{\rho}_*^{\top})$ then $(\hat{C}, \hat{\rho}) \models t_*^{\ell_*}$

Does the converse hold?

1. $(\hat{C}, \hat{\rho}) \models e_*$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$
 2. $(\hat{C}, \hat{\rho}) \models e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^{\top}, \hat{\rho}_*^{\top})$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$
1. does not hold:
- the original CFA can introduce and treat consistently “spurious” abstract values, without loosing acceptability, since it provides an over-approximation
 - the reachability CFA, instead, even forcing to **on** the “spurious” value, cannot analyze the abstraction (and its body), since it do not occur in e_*
2. $(\hat{C}, \hat{\rho}) \models e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^{\top}, \hat{\rho}_*^{\top})$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$

Reachability

Proposition

If $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s t_*^{\ell_*}$ and $\{\mathbf{on}\} \subseteq \hat{R}(\ell_*)$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^{\top}, \hat{\rho}_*^{\top})$ then $(\hat{C}, \hat{\rho}) \models t_*^{\ell_*}$

Does the converse hold?

1. $(\hat{C}, \hat{\rho}) \models e_*$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$
2. $(\hat{C}, \hat{\rho}) \models e_*$ and $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^{\top}, \hat{\rho}_*^{\top})$ then $(\hat{C}, \hat{\rho}, \hat{R}) \models'_s e_*$ for some $\hat{R}$
2. Since $(\hat{C}, \hat{\rho}) \sqsubseteq (\hat{C}_*^{\top}, \hat{\rho}_*^{\top})$ then no “spurious” value can be added. In this case the bodies analyzed in the two analyses are the same, provided a suitable $\hat{R}$ is provided

Program Analysis 2025-26

Lecture #20

Control Flow Analysis



UNIVERSITÀ
DI PISA

Constraint Based 0-CFA Analysis

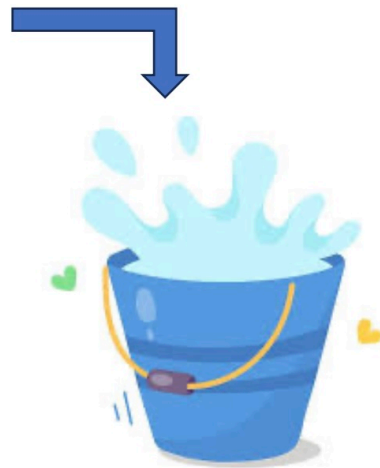
Static Analysis: a constraint-based approach

The basic idea is to separate the analysis specification from algorithmic aspects and implementation details

```
1 #include <iostream>
2 using namespace std;
3
4 void bubbleSort(int a[]) {
5     for (int i = 0; i < 5; i++) {
6         for (int j = 0; j < (5 - i - 1); j++) {
7             if (a[j] > a[j + 1]) {
8                 int temp = a[j];
9                 a[j] = a[j + 1];
10                a[j + 1] = temp;
11            }
12        }
13    }
14 }
15
16 int main() {
17     int myarray[5];
18     cout << "Enter 5 integers in any order " << endl;
19     for (int i = 0; i < 5; i++) {
20         cin >> myarray[i];
21     }
22     cout << "Before Sorting" << endl;
23     for (int i = 0; i < 5; i++) {
24         cout << myarray[i] << " ";
25     }
26
27     bubbleSort(myarray); // sorting
28
29     cout << endl << "After Sorting" << endl;
30     for (int i = 0; i < 5; i++) {
31         cout << myarray[i] << " ";
32     }
33
34     return 0;
35 }
36
37
38
39
40
```

Program to analyze

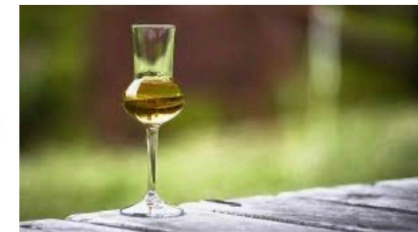
The analysis



Constraints



Constraint Solver



SOLUTION

Constraint Based 0-CFA Analysis

- We are now ready to compute the least solution $(\hat{C}, \hat{\rho})$ such that
- We have learnt that constraint generation is a syntax driven approach
- We turn semantic rules into constraints
- To this aim, we first construct a finite set $C_*[[e_*]]$ of *constraints* and *conditional constraints* ls in the form:

$$lhs \subseteq rhs$$
$$\{t\} \subseteq rhs' \Rightarrow lhs \subseteq rhs$$

if t may flow to rhs' , then
lhs must flow to rhs

where:

- $lhs ::= \hat{C}(\ell) \mid \hat{\rho}(x), \{t\}$ where t is a function abstraction
- $rhs ::= \hat{C}(\ell) \mid \hat{\rho}(x)$

From rules to constraints

- Expand each clause defining $(\hat{C}, \hat{\rho}) \models_s e_*$ into a **finite set of constraints** and
- Collect all constraints into $C_*[[e_*]]$

logical formula

set of syntactic
entities

Idea:

Occurrences of $\hat{C}$ are changed into C and
occurrences of $\hat{\rho}$ are changed into r

C and r are the **constraint variables** representing the unknown solution

The function C_* given an expression e_* returns a set of constraints $C_*[[e_*]]$

We generate constraints from the syntax-directed rules

Clauses for constants

Constants generate no constraints

$$[con] (\hat{C}, \hat{\rho}) \models_s c^\ell \text{ iff } \emptyset \subseteq \hat{C}(\ell)$$


$$C_*[[c^\ell]] = \emptyset$$

constants generate
no constraints:

Example

$$e \triangleq 7^1$$

$$C_*[[7^1]] = \emptyset$$

Clauses for variables

Variables generate a constraint reflecting a flow from $r(x)$ to $C(\ell)$

$$[var] (\hat{C}, \hat{\rho}) \models_s x^\ell \text{ iff } \hat{\rho}(x) \subseteq \hat{C}(\ell)$$


$$C_*[[x^\ell]] = \{r(x) \subseteq C(\ell)\}$$

Example

$$e \triangleq x^1$$

$$C_*[[x^1]] = \{r(x) \subseteq C(1)\}$$

Clauses for functions

Function abstractions generate:

- a flow constraint, and, recursively,
- constraints from the body

$$[fn] (\hat{C}, \hat{\rho}) \models_s (fn\ x \Rightarrow e_0)^\ell \text{ iff } \{fn\ x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge (\hat{C}, \hat{\rho}) \models_s e_0$$

$$C_* \llbracket (fn\ x \Rightarrow e_0)^\ell \rrbracket = \{ \{fn\ x \Rightarrow e_0\} \subseteq C(\ell) \} \cup C_* \llbracket e_0^\ell \rrbracket$$

Example

$$e \triangleq (fn\ x \Rightarrow x^1)^2$$

$$C_* \llbracket (fn\ x \Rightarrow x^1)^2 \rrbracket = \{ \{fn\ x \Rightarrow x^1\} \subseteq C(2) \} \cup \{ r(x) \subseteq C(1) \}$$

function constraint

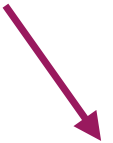
Body constraint

Clauses for recursive functions

Function abstractions generate:

- a flow constraint, and, recursively,
- constraints from the body

$$[fun] (\hat{C}, \hat{\rho}) \models_s (\text{fun } f \ x \Rightarrow e_0)^\ell \text{ iff } \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{C}(\ell) \wedge$$
$$(\hat{C}, \hat{\rho}) \models_s e_0 \wedge \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \hat{\rho}(f)$$


$$C_*[\![(\text{fun } f \ x \Rightarrow e_0)^\ell]\!] = \{ \{\text{fun } f \ x \Rightarrow e_0\}^\ell \subseteq C(\ell) \} \cup C_*[\![e_0^\ell]\!]$$
$$\cup \{ \{\text{fun } f \ x \Rightarrow e_0\}^\ell \subseteq r(f) \}$$

recursive binding: function flows to f

Clause for function application

Application generates recursively constraints from its terms plus the body conditional constraints

$$[app] (\hat{C}, \hat{\rho}) \models_s (t_1^{\ell_1} t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_s t_2^{\ell_2} \wedge \\ (\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) : \\ \hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge \dots$$

$$C_*[[(t_1^{\ell_1} t_2^{\ell_2})^\ell]] = C_*[[t_1^{\ell_1}]] \cup C_*[[t_2^{\ell_2}]] \cup$$

$$\{ \{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_2) \subseteq r(x) \mid t = (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_* \} \cup$$

$$\{ \{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_0) \subseteq C(\ell) \mid t = (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_* \} \cup \dots$$

For each possible function t at ℓ_1 in $\mathbf{Term}_*$: if t flows to ℓ_1 , then:

- argument flows to parameter
- result flows to ℓ

Conditional constraints

The set of function abstractions in the terms to analyze

Clause for function application

$[app] (\hat{C}, \hat{\rho}) \models_s (t_1^{\ell_1} t_2^{\ell_2})^\ell$ iff $(\hat{C}, \hat{\rho}) \models_s t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_s t_2^{\ell_2} \wedge$
 $(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell)) \wedge$
 $(\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \hat{C}(\ell_1) :$
 $\hat{C}(\ell_2) \subseteq \hat{\rho}(x) \wedge \hat{C}(\ell_0) \subseteq \hat{C}(\ell))$



$$C_* \llbracket (t_1^{\ell_1} t_2^{\ell_2})^\ell \rrbracket = C_* \llbracket t_1^{\ell_1} \rrbracket \cup C_* \llbracket t_2^{\ell_2} \rrbracket \cup \{ \{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_2) \subseteq r(x) \mid t = (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_* \}$$

$$\cup \{ \{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_0) \subseteq C(\ell) \mid t = (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_* \}$$

$$\cup \{ \{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_2) \subseteq r(x) \mid t = (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_* \}$$

$$\cup \{ \{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_0) \subseteq C(\ell) \mid t = (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_* \}$$

See **Example 1** next

Constraint based CFA

$$[con] \quad C_\star[c^\ell] = \emptyset$$

$$[var] \quad C_\star[x^\ell] = \{r(x) \subseteq C(\ell)\}$$

$$[fn] \quad C_\star[(\text{fn } x \Rightarrow e_0)^\ell] = \{\{\text{fn } x \Rightarrow e_0\} \subseteq C(\ell)\} \cup C_\star[e_0]$$

$$[fun] \quad C_\star[(\text{fun } f \ x \Rightarrow e_0)^\ell] = \{\{\text{fun } f \ x \Rightarrow e_0\} \subseteq C(\ell)\} \cup C_\star[e_0] \cup \{\{\text{fun } f \ x \Rightarrow e_0\} \subseteq r(f)\}$$

$$[app] \quad C_\star[(t_1^{\ell_1} \ t_2^{\ell_2})^\ell] = C_\star[t_1^{\ell_1}] \cup C_\star[t_2^{\ell_2}] \cup \{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_2) \subseteq r(x) \mid t = (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_\star\} \cup \{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_0) \subseteq C(\ell) \mid t = (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_\star\} \cup \{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_2) \subseteq r(x) \mid t = (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_\star\} \cup \{\{t\} \subseteq C(\ell_1) \Rightarrow C(\ell_0) \subseteq C(\ell) \mid t = (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \mathbf{Term}_\star\}$$

$$[if] \quad C_\star[(\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell] = C_\star[t_0^{\ell_0}] \cup C_\star[t_1^{\ell_1}] \cup C_\star[t_2^{\ell_2}] \cup \{C(\ell_1) \subseteq C(\ell)\} \cup \{C(\ell_2) \subseteq C(\ell)\}$$

$$[let] \quad C_\star[(\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^\ell] = C_\star[t_1^{\ell_1}] \cup C_\star[t_2^{\ell_2}] \cup \{C(\ell_1) \subseteq r(x)\} \cup \{C(\ell_2) \subseteq C(\ell)\}$$

$$[op] \quad C_\star[(t_1^{\ell_1} \ op \ t_2^{\ell_2})^\ell] = C_\star[t_1^{\ell_1}] \cup C_\star[t_2^{\ell_2}]$$

Back to Example 1

We generate the following set of constraints for our expression

$$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$$

$$C_*[[e_*]] = C_*[(\text{fn } x \Rightarrow x^1)^2] \cup C_*[(\text{fn } y \Rightarrow y^3)^4] \cup$$

[app]

$\{\text{fn } y \Rightarrow y^3\} \in \text{Terms}_*$

	$(\hat{C}_e, \hat{p}_e)$
1	$\{\text{fn } y \Rightarrow y^3\}$
2	$\{\text{fn } x \Rightarrow x^1\}$
3	$\emptyset$
4	$\{\text{fn } y \Rightarrow y^3\}$
5	$\{\text{fn } y \Rightarrow y^3\}$
x	$\{\text{fn } y \Rightarrow y^3\}$
y	$\emptyset$

$$\{\text{fn } x \Rightarrow x^1\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$$

$$\{\text{fn } x \Rightarrow x^1\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$$

$$\{\text{fn } y \Rightarrow y^3\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$$

$$\{\text{fn } y \Rightarrow y^3\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$$

$$C_*[(\text{fn } x \Rightarrow x^1)^2] = \{\text{fn } x \Rightarrow x^1\} \subseteq C(2) \cup C_*[x^1]$$

[fn]

$$C_*[(\text{fn } y \Rightarrow y^3)^4] = \{\text{fn } y \Rightarrow y^3\} \subseteq C(4) \cup C_*[y^3]$$

$$C_*[x^1] = r(x) \subseteq C(1)$$

[var]

$$C_*[y^3] = r(y) \subseteq C(3)$$

We use that $\text{fn } x \Rightarrow x^1$ and $\text{fn } y \Rightarrow y^3$ are the only abstractions in Term_*

Back to Example 1

We generate the following set of constraints for our expression

$$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$$

	$(\hat{C}_e, \hat{\rho}_e)$	
1	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
2	$\{\text{fn } x \Rightarrow x^1\}$	$\{\text{fn } x \Rightarrow x^1\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
3	$\emptyset$	$\{\text{fn } y \Rightarrow y^3\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
4	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$
5	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } x \Rightarrow x^1\} \subseteq C(2)$
x	$\{\text{fn } y \Rightarrow y^3\}$	$\{\text{fn } y \Rightarrow y^3\} \subseteq C(4)$
y	$\emptyset$	$r(x) \subseteq C(1)$
		$r(y) \subseteq C(3)$

Complexity

The function C^* uses the set of terms $\mathbf{Term}_*$. Let n be the size of the expression e_*

Naive estimate:

- $O(n^2)$ constraints
- $O(n^4)$ conditional constraints

the number of sub-terms

Refined estimate:

Key observation: Every construct, except applications, gives us only one constraint.

Applications may give us $O(n)$ constraints:

- $O(n)$ constraints
- $O(n^2)$ conditional constraints

Each of the $O(n)$ sub-terms only generate $O(1)$ constraints and $O(n)$ conditional constraints

The analysis remains **polynomial** despite higher-order functions

Back to Example 1

$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

$n = 5$

Refined estimate:

- $O(n) = 5$ constraints
- $O(n^2) = 25$ conditional constraints

worst-case bounds, but not reached here

Here, we just have **two functions** and **one application**. For each $t \in C(2)$ we generate two conditional constraints. For each subterm we only generate one constraint

- 4 constraints
- 4 conditional constraints

We use that $\text{fn } x \Rightarrow x^1$ and $\text{fn } y \Rightarrow y^3$ are the only abstractions in **Term**_{*}

$\{\text{fn } x \Rightarrow x^1\} \subseteq C(2)$ $\{\text{fn } y \Rightarrow y^3\} \subseteq C(4)$ $r(x) \subseteq C(1)$ $r(y) \subseteq C(3)$	$\{\text{fn } x \Rightarrow x^1\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$ $\{\text{fn } x \Rightarrow x^1\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$ $\{\text{fn } y \Rightarrow y^3\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$ $\{\text{fn } y \Rightarrow y^3\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$
--	--

From rules to constraints

To give meaning to the syntax, we can translate syntactic entities to sets of terms:

$$(\hat{C}, \hat{\rho}) \llbracket C(\ell) \rrbracket = \hat{C}(\ell)$$

$$(\hat{C}, \hat{\rho}) \llbracket r(x) \rrbracket = \hat{\rho}(x)$$

$$(\hat{C}, \hat{\rho}) \llbracket \{t\} \rrbracket = \{t\}$$

$$(\hat{C}, \hat{\rho}) \llbracket \{t\} \subseteq rhs' \Rightarrow lhs \rrbracket = \begin{cases} (\hat{C}, \hat{\rho}) \llbracket lhs \rrbracket & \text{if } \{t\} \in (\hat{C}, \hat{\rho}) \llbracket rhs' \rrbracket \\ \emptyset & \text{otherwise} \end{cases}$$

Satisfaction relation for constraints:

$$(\hat{C}, \hat{\rho}) \llbracket C(\ell) \rrbracket \models_c (ls \subseteq rhs) \text{ iff } (\hat{C}, \hat{\rho}) \llbracket ls \rrbracket \subseteq (\hat{C}, \hat{\rho}) \llbracket rhs \rrbracket$$

$$(\hat{C}, \hat{\rho}) \llbracket C(\ell) \rrbracket \models_c C \text{ iff } \forall (ls \subseteq rhs) \in C : (\hat{C}, \hat{\rho}) \models_c (ls \subseteq rhs)$$

set of constraints

Preservation of solutions

- All solutions to the set $C_*[[e_*]]$ of constraints also satisfy the syntax directed specification of the CFA and vice versa (when restricted to program terms)
- Thus computing the least solution of the constraints $C_*[[e_*]]$ gives exactly the least CFA solution $(\hat{C}, \hat{\rho})$ to $(\hat{C}, \hat{\rho}) \models_s e_*$

Proposition

If $(\hat{C}, \hat{\rho}) \subseteq (\hat{C}^\top, \hat{\rho}^\top)$
then $(\hat{C}, \hat{\rho}) \models_s e_*$ iff $(\hat{C}, \hat{\rho}) \models_c C_*[[e_*]]$

Preservation of solutions

- All solutions to the set $C_*[[e_*]]$ of constraints also satisfy the syntax directed specification of the CFA and vice versa (when restricted to program terms)
- Thus computing the least solution of the constraints $C_*[[e_*]]$ gives exactly the least CFA solution $(\hat{C}, \hat{\rho})$ to $(\hat{C}, \hat{\rho}) \models_s e_*$

Proposition

If $(\hat{C}, \hat{\rho}) \subseteq (\hat{C}^\top, \hat{\rho}^\top)$

then $(\hat{C}, \hat{\rho}) \models_s e_*$ iff $(\hat{C}, \hat{\rho}) \models_c C_*[[e_*]]$

Proof sketch

A simple structural induction on e shows that $(\hat{C}, \hat{\rho}) \models_s e$ iff $(\hat{C}, \hat{\rho}) \models_c C_*[[e]]$ for all sub-terms e of e_* .

In the case of function application $(t_1^{\ell_1} t_2^{\ell_2})^\ell$ the assumption $(\hat{C}, \hat{\rho}) \subseteq (\hat{C}^\top, \hat{\rho}^\top)$ is used to ensure that $\hat{C}(\ell_1) \subseteq \mathbf{Terms}_*$

Solving the Constraints

Finding the **least solution** of the constraints amounts to computing the **least fixed point** of a function

1. Define a suitable monotone function F_* over analyses that propagates constraints over C and r

$$F_* : \widehat{\mathbf{Cache}_*} \times \widehat{\mathbf{Env}_*} \rightarrow \widehat{\mathbf{Cache}_*} \times \widehat{\mathbf{Env}_*}$$

show that it has a least fixed point $lfp(F_*)$ that is indeed the **least solution**

2. Compute the $lfp(F_*)$

Straightforward techniques allow us to compute the least fixed point in $O(n^5)$ time when n is the size of the expression e_*

But we can do better, using **graph formulation of the constraints**: we obtain $O(n^3)$

Solving the Constraints

We can visualize a constraint system as a **constraint graph**:

Nodes

- a node $C(\ell)$ for each label $\ell \in \mathbf{Lab}_*$
- a node $r(x)$ for each variable $x \in \mathbf{Var}_*$

Edges

- Each constraint $p_1 \subseteq p_2$ adds an initial edge (p_1, p_2) to the graph
- Each constraint $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$ adds
 - a candidate edge (p_1, p_2) , with “trigger” $\{t\} \subseteq p$, and
 - an edge (p, p_2)

Example 1

$$\{id_x\} \subseteq C(2)$$

$$\{id_y\} \subseteq C(4)$$

$$r(x) \subseteq C(1)$$

$$r(y) \subseteq C(3)$$

$$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$$

$$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$$

$$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$$

$$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$$

Solving the Constraints

We can visualize a constraint system as a **constraint graph**:

Nodes

- a node $C(\ell)$ for each label $\ell \in \mathbf{Lab}_*$
- a node $r(x)$ for each variable $x \in \mathbf{Var}_*$

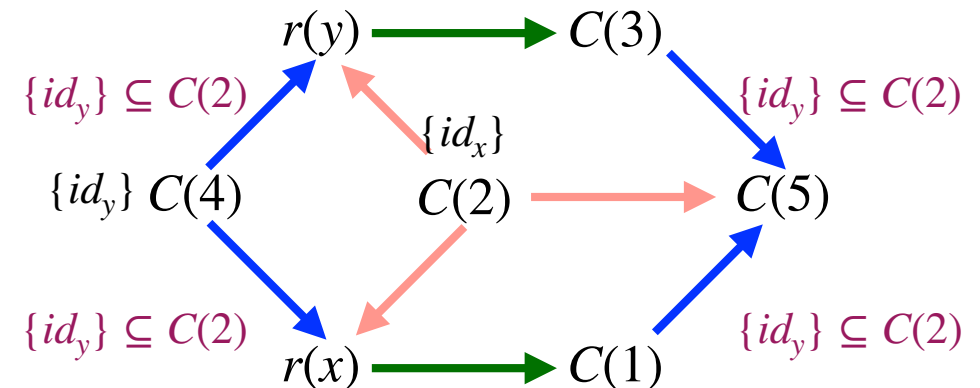
Edges

- Each constraint $p_1 \subseteq p_2$ adds an initial edge (p_1, p_2) to the graph
- Each constraint $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$ adds
 - a candidate edge (p_1, p_2) , with “trigger” $\{t\} \subseteq p$, and
 - an edge (p, p_2)

Example 1

$\{id_x\} \subseteq C(2)$
$\{id_y\} \subseteq C(4)$
$r(x) \subseteq C(1)$
$r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$



Solving the Constraints: idea

Associated with each node p we have a data field $D[p]$ representing the current knowledge. Initially $D[p]$ is given by

$$D[p] = \{t \mid (\{t\} \subseteq p) \in C_*[[e_*]]\}$$

All edges are traversed in order to propagate information from one data field to another

Value sets

Each **node** p of the constraint graph is associated with a set (Data array) $D[p]$ of values
Goal: find the value set of every node

syntactical representations
of functions

Initially, we have

$$D[p] = \{t \mid (\{t\} \subseteq p) \in C_*[[e_*]]\}$$

Example 1

$$D[C(2)] = \{id_x\}$$

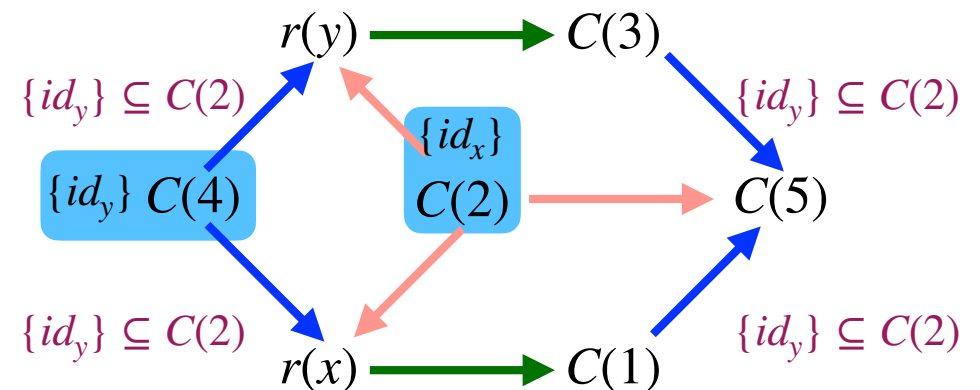
$$D[C(4)] = \{id_y\}$$

$$D[p] = \emptyset \text{ for other nodes } p$$

Example 1

$$\begin{aligned} \{id_x\} &\subseteq C(2) \\ \{id_y\} &\subseteq C(4) \\ r(x) &\subseteq C(1) \\ r(y) &\subseteq C(3) \end{aligned}$$

$$\begin{aligned} \{id_x\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(x) \\ \{id_x\} &\subseteq C(2) \Rightarrow C(1) \subseteq C(5) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(y) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(3) \subseteq C(5) \end{aligned}$$



Constraint solving =
graph reachability +
propagation

Propagation

The idea of the algorithm is to iteratively propagate the information from one data field to another according to the constraints

- If we have an edge (p_1, p_2) in our graph, then the value set of p_1 must flow into the value set of p_2

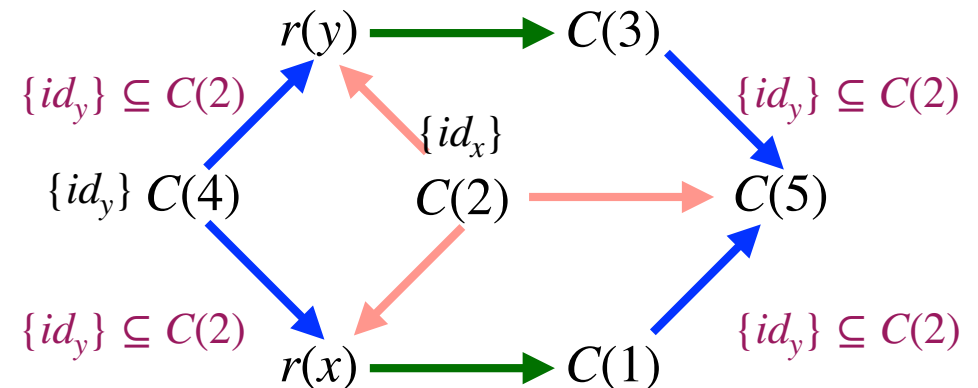


- If a trigger is true then also in the corresponding edge the value set flows

Example 1

$$\begin{aligned} \{id_x\} &\subseteq C(2) \\ \{id_y\} &\subseteq C(4) \\ r(x) &\subseteq C(1) \\ r(y) &\subseteq C(3) \end{aligned}$$

$$\begin{aligned} \{id_x\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(x) \\ \{id_x\} &\subseteq C(2) \Rightarrow C(1) \subseteq C(5) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(y) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(3) \subseteq C(5) \end{aligned}$$



The Constraint Solver

Input: a set of constraint $C_*[[e_*]]$

Output: the least solution $(\hat{C}, \hat{r})$ to the constraints (an assignment to variables $C(\ell)$ and $r(x)$)

Data structures:

- a **worklist** W containing the nodes whose data array should be propagated (a list of nodes whose outgoing edges should be traversed)
- a **data array** D storing for each node the current abstract value in $\mathbf{Val}_*$
- an **edge array** E storing for each node a list of the constraints that the node may affect (i.e., a list of constraints from which a list of the successor nodes can be computed)

The **nodes** consist $C(\ell)$ for all labels ℓ and $r(x)$ for all labels x

The solver computes a least fixed point by propagating values until saturation

The algorithm

Constraint solving =
graph reachability +
propagation

Idea

Each node stores a set of abstract values, and edges describe how these values must be propagated

The algorithm propagates abstract values along edges until no new information appears

- D = current knowledge
- E = how knowledge propagates
- W = what still needs to be processed

The algorithm: step 1

Initialize all the data structures D , E and W to $[]$

The algorithm: step 2

Build the graph:

Nodes

- a node $C(\ell)$ for each label $\ell \in \mathbf{Lab}_*$
- a node $r(x)$ for each variable $x \in \mathbf{Var}_*$

$$E[p_1] := \text{cons}(p_1 \subseteq p_2, E[p_1])$$

Edges $E[p_1] := \text{cons}(\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2, E[p_1])$

$$E[p] := \text{cons}(\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2, E[p])$$

- Each constraint $p_1 \subseteq p_2$ adds an initial edge (p_1, p_2) to the graph
- Each constraint $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$ adds a candidate edge (p_1, p_2) , with “trigger” $\{t\} \subseteq p$, and an edge (p, p_2)

Initial assignments to D : if $(\{t\} \subseteq p) \in C_*[[e_*]]$ and $\{t\} \not\subseteq D[p]$: $\text{add}(p, \{t\})$

- incorporates $\{t\}$ into $D[p]$ and
- adds p to the worklist W

$$D[p] = \{t \mid (\{t\} \subseteq p) \in C_*[[e_*]]\}$$

The algorithm: step 3

Continue propagating the information from one data field to another, along **edges**, as long as W is non-empty

$E[q]$: list of the constraints that q may affect

We extract the head element q of W and for all constraints cc in $E[q]$

- $cc = p_1 \subseteq p_2$: If we have an edge (p_1, p_2) in our graph, then the value set of p_1 must flow into the value set of p_2 ($\text{add}(p_2, D[p_1])$)
- $cc = \{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$: If the trigger is true ($t \in D[p]$), then also in the corresponding edge the value set flows ($\text{add}(p_2, D[p_1])$)

The algorithm: step 4

Record the solution in a more familiar form:

- $\forall \ell \in \mathbf{Lab}_* : \hat{C}(\ell) := D[C(\ell)]$
- $\forall x \in \mathbf{Var}_* : \hat{\rho}(x) := D[r(x)]$

The algorithm pseudocode (1)

```
(* Step 1: Initialization *)  
W := [];  
for q in Vars do  
  D[q] := [];  
  E[q] := [];  
done
```

The algorithm pseudocode (2)

```
(* Step 2: Build the graph *)
for cc in  $C_*[[e_*]]$  do
match cc with
|  $\{t\} \subseteq p \rightarrow \text{add}(p, \{t\})$ 
|  $p_1 \subseteq p_2 \rightarrow E[p_1] := cc :: E[p_1]$ 
|  $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2 \rightarrow E[p_1] := cc :: E[p_1];$ 
                                    $E[p] := cc :: E[p];$ 
done
```

```
let add(q,d) =
  if  $\neg (d \subseteq D[q])$ 
  then  $D[q] := D[q] \cup d;$ 
  W := push W q;
```

Back to ex. 1: step 2

Initially, we have

$$D[p] = \{t \mid (\{t\} \subseteq p) \in C_*[[e_*]]\}$$

Current
abstract values

$$\begin{aligned} \{id_x\} &\subseteq C(2) \\ \{id_y\} &\subseteq C(4) \\ r(x) &\subseteq C(1) \\ r(y) &\subseteq C(3) \end{aligned}$$

$$\begin{aligned} \{id_x\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(x) \\ \{id_x\} &\subseteq C(2) \Rightarrow C(1) \subseteq C(5) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(y) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(3) \subseteq C(5) \end{aligned}$$

$$\begin{aligned} D[C(2)] &= \{id_x\} \\ D[C(4)] &= \{id_y\} \end{aligned}$$

$D[p] = \emptyset$ for other nodes p

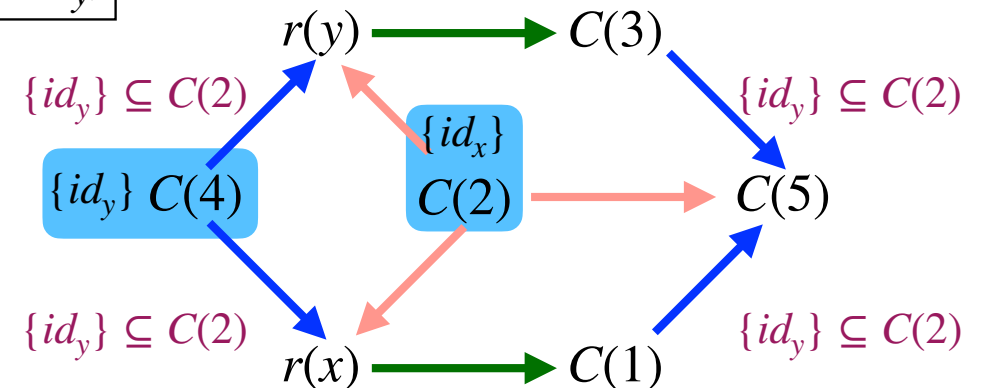
Initial worklist

$$W = [C(4), C(2)]$$

These are the nodes, whose outgoing edges should be traversed

add($C(2)$, id_x)
add($C(4)$, id_y)

```
(* Step 2: Build the graph *)
for cc in C_*[[e_*]] do
  match cc with
  | {t} ⊆ p -> add(p, {t})
  | p1 ⊆ p2 -> E[p1] := cc :: E[p1]
  | {t} ⊆ p => p1 ⊆ p2 -> E[p1] := cc :: E[p1];
                        E[p] := cc :: E[p];
done
```



Back to ex. 1: step 2

```
(* Step 2: Build the graph *)
for cc in Ce do
match cc with
| {t} ⊆ p -> add(p, {t})
| p1 ⊆ p2 -> E[p1] := cc :: E[p1]
| {t} ⊆ p => p1 ⊆ p2 -> E[p1] := cc :: E[p1];
                        E[p] := cc :: E[p];
done
```

$\{id_x\} \subseteq C(2)$
 $\{id_y\} \subseteq C(4)$
 $r(x) \subseteq C(1)$
 $r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
 $\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
 $\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
 $\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

$E[r(x)] := \{r(x) \subseteq C(1)\} :: []$
 $E[r(y)] := \{r(y) \subseteq C(3)\} :: []$

$E[p]$: list of the constraints that
 p may affect

$E[C(4)] := \{\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)\} :: []$
 $E[C(2)] := \{\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)\} :: []$
 $E[C(1)] := \{\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)\} :: []$
 $E[C(2)] := \{\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)\} :: E[C(2)]$
 $E[C(4)] = \{\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)\}$
 $E[C(2)] = \{\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)\} :: E[C(2)]$
 $E[C(3)] = \{\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)\}$
 $E[C(2)] = \{\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)\} :: E[C(2)]$

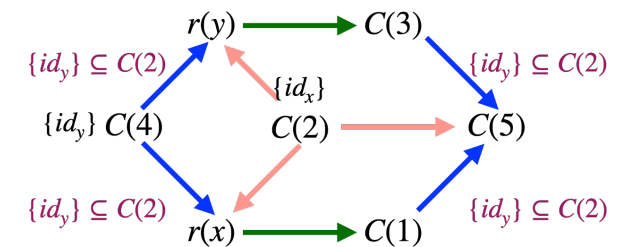
Back to ex. 1: after Step 2

$W = [C(4), C(2)]$

list of the constraints
that p may affect

p	$D[p]$	$E[p]$
C(1)	$\emptyset$	$[id_x \subseteq C(2) \Rightarrow C(1) \subseteq C(5)]$
C(2)	id_x	$[id_y \subseteq C(2) \Rightarrow C(3) \subseteq C(5), id_y \subseteq C(2) \Rightarrow C(4) \subseteq r(y), id_x \subseteq C(2) \Rightarrow C(1) \subseteq C(5), id_x \subseteq C(2) \Rightarrow C(4) \subseteq r(x)]$
C(3)	$\emptyset$	$[id_y \subseteq C(2) \Rightarrow C(3) \subseteq C(5)]$
C(4)	id_y	$[id_y \subseteq C(2) \Rightarrow C(4) \subseteq r(y), id_x \subseteq C(2) \Rightarrow C(4) \subseteq r(x)]$
C(5)	$\emptyset$	$[]$
$r(x)$	$\emptyset$	$[r(x) \subseteq C(1)]$
$r(y)$	$\emptyset$	$[r(y) \subseteq C(3)]$

Current
abstract value



The algorithm

We have now built the graph and initialized the analysis

Next, we propagate abstract values along edges, through Step 3, until we reach a fixpoint

This is a standard worklist-based fixpoint computation

The algorithm pseudocode (3,4)

```
(* Step 3: Iteration *)  
while W  $\neq$  nil do  
  q := head(W); W := tail(W);  
  for cc in E[q] do  
    case cc of  
      p1  $\subseteq$  p2: add(p2, D[p1]); // propagate  
                // all values from p1 to p2  
      {t}  $\subseteq$  p  $\Rightarrow$  p1  $\subseteq$  p2:  
        if t  $\in$  D(p) then add(p2, D[p1]);  
        // // activate conditional constraint
```

Iterative fixpoint

It propagates
D[p1] in D[p2]

if t $\in$ D[p] then
propagate
D[p1] in D[p2]

```
(* Step 4: Recording the solution *)  
for  $\ell \in \text{Lab}_*$  do  $\hat{C}(\ell) := D[C(\ell)]$ ;  
for  $x \in \text{Var}_*$ , do  $\hat{r}(x) := D[r(x)]$ ;
```

The algorithm: step 3

Continue propagating the information from one data field to another, along **edges**, as long as W is non-empty

$E[q]$: list of the constraints that q may affect

We extract the head element q of W and for all constraints cc in $E[q]$

- $cc = p_1 \subseteq p_2$: If we have an edge (p_1, p_2) in our graph, then the value set of p_1 must flow into the value set of p_2 ($\text{add}(p_2, D[p_1])$)
- $cc = \{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$: If the trigger is true ($t \in D[p]$), then also in the corresponding edge the value set flows ($\text{add}(p_2, D[p_1])$)

Back to ex. 1: 1st iteration (C(4))

$W = [C(4), C(2)]$

Take $C(4)$ from the top

$D[C(4)] = \{id_y\}$

$\{id_x\} \subseteq C(2)$
 $\{id_y\} \subseteq C(4)$
 $r(x) \subseteq C(1)$
 $r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
 $\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
 $\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
 $\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

$E(C(4)) =$

$\{\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x),$
 $\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)\}$

Constraint is triggered since

$id_x \in D[C(2)] = \{id_x\}:$
 $\text{add}(r(x), D[C(4)])$

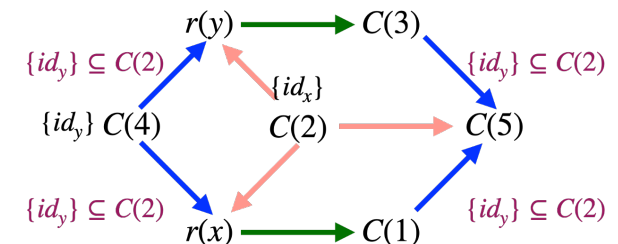
propagate
 $D[C(4)]$ to $r(x)$

- incorporate $\{id_y\}$ into $D[r(x)]$
- add $r(x)$ to W

$D[r(x)] := \{id_y\}$

$W := [r(x), C(2)]$

```
(* Step 3: Iteration *)
while W ≠ nil do
  q:= head(W); W := tail(W);
  for cc in E[q] do
    case cc of
      p1 ⊆ p2: add(p2,D[p1]);
      {t} ⊆ p ⇒ p1 ⊆ p2:
        if t ∈ D[p] then add(p2,D[p1]);
```



Back to ex. 1: 1st iteration (C(4))

$$W = [C(4), C(2)]$$

Take $C(4)$ from the top

$$D[C(4)] = \{id_y\}$$

$$\begin{aligned} \{id_x\} &\subseteq C(2) \\ \{id_y\} &\subseteq C(4) \\ r(x) &\subseteq C(1) \\ r(y) &\subseteq C(3) \end{aligned}$$

$$\begin{aligned} \{id_x\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(x) \\ \{id_x\} &\subseteq C(2) \Rightarrow C(1) \subseteq C(5) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(y) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(3) \subseteq C(5) \end{aligned}$$

$$E(C(4)) =$$

$$\begin{aligned} &\{\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x), \\ &\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)\} \end{aligned}$$

Constraint is triggered since

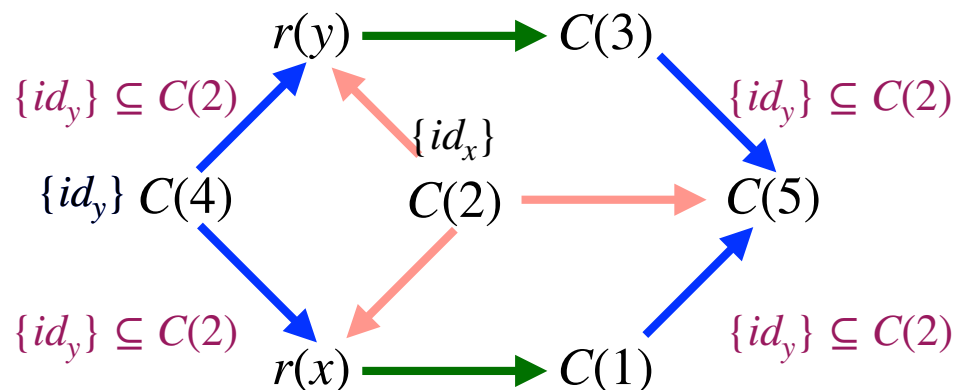
$$\begin{aligned} id_x &\in D[C(2)] = \{id_x\}: \\ &\text{add}(r(x), D[C(4)]) \end{aligned}$$

propagate
 $D[C(4)]$ to $r(x)$

- incorporate $\{id_y\}$ into $D[r(x)]$
- add $r(x)$ to W

$$D[r(x)] := \{id_y\}$$

$$W := [r(x), C(2)]$$



Back to ex. 1: 1st iteration (C(4))

$W = [C(4), C(2)]$

Take $C(4)$ from the top

$D[C(4)] = \{id_y\}$

$$\begin{aligned} \{id_x\} &\subseteq C(2) \\ \{id_y\} &\subseteq C(4) \\ r(x) &\subseteq C(1) \\ r(y) &\subseteq C(3) \end{aligned}$$

$$\begin{aligned} \{id_x\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(x) \\ \{id_x\} &\subseteq C(2) \Rightarrow C(1) \subseteq C(5) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(y) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(3) \subseteq C(5) \end{aligned}$$

$E(C(4)) =$

$\{\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x),$
 $\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)\}$

Constraint is not triggered since

$id_y \notin D[C(2)] = \{id_x\}$

hence no propagation from

$D[C(4)]$ to $r(y)$

```
(* Step 3: Iteration *)
while W ≠ nil do
  q:= head(W); W := tail(W);
  for cc in E[q] do
    case cc of
      p1 ⊆ p2: add(p2,D[p1]);
      {t} ⊆ p ⇒ p1 ⊆ p2:
        if t ∈ D[p] then add(p2,D[p1]);
```

Back to ex. 1: 2nd iteration (r(x))

$W = [r(x), C(2)]$

Take $r(x)$ from the top

$D[r(x)] = \{id_y\}$

$\{id_x\} \subseteq C(2)$
 $\{id_y\} \subseteq C(4)$
 $r(x) \subseteq C(1)$
 $r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
 $\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
 $\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
 $\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

$E(r(x)) =$

$\{\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)\}$

Constraint is triggered since

$id_x \in D[C(2)] = \{id_x\}$:

$\text{add}(C(1), D[r(x)])$

propagate
 $D[r(x)]$ to $C(1)$

```
(* Step 3: Iteration *)
while W ≠ nil do
q:= head(W); W := tail(W);
for cc in E[q] do
case cc of
  p1 ⊆ p2: add(p2,D[p1]);
  {t} ⊆ p ⇒ p1 ⊆ p2:
    if t ∈ D[p] then add(p2,D[p1]);
```

$D[C(1)] = \{id_y\}$

$W = [C(1), C(2)]$

Back to ex. 1: 2nd iteration ($r(x)$)

$$W = [r(x), C(2)]$$

Take $r(x)$ from the top

$$D[r(x)] = \{id_y\}$$

$$\begin{aligned} \{id_x\} &\subseteq C(2) \\ \{id_y\} &\subseteq C(4) \\ r(x) &\subseteq C(1) \\ r(y) &\subseteq C(3) \end{aligned}$$

$$\begin{aligned} \{id_x\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(x) \\ \{id_x\} &\subseteq C(2) \Rightarrow C(1) \subseteq C(5) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(y) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(3) \subseteq C(5) \end{aligned}$$

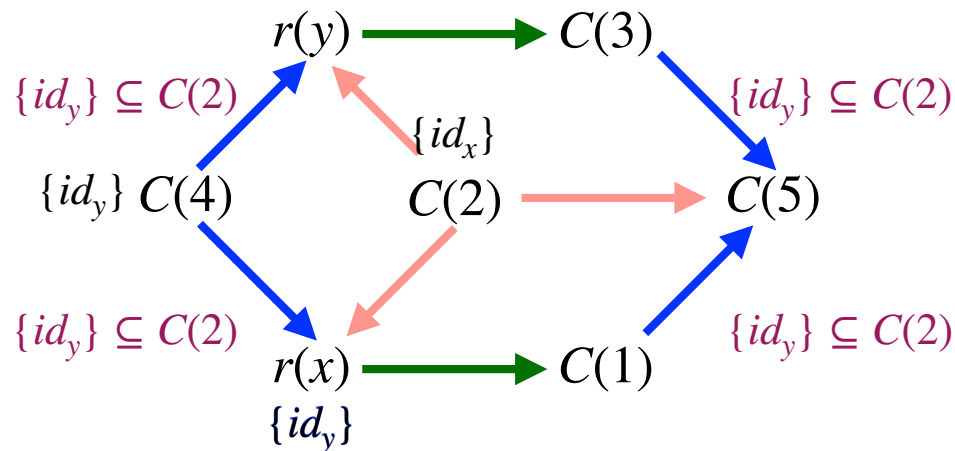
$$E(r(x)) =$$

$$\{\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)\}$$

Constraint is triggered since

$$id_x \in D[C(2)] = \{id_x\}: \\ \text{add}(C(1), D[r(x)])$$

propagate
 $D[r(x)]$ to $C(1)$



$$D[C(1)] = \{id_y\}$$

$$W = [C(1), C(2)]$$

Back to ex. 1: 3rd iteration (C(1))

$W = [C(1), C(2)]$

Take $C(1)$ from the top

$D[C(1)] = \{id_y\}$

$\{id_x\} \subseteq C(2)$
 $\{id_y\} \subseteq C(4)$
 $r(x) \subseteq C(1)$
 $r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
 $\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
 $\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
 $\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

$E(C(1)) =$

$\{\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)\}$

Constraint is triggered since

$id_x \in D[C(2)]:$

$\text{add}(C(5), D[C(1)])$

propagate
 $D[C(1)]$ to $C(5)$

```
(* Step 3: Iteration *)
while W ≠ nil do
q:= head(W); W := tail(W);
for cc in E[q] do
case cc of
  p1 ⊆ p2: add(p2,D[p1]);
  {t} ⊆ p ⇒ p1 ⊆ p2:
    if t ∈ D[p] then add(p2,D[p1]);
```

$D[C(5)] = \{id_x\}$
 $W = [C(5), C(2)]$

Back to ex. 1: 3rd iteration (C(1))

$W = [C(1), C(2)]$

Take $C(1)$ from the top

$D[C(1)] = \{id_y\}$

$\{id_x\} \subseteq C(2)$
 $\{id_y\} \subseteq C(4)$
 $r(x) \subseteq C(1)$
 $r(y) \subseteq C(3)$

$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$
 $\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$
 $\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$
 $\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$

$E(C(1)) =$

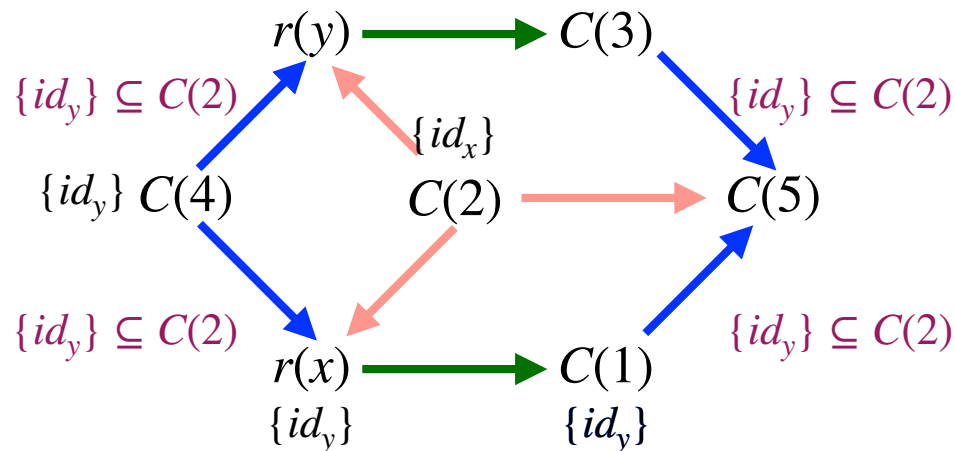
$\{\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)\}$

Constraint is triggered since

$id_x \in D[C(2)]:$

$\text{add}(C(5), D[C(1)])$

propagate
 $D[C(1)]$ to $C(5)$



$D[C(5)] = \{id_x\}$
 $W = [C(5), C(2)]$

Back to ex. 1: 4th iteration (C(5))

$$W = [C(5), C(2)]$$

Take $C(5)$ from the top

$$D[C(1)] = \{id_x\}$$

$$\{id_x\} \subseteq C(2)$$

$$\{id_y\} \subseteq C(4)$$

$$r(x) \subseteq C(1)$$

$$r(y) \subseteq C(3)$$

$$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$$

$$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$$

$$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$$

$$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$$

$$E(C(5)) = \emptyset$$



No update



$$W = [C(2)]$$

Back to ex. 1: 4th iteration (C(5))

$$W = [C(5), C(2)]$$

Take $C(5)$ from the top

$$D[C(1)] = \{id_x\}$$

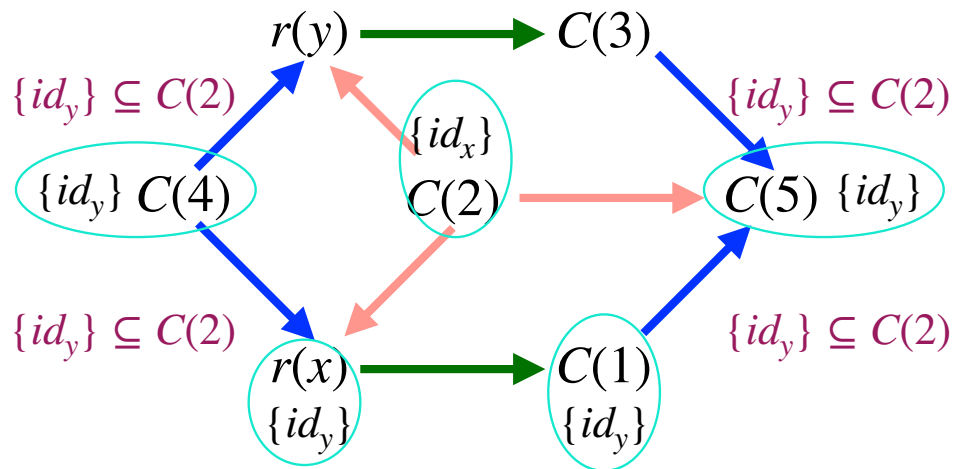
$$\begin{aligned} \{id_x\} &\subseteq C(2) \\ \{id_y\} &\subseteq C(4) \\ r(x) &\subseteq C(1) \\ r(y) &\subseteq C(3) \end{aligned}$$

$$\begin{aligned} \{id_x\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(x) \\ \{id_x\} &\subseteq C(2) \Rightarrow C(1) \subseteq C(5) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(y) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(3) \subseteq C(5) \end{aligned}$$

$$E(C(5)) = \emptyset$$

No update

$$W = [C(2)]$$



W	[C(2)]
q	D[q]
C(1)	id_y
C(2)	id_x
C(3)	$\emptyset$
C(4)	id_y
C(5)	id_y
$r(x)$	id_y
$r(y)$	$\emptyset$

Back to ex. 1: 5th iteration (C(2))

$$W = [C(2)]$$

Take $C(2)$ from the top

$$D[C(2)] = \{id_x\}$$

$$\begin{aligned} \{id_x\} &\subseteq C(2) \\ \{id_y\} &\subseteq C(4) \\ r(x) &\subseteq C(1) \\ r(y) &\subseteq C(3) \end{aligned}$$

$$\begin{aligned} \{id_x\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(x) \\ \{id_x\} &\subseteq C(2) \Rightarrow C(1) \subseteq C(5) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(4) \subseteq r(y) \\ \{id_y\} &\subseteq C(2) \Rightarrow C(3) \subseteq C(5) \end{aligned}$$

$$E(C(2)) = \{$$

$$\{id_x\} \subseteq C(2) \Rightarrow C(4) \subseteq r(x)$$

$$\{id_x\} \subseteq C(2) \Rightarrow C(1) \subseteq C(5)$$

$$\{id_y\} \subseteq C(2) \Rightarrow C(4) \subseteq r(y)$$

$$\{id_y\} \subseteq C(2) \Rightarrow C(3) \subseteq C(5)$$

$$\}$$

Only constraints for

id_x are triggered

$\text{add}(r(x), D[C(4)])$ ✓

$\text{add}(C(5), D[C(1)])$ ✓

No new information is added

$\Rightarrow$ no node is reinserted in W

$\Rightarrow W$ becomes empty

(fixpoint reached)

$$W = []$$

[C(2)]	[]
D[q]	D[q]
id_y	id_y
id_x	id_x
$\emptyset$	$\emptyset$
id_y	id_y
id_y	id_y
id_y	id_y
$\emptyset$	$\emptyset$

Back to ex. 1: iteration steps

W	[C(4), C(2)]	[r(x), C(2)]	[C(1), C(2)]	[C(5), C(2)]	[C(2)]	[]
q	D[q]	D[q]	D[q]	D[q]	D[q]	D[q]
C(1)	$\emptyset$	$\emptyset$	id_y	id_y	id_y	id_y
C(2)	id_x	id_x	id_x	id_x	id_x	id_x
C(3)	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
C(4)	id_y	id_y	id_y	id_y	id_y	id_y
C(5)	$\emptyset$	$\emptyset$	$\emptyset$	id_y	id_y	id_y
r(x)	$\emptyset$	id_y	id_y	id_y	id_y	id_y
r(y)	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

fix point

Back to ex. 1: the solution

$((\text{fn } x \Rightarrow x^1)^2 (\text{fn } y \Rightarrow y^3)^4)^5$

W	$\square$
q	D[q]
C(1)	id_y
C(2)	id_x
C(3)	$\emptyset$
C(4)	id_y
C(5)	id_y
r(x)	id_y
r(y)	$\emptyset$

The computed solution is $(\hat{C}, \hat{\rho})$:

- $\forall \ell \in \mathbf{Lab}_* : \hat{C}(\ell) := D[C(\ell)]$
- $\forall x \in \mathbf{Var}_* : \hat{\rho}(x) := D[r(x)]$

$$\hat{C}(1) = \{id_y\}$$

$$\hat{C}(2) = \{id_x\}$$

$$\hat{C}(3) = \emptyset$$

$$\hat{C}(4) = \{id_y\}$$

$$\hat{C}(5) = \{id_y\}$$

$$\hat{\rho}(x) = \{id_y\}$$

$$\hat{\rho}(y) = \emptyset$$

Complexity

The proof showing that the algorithm terminates can be refined to show that it takes at most $O(n^3)$ steps if the original expression e_* has size n . To see this recall that $\mathcal{C}_*[e_*]$ contains at most $O(n)$ constraints of the form $\{t\} \subseteq p$ or $p_1 \subseteq p_2$, and at most $O(n^2)$ constraints of the form $\{t\} \subseteq p \Rightarrow p_1 \subseteq p_2$. We therefore know that the graph has at most $O(n)$ nodes and $O(n^2)$ edges and that each data field can be enlarged at most $O(n)$ times. Assuming that the operations upon $D[p]$ take unit time we can perform the following calculations: step 1 takes time $O(n)$, step 2 takes time $O(n^2)$, and step 4 takes time $O(n)$; step 3 traverses each of the $O(n^2)$ edges at most $O(n)$ times and hence takes time $O(n^3)$; it follows that the overall algorithm takes no more than $O(n^3)$ basic steps.

Correctness of the algorithm

Proposition

Given the input $C_*[[e_*]]$, the algorithm terminates and the result $(\hat{C}, \hat{\rho})$ satisfies

$$(\hat{C}, \hat{\rho}) = \sqcap \{(\hat{C}', \hat{\rho}') \mid (\hat{C}', \hat{\rho}') \models_c C_*[[e_*]]\}$$

and hence it is the least solution to $C_*[[e_*]]$

is the least
solution of the
constraints

As a consequence $(\hat{C}, \hat{\rho}) \models_c C_*[[e_*]]$, i.e., the computed solution satisfies all constraints

Corollary

Assume that $(\hat{C}, \hat{\rho})$ is the solution to the constraints $C_*[[e_*]]$ computed by the algorithm, then $(\hat{C}, \hat{\rho}) \models e_*$

$(\hat{C}, \hat{\rho})$ is a valid
CFA of e_*

Solving constraints yields a sound analysis

Correctness of the algorithm

Proposition

Given the input $C_*[[e_*]]$, the algorithm terminates and the result $(\hat{C}, \hat{\rho})$ satisfies

$$(\hat{C}, \hat{\rho}) = \sqcap \{(\hat{C}', \hat{\rho}') \mid (\hat{C}', \hat{\rho}') \models_c C_*[[e_*]]\}$$

and hence it is the least solution to $C_*[[e_*]]$

Proof sketch

Termination:

(The steps 1,2,4 terminate. We have just to look at step 3).

The algorithm uses a finite set of terms (**Term**_{*}) and monotone updates.

Soundness:

If $(\hat{C}', \hat{\rho}') \models_c C_*[[e_*]]$: the following invariant is maintained:

$$D[C(\ell)] \subseteq \hat{C}'(\ell) \text{ and } D[r(x)] \subseteq \hat{\rho}'(x).$$

Hence $(\hat{C}, \hat{\rho}) \subseteq (\hat{C}', \hat{\rho}')$.

We must prove that $(\hat{C}, \hat{\rho}) \models_c C_*[[e_*]]$.

Correctness of the algorithm

Proposition

Given the input $C_*[[e_*]]$, the algorithm terminates and the result $(\hat{C}, \hat{\rho})$ satisfies

$$(\hat{C}, \hat{\rho}) = \sqcap \{(\hat{C}', \hat{\rho}') \mid (\hat{C}', \hat{\rho}') \models_c C_*[[e_*]]\}$$

and hence it is the least solution to $C_*[[e_*]]$

Proof sketch (cont.)

Soundness (cont.):

Assume by contradiction that some constraint $cc \in C_*[[e_*]]$ is not satisfied by $(\hat{C}, \hat{\rho})$. According to the algorithm this could not be true.

$\Rightarrow$ The computed solution satisfies all $cc \in C_*[[e_*]]$, i.e., $(\hat{C}, \hat{\rho}) \models_c cc$.

Minimality:

The result is included in every solution $\Rightarrow$ it is the least solution, computed as

$$\sqcap \{(\hat{C}', \hat{\rho}') \mid (\hat{C}', \hat{\rho}') \models_c C_*[[e_*]]\}$$

Computation vs check of acceptability

Goal: Compute a solution vs check that a solution is acceptable

There are applications where the ability to check $(\hat{C}, \hat{\rho}) \models e_*$ is more important than computing the solution

Scenario

Open system e_0 that uses resources of an unspecified library e_*

Analyses and optimizations of e_0 therefore rely on an abstraction $(\hat{C}, \hat{\rho})$ expressing the relevant properties about the library e_* , meaning that

$$(\hat{C}, \hat{\rho}) \models e_*$$

Idea:

Use $(\hat{C}, \hat{\rho})$ as a **specification** of the library e_*

Guarantee

Any other library e'_* such that $(\hat{C}, \hat{\rho}) \models e'_*$ can **safely** replace e_* , since the specification $(\hat{C}, \hat{\rho})$ still guarantees all required properties about the library e'_*

Adding Data Flow Analysis

CFA + DFA

We extend Control Flow Analysis with data-flow information

Abstract values = functions + data-flow properties

$$\hat{v} \in \widehat{\mathbf{Val}}_d = \wp(\mathbf{Term} \cup \mathbf{Data})$$

Abstract values

Now contain

- possible functions (as before) and
- abstract data-flow properties, (i.e., properties of booleans and integers, such as sign, constantness, or ranges of values)

The data-flow component can be:

- a powerset (e.g., sets of possible values)
- a complete lattice (e.g., intervals, signs, constants)

Abstract domains

$$\hat{v} \in \widehat{\mathbf{Val}}_d = \wp(\mathbf{Term} \cup \mathbf{Data})$$

Constants

Each constant $c \in \mathbf{Const}$ is mapped to an element $d_c \in \mathbf{Data}$ specifying the abstract property of c :

d_c can be thought as $\alpha(c)$ for a suitable abstraction α

Operators are lifted from concrete data to abstract values, i.e., they operate on abstract properties instead of concrete values

To each operator $op \in \mathbf{Op}$, corresponds a total function

$\hat{op} : \widehat{\mathbf{Val}}_d \times \widehat{\mathbf{Val}}_d \rightarrow \widehat{\mathbf{Val}}_d$ telling how op operates on abstract properties

Abstract operators over-approximate concrete operations

Abstract operators

$$\hat{op} : \widehat{\mathbf{Val}}_d \times \widehat{\mathbf{Val}}_d \rightarrow \widehat{\mathbf{Val}}_d$$

We apply the operator to all combinations of abstract data values

Typically, $\hat{op}$ has a definition of the form

$$\hat{v}_1 \hat{op} \hat{v}_2 = \bigcup \{d_{op}(d_1, d_2) \mid d_1 \in \hat{v}_1 \cap \mathbf{Data}, d_2 \in \hat{v}_2 \cap \mathbf{Data}\}$$

for some function $op : \mathbf{Data} \times \mathbf{Data} \rightarrow \wp(\mathbf{Data})$ specifying how the operator computes with the abstract properties of integers and booleans

This over-approximates all possible concrete computations

Example: signs analysis

$$\text{Data}_{\text{sign}} = \{\text{tt}, \text{ff}, -, 0, +\}$$

Each constant is mapped to a sign: $d_{\text{true}} = \text{tt}$ and $d_7 = +, \dots$

Take, e.g., $\hat{+}$ as

$$\hat{v}_1 \hat{+} \hat{v}_2 = \bigcup_{d_1 \in \hat{v}_1, d_2 \in \hat{v}_2} \{d_+(d_1, d_2)\}$$

This defines the
abstract
semantics of $+$

Abstract
addition
on signs

d_+	tt	ff	-	0	+
tt	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
ff	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
-	$\emptyset$	$\emptyset$	$\{-\}$	$\{-\}$	$\{-, 0, +\}$
0	$\emptyset$	$\emptyset$	$\{-\}$	$\{0\}$	$\{+\}$
+	$\emptyset$	$\emptyset$	$\{-, 0, +\}$	$\{+\}$	$\{+\}$

For instance, if

$$\hat{v}_1 = \{-, 0\} \text{ and } \hat{v}_2 = \{+\}$$

then

$$\hat{v}_1 \hat{+} \hat{v}_2 = d_+(-, +) \cup d_+(0, +) = \{-, 0, +\}$$

Acceptability relation

The acceptability relation of the combined analysis has the form

$$(\hat{C}, \hat{\rho}) \models_d e$$

Each rule over-approximates both control-flow and data-flow

The rules are as before, but operate on enriched abstract values

Key clauses:

- [con]: maps constants to abstract data values
- [if]: uses abstract boolean information to select possible branches
- [op]: uses abstract operators on data

Data-flow information is propagated through the evaluation of expressions

Clause for constants

$$[con] (\hat{C}, \hat{\rho}) \models_d c^\ell \text{ iff } \{d_c\} \subseteq \hat{C}(\ell)$$

We now track data values: the CFA records that d_c is a possible value of c

Constants introduce abstract data into the analysis

NEW: constants contribute **data** (not functions)

Example

$$c^\ell = 7^\ell$$

CFA:

$$\hat{C}(\ell) = \emptyset$$

CFA+DFA:

$$\hat{C}(\ell) = \{ + \}$$

Clause for conditional

$$\begin{aligned}
 [if] \ (\hat{C}, \hat{\rho}) \models_d (\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell \\
 \text{iff } (\hat{C}, \hat{\rho}) \models_d t_0^{\ell_0} \wedge \\
 (d_{true} \in \hat{C}_{\ell_0} \Rightarrow ((\hat{C}, \hat{\rho}) \models_d t_1^{\ell_1} \wedge \hat{C}(\ell_1) \subseteq \hat{C}(\ell))) \wedge \\
 (d_{false} \in \hat{C}_{\ell_0} \Rightarrow ((\hat{C}, \hat{\rho}) \models_d t_2^{\ell_2} \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell)))
 \end{aligned}$$

Is flow sensitive: sensitive: can determine whether true or false branches can be taken

- Increased precision: we analyze only those branches consistent with the abstract value of the condition, which now is trackable
- control-flow is now **filtered** by data-flow

Example

$e \triangleq (\text{if } (3^1 > 0^2)^3 \text{ then } (\text{fn } y \Rightarrow y^4)^5 \text{ else } (\text{fn } z \Rightarrow z^6)^7)^8$

$\delta_{true} \sqsubseteq \hat{C}(3) \Rightarrow \hat{C}(5) \subseteq \hat{C}(8) \dots \delta_{false} \not\sqsubseteq \hat{C}(3) \Rightarrow$

Clause for binary operation

$$[op] (\hat{C}, \hat{\rho}) \models_d (t_1^{\ell_1} \text{ op } t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models_d t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_d t_2^{\ell_2} \wedge \hat{C}(\ell_1) \hat{op} \hat{C}(\ell_2) \subseteq \hat{C}(\ell)$$

NOW: we do track data values

This rule applies abstract operators to propagate data-flow information

Data-flow is computed **locally** using abstract operators

Example

$$e \triangleq (7^1 + 4^2)^3$$

CFA:

$$\hat{C}(3) = \emptyset \dots$$

CFA +DFA:

$$\hat{C}(1) \hat{+} \hat{C}(2) \subseteq \hat{C}(3) \text{ if } \hat{C}(1) \text{ and } \hat{C}(2) \text{ contain } \{ + \} \text{ then } \hat{C}(3) \text{ contains } \{ + \}$$

To recap

- [con]: introduces data (abstract values)
- [if]: uses abstract values to guide control-flow
- [op]: computes data-flow using abstract values

CFA with abstract values as powersets

$$\begin{aligned}
 [\text{con}] \quad & (\widehat{C}, \widehat{\rho}) \models_d c^\ell \text{ iff } \{d_c\} \subseteq \widehat{C}(\ell) \\
 [\text{var}] \quad & (\widehat{C}, \widehat{\rho}) \models_d x^\ell \text{ iff } \widehat{\rho}(x) \subseteq \widehat{C}(\ell) \\
 [\text{fn}] \quad & (\widehat{C}, \widehat{\rho}) \models_d (\text{fn } x \Rightarrow e_0)^\ell \text{ iff } \{\text{fn } x \Rightarrow e_0\} \subseteq \widehat{C}(\ell) \\
 [\text{fun}] \quad & (\widehat{C}, \widehat{\rho}) \models_d (\text{fun } f \ x \Rightarrow e_0)^\ell \text{ iff } \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \widehat{C}(\ell) \\
 [\text{app}] \quad & (\widehat{C}, \widehat{\rho}) \models_d (t_1^{\ell_1} \ t_2^{\ell_2})^\ell \\
 & \quad \text{iff } (\widehat{C}, \widehat{\rho}) \models_d t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models_d t_2^{\ell_2} \wedge \\
 & \quad (\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \widehat{C}(\ell_1) : \\
 & \quad \quad (\widehat{C}, \widehat{\rho}) \models_d t_0^{\ell_0} \wedge \\
 & \quad \quad \widehat{C}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_0) \subseteq \widehat{C}(\ell)) \wedge \\
 & \quad (\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \widehat{C}(\ell_1) : \\
 & \quad \quad (\widehat{C}, \widehat{\rho}) \models_d t_0^{\ell_0} \wedge \\
 & \quad \quad \widehat{C}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_0) \subseteq \widehat{C}(\ell) \wedge \\
 & \quad \quad \{\text{fun } f \ x \Rightarrow t_0^{\ell_0}\} \subseteq \widehat{\rho}(f)) \\
 [\text{if}] \quad & (\widehat{C}, \widehat{\rho}) \models_d (\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell \\
 & \quad \text{iff } (\widehat{C}, \widehat{\rho}) \models_d t_0^{\ell_0} \wedge \\
 & \quad (d_{\text{true}} \in \widehat{C}(\ell_0) \Rightarrow ((\widehat{C}, \widehat{\rho}) \models_d t_1^{\ell_1} \wedge \widehat{C}(\ell_1) \subseteq \widehat{C}(\ell))) \wedge \\
 & \quad (d_{\text{false}} \in \widehat{C}(\ell_0) \Rightarrow ((\widehat{C}, \widehat{\rho}) \models_d t_2^{\ell_2} \wedge \widehat{C}(\ell_2) \subseteq \widehat{C}(\ell))) \\
 [\text{let}] \quad & (\widehat{C}, \widehat{\rho}) \models_d (\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^\ell \\
 & \quad \text{iff } (\widehat{C}, \widehat{\rho}) \models_d t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models_d t_2^{\ell_2} \wedge \\
 & \quad \widehat{C}(\ell_1) \subseteq \widehat{\rho}(x) \wedge \widehat{C}(\ell_2) \subseteq \widehat{C}(\ell) \\
 [\text{op}] \quad & (\widehat{C}, \widehat{\rho}) \models_d (t_1^{\ell_1} \text{ op } t_2^{\ell_2})^\ell \\
 & \quad \text{iff } (\widehat{C}, \widehat{\rho}) \models_d t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models_d t_2^{\ell_2} \wedge \\
 & \quad \widehat{C}(\ell_1) \widehat{\text{op}} \widehat{C}(\ell_2) \subseteq \widehat{C}(\ell)
 \end{aligned}$$

Example 3

$$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow (\text{if } (x^1 > 0^2)^3 \text{ then } (\text{fn } y \Rightarrow y^4)^5 \\ \text{else } (\text{fn } z \Rightarrow 25^6)^7)^8)^9 \\ \text{in } ((f^{10} \ 3^{11})^{12} \ 0^{13})^{14})^{15}$$

Pure 0-CFA:

it cannot determine that the `else` branch is *unreachable*

CFA + Sign Analysis:

- since $x \in \{+\}$, the condition $x > 0$ is always true
- `else` branch is unreachable
- $\hat{C}(12) = \{ \text{fn } y \Rightarrow y^4 \}$, i.e., only $\text{fn } y \Rightarrow y^4$ flows to label 12

Example 3

$$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow (\text{if } (x^1 > 0^2)^3 \text{ then } (\text{fn } y \Rightarrow y^4)^5 \\ \text{else } (\text{fn } z \Rightarrow 25^6)^7)^8)^9 \\ \text{in } ((f^{10} \ 3^{11})^{12} \ 0^{13})^{14})^{15}$$

At label 12:

- Pure CFA: $\hat{C}(12) = \{ \text{fn } y \Rightarrow y^4, \text{fn } z \Rightarrow 25^6 \}$
- With signs: $\hat{C}(12) = \{ \text{fn } y \Rightarrow y^4 \}$

Note, in particular, that

- $\mathbf{x}$ is always positive: $\hat{\rho}(\mathbf{x}) = \{ + \}$
- condition is always true: $\hat{C}(3) = \{ \text{tt} \}$
- else branch is unreachable: $\hat{C}(7) = \emptyset$
- e will evaluate to a value with property $\{0\}$

	$(\hat{C}, \hat{\rho})$	$(\hat{C}, \hat{\rho})$
1	$\emptyset$	$\{+\}$
2	$\emptyset$	$\{0\}$
3	$\emptyset$	$\{tt\}$
4	$\emptyset$	$\{0\}$
5	$\{fn\ y \Rightarrow y^4\}$	$\{fn\ y \Rightarrow y^4\}$
6	$\emptyset$	$\emptyset$
7	$\{fn\ z \Rightarrow 25^6\}$	$\emptyset$
8	$\{fn\ y \Rightarrow y^4, \\ fn\ z \Rightarrow 25^6\}$	$\{fn\ y \Rightarrow y^4\}$
9	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$
10	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$
11	$\emptyset$	$\{+\}$
12	$\{fn\ y \Rightarrow y^4, \\ fn\ z \Rightarrow 25^6\}$	$\{fn\ y \Rightarrow y^4\}$
13	$\emptyset$	$\{0\}$
14	$\emptyset$	$\{0\}$
15	$\emptyset$	$\{0\}$
f	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$
x	$\emptyset$	$\{+\}$
y	$\emptyset$	$\{0\}$
z	$\emptyset$	$\emptyset$

Observation

The combined analysis does not directly yield a valid 0-CFA solution

Reason

data-flow information restricts the control flow

Intuition

If the condition is known to be false (or true), some branches are not analyzed at all

Even if we restrict to terms:

- $\hat{C}'(\ell) = \hat{C}(\ell) \cap \mathbf{Term}$ and
- $\hat{\rho}'(x) = \hat{\rho}(x) \cap \mathbf{Term}$

this does not guarantee a valid CFA solution, i.e.,

$(\hat{C}, \hat{\rho}) \models_d e$ does **not** guarantee that $(\hat{C}', \hat{\rho}') \models e$

Program Analysis 2025-26

Lecture #21

Control Flow Analysis



UNIVERSITÀ
DI PISA

Abstract Values as Complete Lattices

- Instead of using sets of concrete data, we can use structured abstract domains
- We separate:
 - **Term** information
 - **Data** information

Two components

$$\hat{v} \in \widehat{\mathbf{Val}}_d = \wp(\mathbf{Term} \cup \mathbf{Data}) \equiv \wp(\mathbf{Term}) \times \wp(\mathbf{Data})$$

- replace $\wp(\mathbf{Data})$ by a complete lattice L
- and perform a development closely related to that of the (forward) Monotone Frameworks

Abstract Values as Complete Lattices

A **monotone structure** consists of:

- a complete lattice L , and
- a set F of monotone functions of $L \times L \rightarrow L$

An **instance** of a monotone structure then consists of the structure (L, F)

- a mapping ι from the constants $c \in \mathbf{Const}$ to values in L , and
- a mapping f from the binary operators $op \in \mathbf{Op}$ to functions of F

No need of flow component: it is up to the Control Flow Analysis to determine the flow

Example

Lift concrete operations to abstract values

Take the following **monotone structure**:

- $L = \wp(\mathbf{Data})$
- F : monotone functions of $L \times L \rightarrow L$

An **instance** of this monotone structure is obtained by taking

- $\iota_c = \{d_c\}$ for each constant $c \in \mathbf{Const}$ (and with $d_c \in \mathbf{Data}$)
- $f_{op}(l_1, l_2) = \bigcup \{d_{op}(d_1, d_2) \mid d_1 \in l_1, d_2 \in l_2\}$

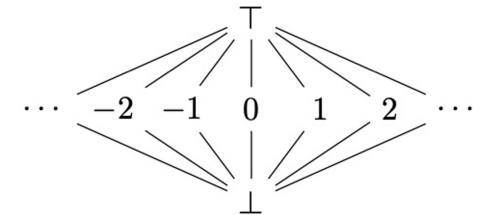
for all $op \in \mathbf{Op}$ (and where $d_{op} : \mathbf{Data} \times \mathbf{Data} \rightarrow \mathbf{Data}$ is like before)

Apply the
operator to all
pairs of possible
values

Example: Constant Propagation Analysis

A **monotone structure** for Constant Propagation Analysis

- $L = \mathbb{Z}_{\perp}^{\top} \times \wp(\{\text{tt}, \text{ff}\})$ with $\mathbb{Z}_{\perp}^{\top}$ is *flat*($\mathbb{Z}$) where $\top$ indicates that a variable is non constant
- F : monotone functions of $L \times L \rightarrow L$



An **instance** of this monotone structure is obtained by taking, e.g.,

$$l_c = (7, \emptyset) \text{ and } l_{\text{true}} = (\perp, \{\text{tt}\})$$

$$f_+(l_1, l_2) = \begin{cases} (z_1 + z_2, \emptyset) & \text{if } l_1 = (z_1, \dots), l_2 = (z_2, \dots) \text{ and } z_1, z_2 \in \mathbb{Z} \\ (\perp, \emptyset) & \text{if } l_1 = (z_1, \dots), l_2 = (z_2, \dots) \text{ and } z_1 = \perp \text{ or } z_2 = \perp \in \mathbb{Z} \\ (\top, \emptyset) & \text{otherwise} \end{cases}$$

For example:

$$f_+(l_c, l_{\text{true}}) = (\perp, \emptyset) \text{ since } z_1 \in \mathbb{Z} \text{ and } z_2 = \perp$$

f_+ propagates constants when possible, otherwise returns $\perp$ (unknown)

New CFA

We combine CFA and DFA information, by tracking both functions and data simultaneously

Abstract domains

$$\widehat{\mathbf{Val}}_d = \wp(\mathbf{Term}) \times \wp(\mathbf{Data})$$

decoupling

CFA part

DFA part

$$\begin{aligned} \hat{v} &\in \widehat{\mathbf{Val}} = \wp(\mathbf{Term}) \\ \hat{\rho} &\in \widehat{\mathbf{Env}} : \mathbf{Var} \rightarrow \widehat{\mathbf{Val}} \\ \hat{C} &\in \widehat{\mathbf{Cache}} : \mathbf{Lab} \rightarrow \widehat{\mathbf{Val}} \end{aligned}$$

$$\begin{aligned} \hat{d} &\in \widehat{\mathbf{Data}} = L \\ \hat{\delta} &\in \widehat{\mathbf{DEnv}} : \mathbf{Var} \rightarrow \widehat{\mathbf{Data}} \\ \hat{D} &\in \widehat{\mathbf{DCache}} : \mathbf{Lab} \rightarrow \widehat{\mathbf{Data}} \end{aligned}$$

Control-flow and data-flow are computed together

Acceptability relation

$$(\hat{C}, \hat{D}, \hat{\rho}, \hat{\delta}) \models_D e$$

Clause for constants

$$[con] (\hat{C}, \hat{D}, \hat{\rho}, \hat{\delta}) \models_D c^\ell \text{ iff } \iota_c \sqsubseteq \hat{D}(\ell)$$

At label ℓ , the abstract data value of c must be recorded
 ι tells us which abstract value a constant contributes

Constants inject
data-flow information
into the analysis

Example

$$c = 7$$

$$\iota_c = \{ + \}$$

$$\Rightarrow \hat{D}(\ell) \text{ must contain } \{ + \}$$

Clause for binary operation

$$[op] (\hat{C}, \hat{D}, \hat{\rho}, \hat{\delta}) \models_D (t_1^{\ell_1} \text{ op } t_2^{\ell_2})^\ell \text{ iff } (\hat{C}, \hat{D}, \hat{\rho}, \hat{\delta}) \models_D t_1^{\ell_1} \wedge (\hat{C}, \hat{D}, \hat{\rho}, \hat{\delta}) \models_D t_2^{\ell_2} \wedge$$
$$f_{op}(\hat{D}(\ell_1), \hat{D}(\ell_2)) \sqsubseteq \hat{D}(\ell)$$

Now, we use the component f .

Compute the abstract result of the operation and propagate it to the result label

Example

$x \in \{+, 0\}, y \in \{+\}$

$\Rightarrow x + y \in \{+, 0, -\}$

$\Rightarrow \hat{D}(\ell)$ must include $\{+, 0, -\}$

Clause for conditional

$$[if] (\hat{C}, \hat{D}, \hat{\rho}, \hat{\delta}) \models_D (\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell$$

$$\text{iff } (\hat{C}, \hat{D}, \hat{\rho}, \hat{\delta}) \models_D t_0^{\ell_0} \wedge$$

$$(\iota_{\text{true}} \sqsubseteq \hat{D}(\ell_0) \Rightarrow (\hat{C}, \hat{D}, \hat{\rho}, \hat{\delta}) \models_D t_1^{\ell_1} \wedge \hat{C}(\ell_1) \subseteq \hat{C}(\ell) \wedge \hat{D}(\ell_1) \sqsubseteq \hat{D}(\ell)) \wedge$$

$$(\iota_{\text{false}} \sqsubseteq \hat{D}(\ell_0) \Rightarrow (\hat{C}, \hat{D}, \hat{\rho}, \hat{\delta}) \models_D t_2^{\ell_2} \wedge \hat{C}(\ell_2) \subseteq \hat{C}(\ell) \wedge \hat{D}(\ell_2) \sqsubseteq \hat{D}(\ell))$$

The clause [if]

- analyze condition
- use abstract boolean value and propagate only analysed branches

Example

$$e \triangleq (\text{if } (3^1 > 0^2)^3 \text{ then } (\text{fn } y \Rightarrow y^4)^5 \text{ else } (\text{fn } z \Rightarrow z^6)^7)^8$$

$$\iota_{\text{true}} \sqsubseteq \hat{D}(3) \Rightarrow \hat{C}(5) \subseteq \hat{C}(8) \text{ and } \hat{D}(5) \subseteq \hat{D}(8) \dots \iota_{\text{false}} \not\sqsubseteq \hat{D}(3) \Rightarrow$$

CFA with abstract values as complete lattices

$$\begin{aligned}
 [\text{con}] \quad & (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D c^\ell \text{ iff } \iota_c \subseteq \widehat{D}(\ell) \\
 [\text{var}] \quad & (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D x^\ell \text{ iff } \widehat{\rho}(x) \subseteq \widehat{C}(\ell) \wedge \widehat{\delta}(x) \subseteq \widehat{D}(\ell) \\
 [\text{fn}] \quad & (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D (\text{fn } x \Rightarrow e_0)^\ell \text{ iff } \{\text{fn } x \Rightarrow e_0\} \subseteq \widehat{C}(\ell) \\
 [\text{fun}] \quad & (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D (\text{fun } f \ x \Rightarrow e_0)^\ell \text{ iff } \{\text{fun } f \ x \Rightarrow e_0\} \subseteq \widehat{C}(\ell) \\
 [\text{app}] \quad & (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D (t_1^{\ell_1} \ t_2^{\ell_2})^\ell \\
 & \text{ iff } (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D t_1^{\ell_1} \wedge (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D t_2^{\ell_2} \wedge \\
 & \quad (\forall (\text{fn } x \Rightarrow t_0^{\ell_0}) \in \widehat{C}(\ell_1) : \\
 & \quad \quad (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D t_0^{\ell_0} \wedge \\
 & \quad \quad \widehat{C}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{D}(\ell_2) \subseteq \widehat{\delta}(x) \wedge \\
 & \quad \quad \widehat{C}(\ell_0) \subseteq \widehat{C}(\ell) \wedge \widehat{D}(\ell_0) \subseteq \widehat{D}(\ell)) \wedge \\
 & \quad (\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}) \in \widehat{C}(\ell_1) : \\
 & \quad \quad (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D t_0^{\ell_0} \wedge \\
 & \quad \quad \widehat{C}(\ell_2) \subseteq \widehat{\rho}(x) \wedge \widehat{D}(\ell_2) \subseteq \widehat{\delta}(x) \wedge \\
 & \quad \quad \widehat{C}(\ell_0) \subseteq \widehat{C}(\ell) \wedge \widehat{D}(\ell_0) \subseteq \widehat{D}(\ell) \wedge \\
 & \quad \quad \{\text{fun } f \ x \Rightarrow t_0^{\ell_0}\} \subseteq \widehat{\rho}(f)) \\
 [\text{if}] \quad & (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D (\text{if } t_0^{\ell_0} \text{ then } t_1^{\ell_1} \text{ else } t_2^{\ell_2})^\ell \\
 & \text{ iff } (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D t_0^{\ell_0} \wedge \\
 & \quad (\iota_{\text{true}} \subseteq \widehat{D}(\ell_0) \Rightarrow (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D t_1^{\ell_1} \wedge \\
 & \quad \quad \widehat{C}(\ell_1) \subseteq \widehat{C}(\ell) \wedge \\
 & \quad \quad \widehat{D}(\ell_1) \subseteq \widehat{D}(\ell)) \wedge \\
 & \quad (\iota_{\text{false}} \subseteq \widehat{D}(\ell_0) \Rightarrow (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D t_2^{\ell_2} \wedge \\
 & \quad \quad \widehat{C}(\ell_2) \subseteq \widehat{C}(\ell) \wedge \\
 & \quad \quad \widehat{D}(\ell_2) \subseteq \widehat{D}(\ell)) \\
 [\text{let}] \quad & (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D (\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^\ell \\
 & \text{ iff } (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D t_1^{\ell_1} \wedge (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D t_2^{\ell_2} \wedge \\
 & \quad \widehat{C}(\ell_1) \subseteq \widehat{\rho}(x) \wedge \widehat{D}(\ell_1) \subseteq \widehat{\delta}(x) \wedge \\
 & \quad \widehat{C}(\ell_2) \subseteq \widehat{C}(\ell) \wedge \widehat{D}(\ell_2) \subseteq \widehat{D}(\ell) \\
 [\text{op}] \quad & (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D (t_1^{\ell_1} \text{ op } t_2^{\ell_2})^\ell \\
 & \text{ iff } (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D t_1^{\ell_1} \wedge (\widehat{C}, \widehat{D}, \widehat{\rho}, \widehat{\delta}) \models_D t_2^{\ell_2} \wedge \\
 & \quad f_{\text{op}}(\widehat{D}(\ell_1), \widehat{D}(\ell_2)) \subseteq \widehat{D}(\ell)
 \end{aligned}$$

Back to our Example 3

$$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow (\text{if } (x^1 > 0^2)^3 \text{ then } (\text{fn } y \Rightarrow y^4)^5 \\ \text{else } (\text{fn } z \Rightarrow 25^6)^7)^8)^9 \\ \text{in } ((f^{10} \ 3^{11})^{12} \ 0^{13})^{14})^{15}$$

The last column describes the solution $(\hat{C}, \hat{D}, \hat{\rho}, \hat{\delta})$, where it is evident the splitting between

- the control flow component $(\hat{C}, \hat{\rho})$
- and the data flow component $(\hat{D}, \hat{\delta})$

Note, e.g., that

- x is always positive: $\delta(\hat{x}) = \{ + \}$
- condition is always true: $\hat{D}(3) = \{ \text{tt} \}$

	Pure CFA	CFA (powersets)	CFA (complete lattices)
	Section 3.1	Subsection 3.5.1	Subsection 3.5.2
	$(\widehat{C}, \widehat{\rho})$	$(\widehat{C}, \widehat{\rho})$	$(\widehat{C}, \widehat{\rho})$ $(\widehat{D}, \widehat{\delta})$
1	$\emptyset$	$\{+\}$	$\emptyset$ $\{+\}$
2	$\emptyset$	$\{0\}$	$\emptyset$ $\{0\}$
3	$\emptyset$	$\{tt\}$	$\emptyset$ $\{tt\}$
4	$\emptyset$	$\{0\}$	$\emptyset$ $\{0\}$
5	$\{fn\ y \Rightarrow y^4\}$	$\{fn\ y \Rightarrow y^4\}$	$\{fn\ y \Rightarrow y^4\}$ $\emptyset$
6	$\emptyset$	$\emptyset$	$\emptyset$ $\emptyset$
7	$\{fn\ z \Rightarrow 25^6\}$	$\emptyset$	$\emptyset$ $\emptyset$
8	$\{fn\ y \Rightarrow y^4, \\ fn\ z \Rightarrow 25^6\}$	$\{fn\ y \Rightarrow y^4\}$	$\{fn\ y \Rightarrow y^4\}$ $\emptyset$
9	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$ $\emptyset$
10	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$ $\emptyset$
11	$\emptyset$	$\{+\}$	$\emptyset$ $\{+\}$
12	$\{fn\ y \Rightarrow y^4, \\ fn\ z \Rightarrow 25^6\}$	$\{fn\ y \Rightarrow y^4\}$	$\{fn\ y \Rightarrow y^4\}$ $\emptyset$
13	$\emptyset$	$\{0\}$	$\emptyset$ $\{0\}$
14	$\emptyset$	$\{0\}$	$\emptyset$ $\{0\}$
15	$\emptyset$	$\{0\}$	$\emptyset$ $\{0\}$
f	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$ $\emptyset$
x	$\emptyset$	$\{+\}$	$\emptyset$ $\{+\}$
y	$\emptyset$	$\{0\}$	$\emptyset$ $\{0\}$
z	$\emptyset$	$\emptyset$	$\emptyset$ $\emptyset$

Correctness of the analysis

The combined CFA + DFA is sound w.r.t. the operational semantics

- Same proof techniques as before apply
- The algorithm terminates if L satisfies the **Ascending Chain Condition (ACC)**, as in the case for Monotone Frameworks

no infinite strictly increasing sequences
⇒ fixpoint computation terminates



Staging the specification

- If we change the [if] clause in such a way that the data flow component cannot influence the control flow component, with a modified relation $\models'_D$, then $(\hat{C}, \hat{D}, \hat{\rho}, \hat{\delta}) \models'_D e$ guarantees that $(\hat{C}, \hat{\rho}) \models e$

We can solve:

1. control-flow ($\hat{C}$ and $\hat{\rho}$) first
2. data-flow ($\hat{D}$ and $\hat{\delta}$) afterwards

- The analysis still yields the **least solution** of the combined set of constraints

Back to Example 3

$$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow (\text{if } (x^1 > 0^2)^3 \text{ then } (\text{fn } y \Rightarrow y^4)^5 \\ \text{else } (\text{fn } z \Rightarrow 25^6)^7)^8)^9 \\ \text{in } ((f^{10} \ 3^{11})^{12} \ 0^{13})^{14})^{15}$$

If we modify the [if] clause as discussed above then $\hat{C}$ and $\hat{\rho}$ are as in pure CFA, while $\hat{D}$ and $\hat{\delta}$ associate slightly larger sets with some labels and variables

- $\hat{D}(6) = \{ + \}$
- $\hat{D}(14) = \{ 0, + \}$
- $\hat{D}(15) = \{ 0, + \}$
- $\hat{\delta}(z) = \{ 0 \}$

	Pure CFA	CFA (powersets)	CFA (complete lattices)	
	Section 3.1	Subsection 3.5.1	Subsection 3.5.2	
	$(\widehat{C}, \widehat{\rho})$	$(\widehat{C}, \widehat{\rho})$	$(\widehat{C}, \widehat{\rho})$	$(\widehat{D}, \widehat{\delta})$
1	$\emptyset$	$\{+\}$	$\emptyset$	$\{+\}$
2	$\emptyset$	$\{0\}$	$\emptyset$	$\{0\}$
3	$\emptyset$	$\{tt\}$	$\emptyset$	$\{tt\}$
4	$\emptyset$	$\{0\}$	$\emptyset$	$\{0\}$
5	$\{fn\ y \Rightarrow y^4\}$	$\{fn\ y \Rightarrow y^4\}$	$\{fn\ y \Rightarrow y^4\}$	$\emptyset$
6	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
7	$\{fn\ z \Rightarrow 25^6\}$	$\emptyset$	$\emptyset$	$\emptyset$
8	$\{fn\ y \Rightarrow y^4,$ $fn\ z \Rightarrow 25^6\}$	$\{fn\ y \Rightarrow y^4\}$	$\{fn\ y \Rightarrow y^4\}$	$\emptyset$
9	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$	$\emptyset$
10	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$	$\emptyset$
11	$\emptyset$	$\{+\}$	$\emptyset$	$\{+\}$
12	$\{fn\ y \Rightarrow y^4,$ $fn\ z \Rightarrow 25^6\}$	$\{fn\ y \Rightarrow y^4\}$	$\{fn\ y \Rightarrow y^4\}$	$\emptyset$
13	$\emptyset$	$\{0\}$	$\emptyset$	$\{0\}$
14	$\emptyset$	$\{0\}$	$\emptyset$	$\{0\}$
15	$\emptyset$	$\{0\}$	$\emptyset$	$\{0\}$
f	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$	$\{fn\ x \Rightarrow (\dots)^8\}$	$\emptyset$
x	$\emptyset$	$\{+\}$	$\emptyset$	$\{+\}$
y	$\emptyset$	$\{0\}$	$\emptyset$	$\{0\}$
z	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

More instances and precision

Example 4

$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow x^1)^2$
 $\text{in } ((f^3 f^4)^5 (\text{fn } y \Rightarrow y^6)^7)^8)^9$

The least O-CFA analysis is given by $(\hat{C}_{id}, \hat{\rho}_{id})$

- $\hat{C}_{id}(1) = \{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^6\}$
- $\hat{C}_{id}(2) = \{\text{fn } x \Rightarrow x^1\}$
- $\hat{C}_{id}(3) = \{\text{fn } x \Rightarrow x^1\}$
- $\hat{C}_{id}(4) = \{\text{fn } x \Rightarrow x^1\}$
- $\hat{C}_{id}(5) = \{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^6\}$
- $\hat{C}_{id}(6) = \{\text{fn } y \Rightarrow y^6\}$
- $\hat{C}_{id}(7) = \{\text{fn } y \Rightarrow y^6\}$
- $\hat{C}_{id}(8) = \{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^6\}$
- $\hat{C}_{id}(9) = \{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^6\}$
- $\hat{\rho}_{id}(f) = \{\text{fn } x \Rightarrow x^1\}$
- $\hat{\rho}_{id}(x) = \{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^6\}$
- $\hat{\rho}_{id}(y) = \{\text{fn } y \Rightarrow y^6\}$

Only $\text{fn } y \Rightarrow y^6$ is a possible result of the evaluation of e
 O-CFA merges different calling contexts $\Rightarrow$ **loss of precision**

Example 4 bis

Idea: duplicate functions per call site

$e' \triangleq$ (let $f_1 = (\text{fn } x_1 \Rightarrow x_1^1)^2$
in (let $f_2 = (\text{fn } x_2 \Rightarrow x_2^3)^4$
in ((f_1^5 f_2^6)⁷ ($\text{fn } y \Rightarrow y^8$)⁹)¹⁰)¹¹)¹²

separate each use of f

• ...

• $\hat{C}_{id}(12) = \{\text{fn } y \Rightarrow y^8\}$

• $\hat{\rho}_{id}(x_1) = \{\text{fn } x_2 \Rightarrow x_2^3\}$

• $\hat{\rho}_{id}(x_2) = \{\text{fn } y \Rightarrow y^8\}$

Now each parameter is bound in a single context:

• x_1 can only be bound to $\text{fn } x_2 \Rightarrow x_2^3$ and

• x_2 can only be bound to $\text{fn } y \Rightarrow y^8$

• so no merging of flows $\Rightarrow$ precise result: $\text{fn } y \Rightarrow y^8$ only

precision comes from
separating flows

Adding Context Information

Uniform k -CFA Analysis

Adding contexts

- 0-CFA is a **context-insensitive** (or **mono-variant**) analysis
- It does not distinguish various instances of program variables and program points from each other, in particular does not distinguish different calling contexts
- **Context-sensitive** (or **poly-variant**) analysis does distinguish
 - Examples of such analyses include **k -CFA** analyses, uniform **k -CFA** analyses, polynomial **k -CFA** analyses and the Cartesian Product Algorithm
- We focus on **uniform k -CFA** (“uniform” because environment and cache have the same precision)

Abstract domains: contexts

Goal: distinguish different function calls

Idea: extend $\widehat{\mathbf{Val}}$ to include context information

- **Contexts** $\hat{\delta}$ record the k last dynamic call sites, e.g., for $k = 1$, the context amounts just to the label of the expression that called the current function
- In general, contexts are sequences of labels of length at most $\leq k$

$$\hat{\delta} \in \Delta = \mathbf{Lab}^{\leq k}$$

We track information separately for different call strings*

- Same function, different contexts $\Rightarrow$ different analyses

*A **call string** is a list of labels $\ell_1, \dots, \ell_n$ denoting that the last n calls were of the form $(e_{1n} \ e_{2n})^{\ell_n} \dots (e_{11} \ e_{21})^{\ell_1}$ with $(e_{11} \ e_{21})^{\ell_1}$ being the most recent call; we write ϵ for the empty call string

Abstract domains: context environment

- A **context environment** records in which context each variable was defined
- So we associate a context to each variable occurrence

Definition points of free
variables of terms

$$ce \in \mathbf{Cenv} = \mathbf{Var} \rightarrow \Delta$$

- The same variable may have different values in different contexts
- The context environment will play a role similar to the environment of the semantics

Context-sensitive abstract values

In our set of values, instead of just the function abstractions, we need to track **closures**, i.e., a pair of a function label and a context environment

- A value is paired with the context environment where it was created
- We shall extend the abstract values as

$$\hat{v} \in \widehat{\mathbf{Val}} = \wp(\mathbf{Val} \times \mathbf{Cenv})$$

Context-sensitive environments

- Abstract environment $\hat{\rho}$ and abstract cache $\hat{C}$ will now map a variable (a label, resp.) and a context to an abstract value

$$\begin{aligned}\hat{\rho} &\in \widehat{\mathbf{Cenv}} = (\mathbf{Var} \times \Delta) \rightarrow \widehat{\mathbf{Val}} \\ \hat{C} &\in \widehat{\mathbf{Cache}} = (\mathbf{Lab} \times \Delta) \rightarrow \widehat{\mathbf{Val}}\end{aligned}$$

- Values are now indexed by context
- This treatment prevents merging values from different calls (as in the previous example)

k -CFAs

- In **uniform k -CFA** we have a cache for each possible context

$$\hat{C} \in \widehat{\mathbf{Cache}} = (\mathbf{Lab} \times \Delta) \rightarrow \widehat{\mathbf{Val}}$$

where Δ : set of call strings (length $\leq k$)

- In the **classical k -CFA**, we have instead a cache for each possible full environment

more expensive

$$\hat{C} \in \widehat{\mathbf{Cache}} = (\mathbf{Lab} \times \mathbf{CEnv}) \rightarrow \widehat{\mathbf{Val}}$$

- In the **polynomial k -CFA**, instead of $\widehat{\mathbf{Val}} = \wp(\mathbf{Val} \times \mathbf{Cenv})$ we have

$$\hat{v} \in \widehat{\mathbf{Val}} = \wp(\mathbf{Val} \times \Delta)$$

To recap

$$ce \in \mathbf{Cenv} = \mathbf{Var} \rightarrow \Delta$$

$$\hat{\delta} \in \Delta = \mathbf{Lab}^{\leq k}$$

$$\hat{v} \in \widehat{\mathbf{Val}} = \wp(\mathbf{Val} \times \mathbf{Cenv})$$

$$\hat{\rho} \in \widehat{\mathbf{Cenv}} = (\mathbf{Var} \times \Delta) \rightarrow \widehat{\mathbf{Val}}$$

$$\hat{C} \in \widehat{\mathbf{Cache}} = (\mathbf{Lab} \times \Delta) \rightarrow \widehat{\mathbf{Val}}$$

Acceptability relation

$$(\hat{C}, \hat{\rho}) \models_{\delta}^{ce} e$$

- ce is the current context environment:
 - it tracks the context in which each variable of e is bound
 - it changes when variables are bound
- $\hat{\delta}$ is the current calling context
 - it changes when functions are applied

The formula expresses that $(\hat{C}, \hat{\rho})$ is an acceptable analysis of e in the context specified by ce and $\hat{\delta}$

Acceptability is now **context-dependent**: we analyse e in a specific calling context $\hat{\delta}$

Clause for variables

$$[var] (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} x^{\ell} \text{ iff } \hat{\rho}(x, ce(x)) \subseteq \hat{C}(\ell, \delta)$$

We look up x in the context where it was bound and propagate it to the current context

Same variable,
different contexts $\rightarrow$ different values

- $ce(x)$ context where x was defined [definition context]
- δ : current context (where we use x) [use context]

A variable carries the context where it was created, but is used in the current context

Example

Suppose x was bound at call site 5, i.e., $ce_2(x) = 5$, then

$$(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_5^{ce_2} x^1 \text{ iff } \hat{\rho}'_{id}(x, 5) \subseteq \hat{C}'_{id}(1, 5) \text{ [see Example 4]}$$

Clause for functions

$$[fn] (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} (fn\ x \Rightarrow e_0)^{\ell} \text{ iff } \{(fn\ x \Rightarrow e_0, ce_0)\} \subseteq \hat{C}(\ell, \delta) \\ \text{where } ce_0 = ce \upharpoonright FV(fn\ x \Rightarrow e_0)$$

The functional term $fn\ x \Rightarrow e_0$ must be included in $\hat{C}(\ell, \delta)$ together with the current context environment ce , restricted to the free variables of the function (denoted ce_0)

Functions are represented as **closures**: a function is paired with the context in which it is defined, restricted to its free variables

Example: Suppose Λ be the empty sequence of labels and $ce = [x \mapsto 5, y \mapsto 3]$

$$(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce} (fn\ x \Rightarrow x^1)^2 \text{ iff } \{(fn\ x \Rightarrow x^1, ce_0)\} \subseteq \hat{C}'_{id}(2, \Lambda)$$

with $ce_0 = []$ since $FV(fn\ x \Rightarrow x^1) = \emptyset$ [see Example 4]

Note that if the function were $(fn\ x \Rightarrow y)$, then $FV(fn\ x \Rightarrow y) = \{y\}$

environment at application time

Clause for the let binding

$$[let] (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} (\text{let } x = t_1^{\ell_1} \text{ in } t_2^{\ell_2})^{\ell} \text{ iff } (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_{\delta}^{ce'} t_2^{\ell_2} \wedge \hat{C}(\ell_1, \delta) \subseteq \hat{\rho}(x, \delta) \wedge \hat{C}(\ell_2, \delta) \subseteq \hat{C}(\ell, \delta) \text{ where } ce' = ce[x \mapsto \delta]$$

we record that x is defined in the current context δ

This clause

- requires checking the **sub-expressions**
- the definition context of the bound variable of the let-construct is the current context (as no application point is passed)

Double flow: propagation of the values

- from $\hat{C}(\ell_1, \delta)$ to $\hat{\rho}(x, \delta)$ [**binding**]
- from $\hat{C}(\ell_2, \delta)$ to $\hat{C}(\ell, \delta)$ [**evaluation**]

Example

$$(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{\square} (\text{let } f = (\text{fn } x \Rightarrow x^1)^2 \text{ in } (f^3 3^4)^5)^6 \text{ iff } (\hat{C}, \hat{\rho}) \models_{\Lambda}^{\square} t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_{\delta}^{[x \mapsto \delta]} t_2^{\ell_2} \wedge \hat{C}(2, \Lambda) \subseteq \hat{\rho}(f, \Lambda) \text{ and } \hat{C}(5, \Lambda) \subseteq \hat{C}(6, \Lambda)$$

Clause for function application

$$[app] (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff } (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_2^{\ell_2} \wedge$$

both subexpressions are analysed in the same context δ and environment ce

$$(\forall (\text{fn } x \Rightarrow t_0^{\ell_0}, ce_0) \in \hat{C}(\ell_1, \delta) :$$

$$(\hat{C}, \hat{\rho}) \models_{\delta_0}^{ce_0} t_0^{\ell_0} \wedge$$

operator may evaluate to a function closure

$$\hat{C}(\ell_2, \delta) \subseteq \hat{\rho}(x, \delta) \wedge \hat{C}(\ell_0, \delta) \subseteq \hat{C}(\ell, \delta))$$

body analysed in the updated context δ_0 using the updated closure environment ce_0

$$\text{where } \delta_0 = [\delta, \ell]_k \text{ and}$$

$$ce'_0 = ce_0[x \mapsto \delta_0]$$

we extend the current context with the call site ℓ

Propagation as before

x is now bound in the new calling context δ_0 but using the definition environment ce_0

Δ

Clause for function application

$$[app] (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff } (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_2^{\ell_2} \wedge$$

$$\wedge \dots$$

- We proceed in a similar way
- both f and x are bound in the new calling context δ_0 when analyzing the body

$$(\forall (\text{fun } f \ x \Rightarrow t_0^{\ell_0}, ce_0) \in \hat{C}(\ell_1, \delta) :$$

$$(\hat{C}, \hat{\rho}) \models_{\delta_0}^{ce_0'} t_0^{\ell_0} \wedge$$

$$\hat{C}(\ell_2, \delta) \subseteq \hat{\rho}(x, \delta) \wedge \hat{C}(\ell_0, \delta) \subseteq \hat{C}(\ell, \delta))$$

$$\{(\text{fun } f \ x \Rightarrow t_0^{\ell_0}, ce_0)\} \in \hat{\rho}(f, \delta_0) :$$

$$\text{where } \delta_0 = [\delta, \ell]_k \text{ and}$$

$$ce_0' = ce_0[f \mapsto \delta_0, x \mapsto \delta_0]$$

$$\wedge \dots$$

Uniform k -CFA

- [con] $(\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} c^{\ell}$ always
- [var] $(\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} x^{\ell}$ iff $\widehat{\rho}(x, ce(x)) \subseteq \widehat{C}(\ell, \delta)$
- [fn] $(\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} (\mathbf{fn} \ x \Rightarrow e_0)^{\ell}$ iff $\{(\mathbf{fn} \ x \Rightarrow e_0, ce_0)\} \subseteq \widehat{C}(\ell, \delta)$
where $ce_0 = ce \mid FV(\mathbf{fn} \ x \Rightarrow e_0)$
- [fun] $(\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} (\mathbf{fun} \ f \ x \Rightarrow e_0)^{\ell}$ iff $\{(\mathbf{fun} \ f \ x \Rightarrow e_0, ce_0)\} \subseteq \widehat{C}(\ell, \delta)$
where $ce_0 = ce \mid FV(\mathbf{fun} \ f \ x \Rightarrow e_0)$
- [app] $(\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} (t_1^{\ell_1} t_2^{\ell_2})^{\ell}$
iff $(\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} t_2^{\ell_2} \wedge$
 $(\forall (\mathbf{fn} \ x \Rightarrow t_0^{\ell_0}, ce_0) \in \widehat{C}(\ell_1, \delta) :$
 $(\widehat{C}, \widehat{\rho}) \models_{\delta_0}^{ce'_0} t_0^{\ell_0} \wedge$
 $\widehat{C}(\ell_2, \delta) \subseteq \widehat{\rho}(x, \delta_0) \wedge \widehat{C}(\ell_0, \delta_0) \subseteq \widehat{C}(\ell, \delta)$
where $\delta_0 = [\delta, \ell]_k$
and $ce'_0 = ce_0[x \mapsto \delta_0]) \wedge$
 $(\forall (\mathbf{fun} \ f \ x \Rightarrow t_0^{\ell_0}, ce_0) \in \widehat{C}(\ell_1, \delta) :$
 $(\widehat{C}, \widehat{\rho}) \models_{\delta_0}^{ce'_0} t_0^{\ell_0} \wedge$
 $\widehat{C}(\ell_2, \delta) \subseteq \widehat{\rho}(x, \delta_0) \wedge \widehat{C}(\ell_0, \delta_0) \subseteq \widehat{C}(\ell, \delta) \wedge$
 $\{(\mathbf{fun} \ f \ x \Rightarrow t_0^{\ell_0}, ce_0)\} \subseteq \widehat{\rho}(f, \delta_0)$
where $\delta_0 = [\delta, \ell]_k$
and $ce'_0 = ce_0[f \mapsto \delta_0, x \mapsto \delta_0])$
- [if] $(\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} (\mathbf{if} \ t_0^{\ell_0} \mathbf{then} \ t_1^{\ell_1} \mathbf{else} \ t_2^{\ell_2})^{\ell}$
iff $(\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} t_0^{\ell_0} \wedge (\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} t_2^{\ell_2} \wedge$
 $\widehat{C}(\ell_1, \delta) \subseteq \widehat{C}(\ell, \delta) \wedge \widehat{C}(\ell_2, \delta) \subseteq \widehat{C}(\ell, \delta)$
- [let] $(\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} (\mathbf{let} \ x = t_1^{\ell_1} \mathbf{in} \ t_2^{\ell_2})^{\ell}$
iff $(\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce'} t_2^{\ell_2} \wedge$
 $\widehat{C}(\ell_1, \delta) \subseteq \widehat{\rho}(x, \delta) \wedge \widehat{C}(\ell_2, \delta) \subseteq \widehat{C}(\ell, \delta)$
where $ce' = ce[x \mapsto \delta]$
- [op] $(\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} (t_1^{\ell_1} \ \mathbf{op} \ t_2^{\ell_2})^{\ell}$ iff $(\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge (\widehat{C}, \widehat{\rho}) \models_{\delta}^{ce} t_2^{\ell_2}$

Back to Example 4: 0-CFA

$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow x^1)^2$
 $\text{in } ((f^3 f^4)^5 (\text{fn } y \Rightarrow y^6)^7)^8)^9$

The least 0-CFA analysis is given by $(\hat{C}_{id}, \hat{\rho}_{id})$

- $\hat{C}_{id}(1) = \{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^6\}$
- $\hat{C}_{id}(2) = \{\text{fn } x \Rightarrow x^1\}$
- $\hat{C}_{id}(3) = \{\text{fn } x \Rightarrow x^1\}$
- $\hat{C}_{id}(4) = \{\text{fn } x \Rightarrow x^1\}$
- $\hat{C}_{id}(5) = \{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^6\}$
- $\hat{C}_{id}(6) = \{\text{fn } y \Rightarrow y^6\}$
- $\hat{C}_{id}(7) = \{\text{fn } y \Rightarrow y^6\}$
- $\hat{C}_{id}(8) = \{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^6\}$
- $\hat{C}_{id}(9) = \{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^6\}$
- $\hat{\rho}_{id}(f) = \{\text{fn } x \Rightarrow x^1\}$
- $\hat{\rho}_{id}(x) = \{\text{fn } x \Rightarrow x^1, \text{fn } y \Rightarrow y^6\}$
- $\hat{\rho}_{id}(y) = \{\text{fn } y \Rightarrow y^6\}$

Only $\text{fn } y \Rightarrow y^6$ is a possible result of the evaluation of e
 0-CFA merges different calling contexts $\Rightarrow$ **loss of precision**

Back to Example 4: uniform 1-CFA

$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow x^1)^2$
in $((f^3 f^4)^5 (\text{fn } y \Rightarrow y^6)^7)^8)^9$

- The initial context will be Λ , the empty sequence of labels
- In the course of the analysis the current context will be modified at the two application points with labels 5 and 8
- since we only records call strings of length at most 1 the only contexts of interest will therefore be in $\{\Lambda, 5, 8\}$
- In 0-CFA the two calls are merged
- In 1-CFA they are instead separated because they have different call sites: this is enough to restore precision

Back to Example 4: uniform 1-CFA

$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow x^1)^2$
 $\text{in } ((f^3 f^4)^5 (\text{fn } y \Rightarrow y^6)^7)^8)^9$

- Context environments generated during the analysis:
 - $ce_0 = []$ initial (empty) context environment
 - $ce_1 = ce_0[f \mapsto \Lambda]$ used for analyzing the body of the let: f is defined in context Λ
 - $ce_2 = ce_0[x \mapsto 5]$ used for analyzing the body of f : x bound at call site 5
 - $ce_3 = ce_0[x \mapsto 8]$ used for analyzing the body of f : x bound at call site 8

We start from the closure environment ce_0 (captured at definition time), not from the current ce

Back to Example 4: uniform 1-CFA

$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow x^1)^2$
 $\text{in } ((f^3 f^4)^5 (\text{fn } y \Rightarrow y^6)^7)^8)^9$

Look at the solution $(\hat{C}'_{id}, \hat{\rho}'_{id})$ • $\hat{C}'_{id}(1,5) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$ • $\hat{\rho}'_{id}(f, \Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$

• $\hat{C}'_{id}(2, \Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$ • $\hat{\rho}'_{id}(x, 5) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$

• $\hat{C}'_{id}(4, \Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$ • $\hat{\rho}'_{id}(x, 8) = \{(\text{fn } y \Rightarrow y^6, ce_0)\}$

With this CFA we can:

- distinguish the values of x in the two calling contexts

- determine that $\text{fn } y \Rightarrow y^6$ is the **only** result possible

• $\hat{C}'_{id}(7, \Lambda) = \{(\text{fn } y \Rightarrow y^6, ce_0)\}$

• $\hat{C}'_{id}(9, \Lambda) = \{(\text{fn } y \Rightarrow y^6, ce_0)\}$

• $\hat{C}'_{id}(1, 8) = \{(\text{fn } y \Rightarrow y^6, ce_0)\}$

• $\hat{C}'_{id}(3, \Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$

• $\hat{C}'_{id}(5, \Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$

• $\hat{C}'_{id}(8, \Lambda) = \{(\text{fn } y \Rightarrow y^6, ce_0)\}$

Back to Example 4: 1-CFA

$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow x^1)^2$
 $\text{in } ((f^3 f^4)^5 (\text{fn } y \Rightarrow y^6)^7 (8)^9)$

We check acceptability

To check acceptability, i.e.,
 $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_0} e$ we proceed by
 clauses:

- [let], to check the update of the context environment
- [app] at label 8, to check check x^1 in context 8 and
- [app] at label 5, to check check x^1 in context 8
- for each closure, we check argument + body in updated context

- $\hat{C}'_{id}(1,5) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$
- $\hat{C}'_{id}(2,\Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$
- $\hat{C}'_{id}(4,\Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$
- $\hat{C}'_{id}(7,\Lambda) = \{(\text{fn } y \Rightarrow y^6, ce_0)\}$
- $\hat{C}'_{id}(9,\Lambda) = \{(\text{fn } y \Rightarrow y^6, ce_0)\}$
- $\hat{C}'_{id}(1,8) = \{(\text{fn } y \Rightarrow y^6, ce_0)\}$
- $\hat{C}'_{id}(3,\Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$
- $\hat{C}'_{id}(5,\Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$
- $\hat{C}'_{id}(8,\Lambda) = \{(\text{fn } y \Rightarrow y^6, ce_0)\}$
- $\hat{\rho}'_{id}(f, \Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$
- $\hat{\rho}'_{id}(x, 5) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$
- $\hat{\rho}'_{id}(x, 8) = \{(\text{fn } y \Rightarrow y^6, ce_0)\}$

Back to Example 4: 1-CFA

$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow x^1)^2$
 $\text{in } ((f^3 f^4)^5 (\text{fn } y \Rightarrow y^6)^7)^8)^9$

We check acceptability

[let]: To check $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_0} e$ it suffices ([let]) to verify
 $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_0} (\text{fn } x \Rightarrow x^1)^2$ and $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} ((f^3 f^4)^5 (\text{fn } y \Rightarrow y^6)^7)^8$
 because $\hat{C}'_{id}(2, \Lambda) \subseteq \hat{\rho}'_{id}(f, \Lambda)$ and $\hat{C}'_{id}(8, \Lambda) \subseteq \hat{C}'_{id}(9, \Lambda)$

$$ce_0 = []$$

$$ce_1 = ce_0[f \mapsto \Lambda]$$

[app] at label 8: Since $\hat{C}'_{id}(5, \Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$, to check
 $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} ((f^3 f^4)^5 (\text{fn } y \Rightarrow y^6)^7)^8$ suffices ([app]) to verify
 $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} (f^3 f^4)^5$ and $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} (\text{fn } y \Rightarrow y^6)^7$ and $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_8^{ce_3} x^1$ iff $\hat{\rho}_{id}(x, 8) \subseteq \hat{C}'_{id}(8, \Lambda)$
 because $\hat{C}'_{id}(7, \Lambda) \subseteq \hat{\rho}'_{id}(x, 8)$ and $\hat{C}'_{id}(1, 8) \subseteq \hat{C}'_{id}(8, \Lambda)$

$$ce_3 = ce_0[x \mapsto 8]$$

$$\delta_0 = [\Lambda, 8]_1 = 8$$

[app] at label 5: Since $\hat{C}'_{id}(3, \Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$, to check
 $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} (f^3 f^4)^5$ suffices ([app]) to verify
 $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} f^3$ and $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} f^4$ and $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_5^{ce_2} x^1$
 because $\hat{C}'_{id}(4, \Lambda) \subseteq \hat{\rho}'_{id}(x, 5)$ and $\hat{C}'_{id}(1, 5) \subseteq \hat{C}'_{id}(5, \Lambda)$

$$ce_2 = ce_0[x \mapsto 5]$$

$$\delta_0 = [\Lambda, 5]_1 = 5$$

Back to Example 4: 1-CFA

$e \triangleq (\text{let } f = (\text{fn } x \Rightarrow x^1)^2$
 $\text{in } ((f^3 f^4)^5 (\text{fn } y \Rightarrow y^6)^7)^8)^9$

We check acceptability

[let]: To check $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_0} e$ it suffices ([let]) to verify
 $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_0} (\text{fn } x \Rightarrow x^1)^2$ and $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} ((f^3 f^4)^5 (\text{fn } y \Rightarrow y^6)^7)^8$
 because $\hat{C}'_{id}(2, \Lambda) \subseteq \hat{\rho}'_{id}(f, \Lambda)$ and $\hat{C}'_{id}(8, \Lambda) \subseteq \hat{C}'_{id}(9, \Lambda)$

$ce_0 = []$

$ce_1 = ce_0[f \mapsto \Lambda]$

[app] at label 8: Since $\hat{C}'_{id}(5, \Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$, to check
 $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} ((f^3 f^4)^5 (\text{fn } y \Rightarrow y^6)^7)^8$ suffices ([app]) to verify
 $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} (f^3 f^4)^5$ and $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} (\text{fn } y \Rightarrow y^6)^7$ and $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_8^{ce_3} x^1$ iff $\hat{\rho}_{id}(x, 8) \subseteq \hat{C}'_{id}(8, \Lambda)$
 because $\hat{C}'_{id}(7, \Lambda) \subseteq \hat{\rho}'_{id}(x, 8)$ and $\hat{C}'_{id}(1, 8) \subseteq \hat{C}'_{id}(8, \Lambda)$

$ce_3 = ce_0[x \mapsto 8]$

[app] at label 5: Since $\hat{C}'_{id}(3, \Lambda) = \{(\text{fn } x \Rightarrow x^1, ce_0)\}$, to check
 $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} (f^3 f^4)^5$ suffices ([app]) to verify
 $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} f^3$ and $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_{\Lambda}^{ce_1} f^4$ and $(\hat{C}'_{id}, \hat{\rho}'_{id}) \models_5^{ce_2} x^1$
 because $\hat{C}'_{id}(4, \Lambda) \subseteq \hat{\rho}'_{id}(x, 5)$ and $\hat{C}'_{id}(1, 5) \subseteq \hat{C}'_{id}(5, \Lambda)$

$ce_2 = ce_0[x \mapsto 5]$

only $\text{fn } y \Rightarrow y^6$ can flow to $\hat{C}'_{id}(9, \Lambda)$

Correctness and implementation of the uniform k -CFA

Soundness

The analysis is sound w.r.t. the operational semantics

Implementation

The analysis can be implemented via constraint solving / fixpoint iteration

Trade-off

- Higher $k \Rightarrow$ more precision
- But exponential worst-case complexity

Complexity of uniform k -CFA

- even for $k = 1$: exponential worst case complexity

Why? (intuition)

- Program size: n , number of variables: p

exponential blow up comes from the number of abstract closure environments:
each variable can be associated with one term

- Contexts:

- Δ has $O(n)$ elements $\Rightarrow$ $O(n \cdot p)$ different pairs (x, δ) and $O(n \cdot n) = O(n^2)$ different pairs (ℓ, δ)

- Abstract states:

- $(\hat{C}, \hat{\rho})$ can be seen as an $O(n^2)$ tuple of values from $\widehat{\mathbf{Val}}$

closures

- Values:

- $\widehat{\mathbf{Val}}$ itself is a powerset of pairs of the form $(t, ce) \Rightarrow O(n \cdot n^p)$ such pairs,

- Hence $\widehat{\mathbf{Val}}$ has height $O(n \cdot n^p)$

- Since $p = O(n)$, we have the exponential worst case complexity

Example

Suppose to have

- **Var** = $\{x, y\}$
- $\Delta = \{a, b, c\}$ so $|\Delta| = 3$
- For every context ce we need $ce(x)$ and $ce(y)$, so we have **9** contexts with $ce(x)$ and $ce(y)$ ranging over $\{a, b, c\}$
- If we have **4** functions f_i , we have **36** abstract closures

With two variables, each closure environment is a pair of contexts

With p variables, it becomes a p -tuple of contexts, yielding n^p possible environments

Complexity of 0-CFA

- Here for $k = 0$

Why? (intuition)

- Program size: n , number of variables: p
- No contexts:
 - Δ is a singleton ($O(1)$) $\Rightarrow$ $O(1 \cdot p)$ different pairs (x, δ) and $O(1 \cdot n)$ different pairs (ℓ, δ)
- Abstract states:
 - $(\hat{C}, \hat{\rho})$ can be seen as an $O(p + n)$ tuple of values from $\widehat{\mathbf{Val}}$
- Values:
 - $\widehat{\mathbf{Val}}$ (powerset of pairs of the form (t, ce)) $\Rightarrow$ but here only $(t, ce_0) \Rightarrow O(n)$
 - $\widehat{\mathbf{Val}}$ has height $O(n)$ and we have the polynomial complexity we already saw for non contextual analyses

Possible improvements of complexity

Problem:

- In k -CFA, $\widehat{\mathbf{Val}} = \wp(\mathbf{Val} \times \mathbf{Cenv})$
- Context environments cause exponential blow-up

Idea 1: reduce lattice height

- Using abstraction to limit $\widehat{\mathbf{Val}}$
- Standard abstract interpretation techniques

Possible improvements of complexity

Idea 2: simplify contexts $\widehat{\mathbf{Val}} = \wp(\mathbf{Val} \times \mathbf{Cenv})$

Replace context environments with contexts only

$$\widehat{\mathbf{Val}} = \wp(\mathbf{Term} \times \Delta)$$

closures = function +
full environment

closures = function +
call string

- No full environment
- Only call strings δ

Effect:

- Lattice of polynomial height
- No combinations of variables inside ce
- Number of abstract values drastically reduced
- Less precision, but much cheaper

Program Analysis 2025-26

Lecture #22

Control Flow Analysis



UNIVERSITÀ
DI PISA

Cartesian Product Algorithm (CPA)

FUN: syntax

term

$t ::= \dots$

| $\text{fn } x_1, \dots, x_m \Rightarrow e_b$ // function definition
(with m arguments)

$e ::= t^\ell$ | $e_0(e_1, \dots, e_m)$ // function application
(with m arguments)

| $\dots$

- **well-formedness condition:** all function abstractions are closed, i.e.,
 $\text{FV}(\text{fn } x_1, \dots, x_m \Rightarrow e_0) = \emptyset$
- we do not treat recursive functions

The Cartesian Product Algorithm: adding context

We extend the analysis to take **context** into account

In this variant, **contexts** are tuples of argument values

$$\begin{aligned}\delta &\in \Delta = \mathbf{Term}^m \\ \hat{v} &\in \widehat{\mathbf{Val}} = \wp(\mathbf{Term}) \\ \hat{\rho} &\in \widehat{\mathbf{Cenv}} = (\mathbf{Var} \times \Delta) \rightarrow \widehat{\mathbf{Val}} \\ \hat{C} &\in \widehat{\mathbf{Cache}} = (\mathbf{Lab} \times \Delta) \rightarrow \widehat{\mathbf{Val}}\end{aligned}$$

The resulting analysis is a refinement of 0-CFA that partitions the flow of values to expressions and variables according to the actual argument types in their dynamic calling context

Rephrasing the O-CFA analysis: [fn]

$$[fn] (\hat{C}, \hat{\rho}) \models (fn\ x_1, \dots, x_m \Rightarrow e_b)^\ell \text{ iff } \{fn\ x_1, \dots, x_m \Rightarrow e_b\} \subseteq \hat{C}(\ell)$$

Example

$$e \triangleq (fn\ x\ y \Rightarrow (x^1 + y^2)^3)$$

$$\hat{C}(3) \supseteq \{(fn\ x\ y \Rightarrow x^1 + y^2)\} \dots$$

Rephrasing the O-CFA analysis: [app]

$$[app] (\hat{C}, \hat{\rho}) \models (t_0^{\ell_0}(t_1^{\ell_1}, \dots, t_m^{\ell_m}))^\ell \text{ iff } (\hat{C}, \hat{\rho}) \models t_0^{\ell_0} \wedge$$

$$(\hat{C}, \hat{\rho}) \models t_1^{\ell_1} \wedge \dots \wedge (\hat{C}, \hat{\rho}) \models t_m^{\ell_m}$$

$$(\forall (\text{fn } x_1, \dots, x_m \Rightarrow t_b^{\ell_b}) \in \hat{C}(\ell_0) :$$

each argument
flows to its
corresponding
parameter

$$\hat{C}(\ell_1) \times \dots \times \hat{C}(\ell_m) \subseteq \hat{\rho}(x_1) \times \dots \times \hat{\rho}(x_m) \wedge$$

$$(\hat{C}, \hat{\rho}) \models t_b^{\ell_b} \wedge$$

$$\hat{C}(\ell_b) \subseteq \hat{C}(\ell) \wedge \dots$$

In the case where no $\hat{C}(\ell_i)$ is empty
 $\hat{C}(\ell_1) \subseteq \hat{\rho}(x_1) \wedge \dots \wedge \hat{C}(\ell_m) \subseteq \hat{\rho}(x_m)$

Example

$e \triangleq (\text{f}^3(4^4, 6^5))^6$ supposing that $\hat{C}(3) \supseteq \{(\text{fn } x \Rightarrow x^1 + y^2)\}$ then
 $\hat{C}(4) \subseteq \hat{\rho}(1)$ and $\hat{C}(5) \subseteq \hat{\rho}(2)$

Adding context: [var], [fn]

tuple of argument values

$$[var] (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} x^{\ell} \text{ iff } \hat{\rho}(x, \delta) \subseteq \hat{C}(\ell, \delta)$$

$$[fn] (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} (\text{fn } x_1, \dots, x_m \Rightarrow e_b)^{\ell} \text{ iff } \{\text{fn } x_1, \dots, x_m \Rightarrow e_b\} \subseteq \hat{C}(\ell, \delta)$$

Adding context: [app]

$$[app] (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} (t_0^{\ell_0}(t_1^{\ell_1}, \dots, t_m^{\ell_m}))^{\ell} \text{ iff } (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_0^{\ell_0} \wedge$$

we distinguish calls
by what they receive

$$(\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_1^{\ell_1} \wedge \dots \wedge (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_m^{\ell_m} \wedge$$

$$(\forall (\text{fn } x_1, \dots, x_m \Rightarrow t_b^{\ell_b}) \in \hat{C}(\ell_0, \delta) :$$

$$\forall \delta_b \in \hat{C}(\ell_1, \delta) \times \dots \times \hat{C}(\ell_m, \delta) :$$

$$\{\delta_b\} \subseteq \hat{\rho}(x_1) \times \dots \times \hat{\rho}(x_m) \wedge$$

$$(\hat{C}, \hat{\rho}) \models_{CPA}^{\delta_b} t_b^{\ell_b} \wedge$$

$$\hat{C}(\ell_b, \delta_b) \subseteq \hat{C}(\ell, \delta)) \wedge \dots$$

we analyse the body
separately for each δ_b
no data merging

The Cartesian Product Algorithm derives its name from the cartesian product $\hat{C}(\ell_1, \delta) \times \dots \times \hat{C}(\ell_m, \delta)$ over which δ_b ranges

Observation

- The function body is analysed separately for each possible tuple of arguments
- No merging across different tuples
- This creates redundancy: it leads to many re-analyses of the same body: we need **memoisation** so that the bodies are only analysed once
- We memoise analyses of each $(\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_b^{\ell_b}$ as *templates* (with entries like $t_b^{\ell_b}$ associated with $x \mapsto f, y \mapsto a$) and maintain them in a **global pool**
- when a template is created it is only added to the pool if it is not already present (template reuse)

Implementation

- The analysis can be implemented in a straightforward manner because the product grows monotonically:
- $\hat{C}(\ell_1, \delta) \times \cdots \times \hat{C}(\ell_m, \delta)$ will increase each time one of the $\hat{C}(\ell_i, \delta)$ increases (assuming that none is empty)

Exercise

If we remove the **well-formedness condition**, i.e., function abstractions are not closed, i.e., $FV(\text{fn } x_1, \dots, x_m \Rightarrow e_0) \neq \emptyset$, we need to extend the analysis,

- treating functions as closure abstractions:, and
- using new clauses

$$(\hat{C}, \hat{\rho}) \models_{\delta}^{ce} e$$

where

- $\delta \in \Delta = \text{Term}^m$
- $ce \in \text{CEnv} = \text{Var} \rightarrow \Delta$

Adding context: [app]

$$\begin{aligned}
 [app] \quad (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} (t_0^{\ell_0}(t_1^{\ell_1}, \dots, t_m^{\ell_m}))^{\ell} \text{ iff } & (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_0^{\ell_0} \wedge \\
 & (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_1^{\ell_1} \wedge \dots \wedge (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta} t_m^{\ell_m} \\
 & (\forall (\text{fn } x_1, \dots, x_m \Rightarrow t_b^{\ell_b}) \in \hat{C}(\ell_0, \delta) : \\
 & \quad \forall \delta_b \in \hat{C}(\ell_1, \delta) \times \dots \times \hat{C}(\ell_m, \delta) : \\
 & \quad \quad \{\delta_b\} \subseteq \hat{\rho}(x_1) \times \dots \times \hat{\rho}(x_m) \wedge \\
 & \quad \quad (\hat{C}, \hat{\rho}) \models_{CPA}^{\delta_b} t_b^{\ell_b} \wedge \\
 & \quad \quad \hat{C}(\ell_b, \delta_b) \subseteq \hat{C}(\ell, \delta)) \wedge \dots
 \end{aligned}$$

we analyse the body
for each δ_b

The Cartesian Product Algorithm derives its name from the cartesian product $\hat{C}(\ell_1, \delta) \times \dots \times \hat{C}(\ell_m, \delta)$ over which δ_b ranges

Clause for function application

$$\begin{aligned}
 [app] \ (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} (t_1^{\ell_1} t_2^{\ell_2})^{\ell} \text{ iff } & (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_2^{\ell_2} \wedge \\
 & (\forall (\text{fn } x \Rightarrow t_0^{\ell_0}, ce_0) \in \hat{C}(\ell_1, \delta) : \\
 & (\hat{C}, \hat{\rho}) \models_{\delta_0}^{ce_0} t_0^{\ell_0} \wedge \\
 & \hat{C}(\ell_2, \delta) \subseteq \hat{\rho}(x, \delta) \wedge \hat{C}(\ell_0, \delta) \subseteq \hat{C}(\ell, \delta))
 \end{aligned}$$

body analysed in the updated context δ_0 using the updated closure environment ce_0

where $\delta_0 = [\delta, \ell]_k$ and $ce'_0 = ce_0[x \mapsto \delta_0]$

x is now bound in the new calling context δ_0 but using the definition environment ce_0

Adding context: [app]

$$[app] (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} (t_0^{\ell_0}(t_1^{\ell_1}, \dots, t_m^{\ell_m}))^{\ell} \text{ iff } (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_0^{\ell_0} \wedge$$

$$(\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_1^{\ell_1} \wedge \dots \wedge (\hat{C}, \hat{\rho}) \models_{\delta}^{ce} t_m^{\ell_m}$$

$$(\forall (\text{fn } x_1, \dots, x_m \Rightarrow t_b^{\ell_b}, ce_0) \in \hat{C}(\ell_0, \delta) :$$

$$\forall \delta_b \in \hat{C}(\ell_1, \delta) \times \dots \times \hat{C}(\ell_m, \delta) :$$

$$\{\delta_b\} \subseteq \hat{\rho}(x_1) \times \dots \times \hat{\rho}(x_m) \wedge$$

$$(\hat{C}, \hat{\rho}) \models_{\delta_b}^{ce'_0} t_b^{\ell_b} \wedge$$

$$\hat{C}(\ell_b, \delta_b) \subseteq \hat{C}(\ell, \delta) \wedge \dots$$

where

$$ce'_0 = ce_0[x_1 \mapsto \delta_b \downarrow_1, \dots, x_n \mapsto \delta_b \downarrow_n]$$

with $\downarrow_i$ the projection on the i-th component

The evolution of CFA for FUN

0-CFA

Context-insensitive and flow-insensitive flow analysis (merges all calls)

Polynomial-time higher-order analysis

Syntax-directed CFA

Analyze each body once at definition site, but analyzes unreachable code

Constraint-based formulation

Context-sensitive CFA (k-CFA style)

Track calling contexts and closure environments

Distinguishes different calls

Cartesian Product Algorithm

Analyze bodies for each argument tuple

Larger abstract state space

For all CFAs, semantics are preserved, and a (least) solution exists.

Moreover, an implementation can be provided

CFA for FUN: conclusion

Control-flow analysis for higher-order languages is a tradeoff between:

precision vs computational cost

More context $\Rightarrow$ fewer spurious flows $\Rightarrow$ higher complexity

The key challenge of CFA is balancing:

- soundness
- precision
- scalability